

信息时序约束下的世界杯收视预测、赛程协同与动态资源优化

摘 要

针对大型足球赛事中收视预测、赛程安排、动态资源配置和实际赛程评价相互依赖的问题,本文建立遵守信息时点的顺序决策链。首先,仅使用赛前可得信息构造交换球队顺序后不变的对称特征,并在统一特征函数中规范化赛事名称。扩窗时间验证显示,Elastic Net 的 MSE、RMSE 和 MAE 分别为 188.9787、13.7268 和 10.6345,优于均值基线和 Extra Trees;规范化后 72 场未来比赛的未见赛事类别计数为 0。其次,将 72 场收视预测输入时段 MILP 和固定时段后的场馆 MILP,得到综合目标值 0.272453;所有球队均获得 2 场黄金时段和 1 场大容量场馆比赛,同组相邻轮次最小间隔为 63 小时。再次,固定第三轮赛程,在第三轮开始前的统一信息截面上同时模拟 24 场比赛 20000 次,对完全同分情形分配等比例晋级权重。动态总净值为 6.583849,较静态方案的 6.578368 提高 0.083318%;五个随机种子下的改善率位于 0.080406% 至 0.084923%,说明该边际改善对 Monte Carlo 随机种子稳定。最后,以 2022 年世界杯为参照,分层比较直接结构事实、映射辅助指标和完整题内代理指标。最近邻和第二近邻映射下实际赛程代理综合值分别为 0.196995 和 0.210331,均低于优化赛程的 0.272453,但举办区域尺度和赛事规模会显著影响旅行、休息与场馆利用等结构指标,代理综合值不能替代对真实赛事的全面评价。

关键词: 收视预测;混合整数规划;联合泊松模拟;动态资源配置;赛程评价

1 问题重述

赛事组织者需要在比赛尚未开始时估计转播需求，据此安排 72 场小组赛的场馆和开球时段；当前两轮结束后，部分球队的晋级形势、市场反馈和风险水平发生变化，又需要在保持第三轮赛程不变的前提下调整转播、安保、交通和票价。赛后还需将优化方案置于真实赛事背景中评价，判断收益、公平和执行代价之间的取舍。

四个问题并非彼此独立。问题一产生的 72 场转播观看人数预测进入问题二的转播价值；问题二产生的第三轮场馆与时段是问题三固定输入；问题四则沿用问题二的指标定义和权重。若混淆信息时点，例如用测试标签更新后续测试特征，或让某场第三轮结果影响另一场第三轮比赛，就会使方案获得现实中不可用的信息优势。因此，本文将信息可得性与硬约束同时纳入建模。

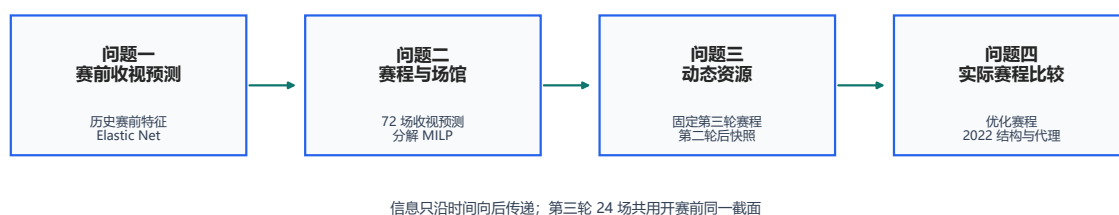


图 1 四问顺序决策与信息流。箭头表示计算输出的传递，不表示允许使用未来信息。

图 1 给出全文证据链：问题一输出进入问题二的转播价值，问题二固定第三轮时空安排，问题三只在该安排上重新分配资源，问题四再将优化结果放回实际赛事中比较。

2 数据、假设与符号

2.1 数据及信息时点

题目附件提供历史比赛、球队属性、72 场对阵及赛前预测、场馆、时段、距离矩阵、安保要求、动态资源限制和前两轮反馈 [1]。问题四另使用固定版本的 2022 年世界杯赛程、赛前 Elo 和实体映射。各阶段允许使用的信息见表 1。

表 1 信息时点与禁用信息

阶段	可用信息	禁用信息
问题一训练	历史比赛的赛前字段及训练标签	同场进球、射门、控球、实际上座等赛后字段作为特征
问题一测试与 72 场预测	统一赛前特征	测试标签、同场赛果和实际观众
问题二	问题一预测、72 场赛前参数、场馆与时段信息	前两轮真实赛果
问题三静态方案	赛前预计观众、预计收视、吸引力和赛前风险	前两轮更新后的晋级概率和风险
问题三动态方案	前两轮结束后的统一信息截面	任一第三轮比赛结果
问题四	2022 年赛程事实、赛前 Elo 和题内映射参数	无来源的真实商业、票务或安保参数

所有时间先转换为 UTC 计算冲突和休息约束，再按场馆时区计算当地日期。问题一测试输出单位为百万人，72 场预测输出单位为人。问题二和问题三的货币量按题目字段分别使用美元或百万美元，表格中均注明口径。随机过程采用固定种子：问题一随机模型、问题三主模拟与模拟次数检查均为 20260816，问题三五种子为 20260816–20260820，问题四 2000 组权重扰动为 20260816。

2.2 基本假设

- 1. 附件中标注为赛前参数的评分和概率在相应决策时点可获得。
- 2. 第三轮 24 场共享第三轮开始前的同一信息截面；给定更新状态后，各场进球数按题面泊松模型生成。
- 3. 场馆容量、安保能力、资源上限和成本系数在研究期内不变。
- 4. 题目给定的候选时段、资源等级和价格边界构成完整可行动作集合。
- 5. 问题四无法直接观测的商业与风险指标仅通过题内映射估计，不视为 2022 年真实经营数据。

2.3 主要符号

主要符号见表 2。

表 2 主要符号及含义

符号	含义
i、t、v、s	比赛、球队、场馆、时段索引
x _{is}	比赛 i 是否选择时段 s
z _{iv}	比赛 i 是否选择场馆 v
V _{hat_i}	问题一预测的转播观看人数
T、B、U、H	票务、转播、悬念、吸引力指标
C、D、F、R	成本、旅行、公平、风险指标
S _t 、h _t	前两轮后的球队状态和伤病影响
p _t	球队晋级概率
delta _i	第三轮比赛票价调整比例
Z ₂ 、Z ₃	问题二和问题三综合目标

3 转播观看人数预测

3.1 建模思路

历史数据只有 560 个训练样本，而特征包含连续变量和类别变量；测试集又处于训练时间之后。随机划分会把较晚比赛的信息混入训练，因此本文按日期采用扩窗验证。另一方面，team_a 和 team_b 只是记录顺序，并非主客队。若直接保留双方有序类别或带符号差值，交换记录顺序就可能改变预测。为此，本文使用统一的对称特征生成函数，并在测试和 72 场预测中保持完全相同的口径。

3.2 对称赛前特征

对排名和 Elo，构造双方的均值、最小值、最大值和绝对差；对市值、平均年龄、明星指数、球迷基础、攻防风格、东道主标记和综合实力采用同样变换。胜平负赔率先转换为归一化赛前概率，保留平局概率、双方胜率的最小值、最大值和绝对差，并构造归一化概率熵：

$$E_i^{\text{prob}} = -\frac{1}{\log 3} \sum_{o \in \{W,D,L\}} p_{io} \log p_{io}.$$

其他特征包括中立场标记、月份、星期、赛事类型和归并后的比赛阶段。赛事名称先经确定性函数清理大小写、年份和 FIFA 前缀，将 World Cup、FIFA World Cup 及带年份的同义写法统一为 World Cup；训练、验证、测试和 72 场预测共用这一函数和类别字典。历史训练集中缺少与未来数据统一生成口径的 expected_attendance_base、attractiveness_index、commercial_value_index 和 uncertainty_index，故这些字段不进入回归；悬念只由统一的赛前胜平负概率表示。模型也不使用球队有序类别、同场赛果和任何测试期滚动标签。

观看人数具有右偏和量级差异。设原始观看人数 y_i 的单位为人，训练目标定义为：

$$q_i = \log \left(1 + \frac{y_i}{10^6} \right).$$

模型预测后通过

$$\hat{y}_i^{\text{million}} = \exp(\hat{q}_i) - 1$$

还原为百万人，再按输出模板决定是否乘以一百万。该变换减弱少数大观看量比赛对拟合的支配，同时保持逆变换结果非负。

3.3 模型与时间验证

本文比较均值基线、Elastic Net 和 Extra Trees[2, 3]。数值特征先以训练子窗中位数填补并标准化，类别特征用众数填补和独热编码。Elastic Net 的目标为：

$$\min_{\beta} \frac{1}{2n} \|\mathbf{q} - X\beta\|_2^2 + \alpha \left[\rho \|\beta\|_1 + \frac{1-\rho}{2} \|\beta\|_2^2 \right],$$

其中程序取 $\alpha=0.02$ 、 $\rho=0.10$ 。扩窗验证依次使用前 60%、70% 和 80% 的时间有序样本训练，并在紧随其后的时间窗验证。误差均在逆变换后的百万人尺度计算：

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

$$\text{RMSE} = \sqrt{\text{MSE}}, \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

表 3 三种模型的扩窗验证结果

模型	MSE	RMSE	MAE
Elastic Net	188.978683	13.726811	10.634511
Extra Trees	227.022897	15.033564	12.094744
均值基线	531.378075	22.996821	18.398928

表 3 中每个指标均先在各折独立计算，再对三折作算术平均，因此汇总 RMSE 不等于汇总 MSE 的平方根。Elastic Net 的三项平均误差均最低，因此选择其作为最终模型。选择依据是时间外验证误差与复现稳定性，而不是预设线性模型一定优于树模型。

为检查平均误差是否掩盖时间段波动，表 4 进一步给出 Elastic Net 的三个扩窗。每个验证窗

均含 56 场比赛，且严格位于对应训练窗之后。

表 4 Elastic Net 的三折扩窗验证结果，误差单位为百万人

折	训练数	验证数	训练截止	验证区间	MSE	RMSE	MAE
1	336	56	2022-01-13	2022-01-18–2022-09-22	201.0861	14.1805	11.2902
2	392	56	2022-09-22	2022-09-24–2023-04-29	205.1182	14.3219	10.9816
3	448	56	2023-04-29	2023-04-30–2024-01-07	160.7317	12.6780	9.6318

前两折 RMSE 只相差 0.1415，第三折下降至 12.6780；因此误差没有随预测时间后移而恶化，但也不能将三折描述为完全不变。MSE 的折间标准差为 24.5456，提示赛事组成和时间窗仍会影响误差。

3.4 系数、验证预测与残差

数值特征经标准化，而类别特征为 0–1 独热指示量，因此两类系数不宜作严格的绝对大小排名。图 2(b) 只用于识别方向和特征族：World Cup 指示量系数为 0.128338；球迷基础均值、最小值和最大值分别为 0.049762、0.038872 和 0.014968；综合实力绝对差与市值绝对差分别为 -0.018812 和 -0.009680 。由于均值、最小、最大和绝对差之间存在线性关联，本文只解释“球迷基础整体正向”、“实力和市值差异整体负向”，不把任一单列系数解释为独立因果效应。

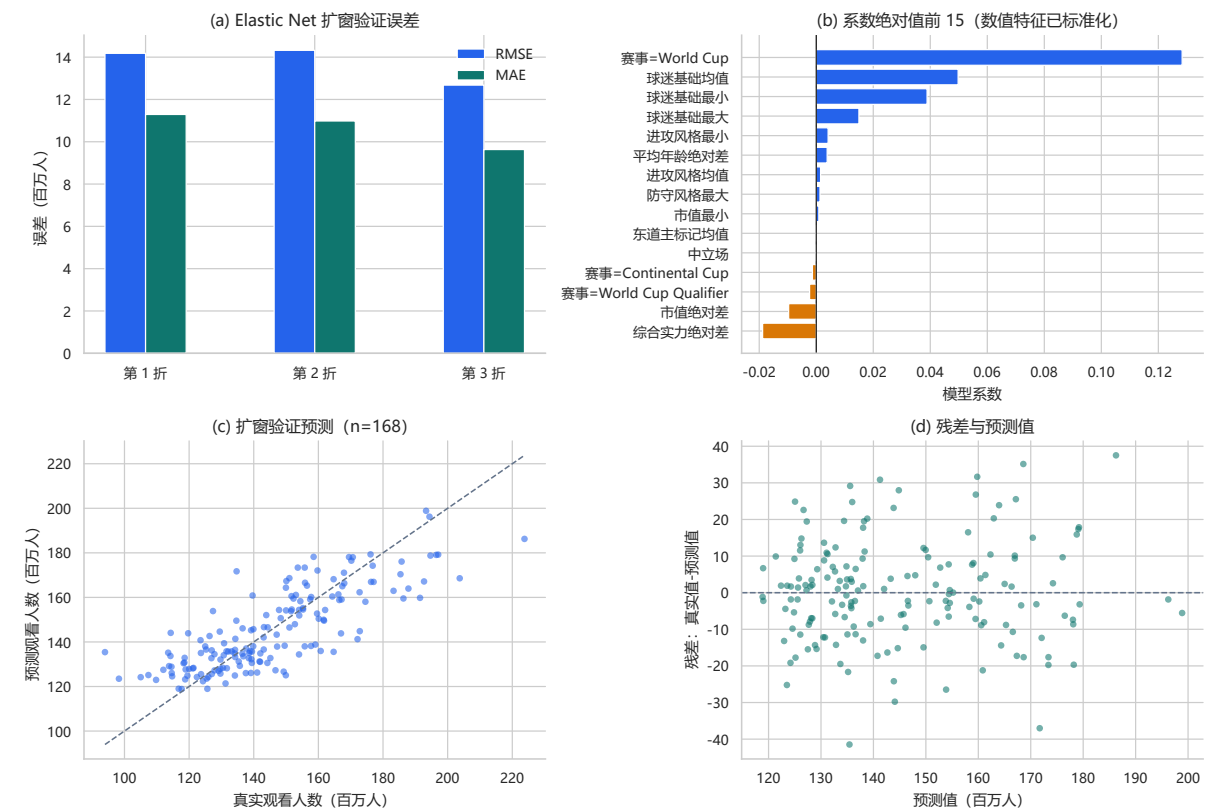


图 2 问题一的扩窗误差、模型系数和验证残差。散点只使用 168 个训练集内时间验证样本，不使用隐藏测试标签。

图 2(c) 显示预测与真实值保持明显正关联，168 个时间外验证样本的相关系数为 0.801153，平均残差（真实值减预测值）为 0.292706 百万人。但最低真实值十分位平均被高估 16.852850 百万人，最高十分位平均被低估 17.624260 百万人。这是对数目标、线性收缩和 Elastic Net 惩罚共同导致的“向中心收缩”：它提高时间外稳定性，但牺牲对极端热门或冷门比赛的幅度拟合。因此后续

排程中更适合将该预测视为统一口径下的相对转播需求，而非极端场次的无偏点预测。

3.5 预测结果与解释

训练集拟合、测试集预测和 72 场预测的分布见表 5。

表 5 问题一预测分布，单位：百万人

数据	均值	标准差	最小值	最大值
训练集拟合	144.811868	17.522965	114.222923	201.907400
测试集预测	144.558531	15.800763	114.044606	180.352371
72 场预测	161.599758	16.777979	134.524563	204.108108

未来 72 场均值比训练集拟合均值高 16.787890 百万人，约为 11.59%。对线性预测器的均值变化分解显示，未来样本全部处于世界杯类别，World Cup 指示量对对数预测的平均贡献为 0.120546，是均值上升的主要来源。综合实力绝对差、球迷基础最小值和市值绝对差分别贡献 -0.005131 、 -0.003977 和 -0.001919 ，部分抵消了赛事类别的正向作用。这说明未来预测的提高主要来自世界杯赛事层级，而强弱分明且较弱一方球迷基础偏低的小组赛组合会拉低具体场次的预测。

规范化前，72 场默认赛事名称为 FIFA World Cup；规范化后均为训练集已见的 World Cup，测试集和未来 72 场的未见赛事类别样本数均为 0。该处理位于统一特征函数，而非对未来 CSV 的事后修正。其他连续特征均未超出训练均值正负 4 个标准差。

交换每场双方及对应赛前字段后，最大预测差为 0，证明记录顺序不会影响结果。最终文件包含 140 条测试预测和 72 条小组赛预测，ID 与模板逐项一致，数值均有限且非负。

4 场馆与开球时段协同优化

4.1 指标与变量

问题二需要在 80 个候选时段和 16 个场馆中安排 72 场比赛。为与实际分解求解一致，定义：

$$x_{is} = \begin{cases} 1, & \text{比赛 } i \text{ 选择时段 } s, \\ 0, & \text{否则,} \end{cases}$$

$$z_{iv} = \begin{cases} 1, & \text{比赛 } i \text{ 选择场馆 } v, \\ 0, & \text{否则.} \end{cases}$$

场馆启用变量记作 y_v 。由于每个场馆的最小承办场次均大于 0，本数据中全部 $y_v=1$ ，启用成本对所有可行方案相同，不作为实际自由决策变量。

比赛 i 在场馆 v 的预计观众和票务价值为：

$$N_{iv} = \min(N_i^{\text{base}}, K_v),$$

$$T_{iv} = P_i^{\text{base}} N_{iv}.$$

问题一输出进入时段转播价值：

$$B_{is} = \hat{V}_i u_i^{\text{broadcast}} g_s^{\text{prime}} w_i^{\text{sponsor}}.$$

赛事组织成本以百万美元计：

$$C = \sum_v c_v^{\text{setup}} y_v + \sum_{i,v} z_{iv} \left(c_v^{\text{operation}} + 0.1 L_i^{\text{req}} c_v^{\text{security}} \right).$$

执行风险为：

$$R_{iv} = 0.5 r_v^{\text{climate}} + 0.3 \frac{N_{iv}}{K_v} + 0.2 \frac{L_i^{\text{req}}}{L_v^{\text{security}}}.$$

悬念 U 直接取附件 `uncertainty_index` 的赛程平均，吸引力 H 取 `attractiveness_index/100` 的赛程平均。由于 72 场对阵固定，二者在排程优化中是常数项，但仍按题目权重进入最终综合评价。

4.2 旅行负担代理与公平性

距离、旅行时间和时区差先分别在全部合法比赛-场馆候选上作 Min-Max 标准化，再合成：

$$D_{iv} = 0.5 d_{iv}^{\text{km}} + 0.3 d_{iv}^{\text{time}} + 0.2 d_{iv}^{\text{tz}}.$$

第一轮使用球队代表地至当前候选场馆的三个分项；第二、三轮使用该队上一轮全部安保合格候选场馆至当前候选场馆的分项均值；一场比赛再对两队取平均。因此 D_{iv} 是候选前序场馆下的期望旅行负担代理，不是最终排程中上一轮实际场馆至当前场馆的真实路线。该定义与题面候选口径一致，也使场馆选择能够在固定时段后独立求解，但没有通过场馆转移变量完全耦合相邻轮次。

黄金时段由 `global_prime_score` 降序、`slot_id` 升序稳定处理并列后严格选取前 20 个；大容量场馆为容量前 4 名。设球队黄金时段次数和大容量场馆次数的极差分别为 `Delta_prime` 与 `Delta_large`，公平性为：

$$F = \frac{\Delta_{\text{prime}} + \Delta_{\text{large}}}{6}.$$

其中黄金时段极差不超过 2 是硬约束，大容量场馆极差主要通过目标惩罚控制。二者最终均为 0 是优化结果，而非均被预先固定为 0。

4.3 标准化与综合目标

对票务、转播、成本、旅行和风险，在全部合法候选组合上确定固定边界。总量指标 T、B、C 使用 72 倍单场候选边界；比例型 U、H、F 使用区间 0 至 1；旅行负担已位于 0 至 1。标准化为：

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}.$$

所有方案共享同一边界，不能使用当前方案自身极值。综合目标为：

$$Z_2 = 0.25 T' + 0.25 B' + 0.15 U' + 0.10 H' - 0.08 C' - 0.07 D' - 0.06 F' - 0.04 R'.$$

4.4 硬约束

每场比赛恰好选择一个时段和一个场馆：

$$\sum_s x_{is} = 1, \quad \sum_v z_{iv} = 1.$$

设 c_s^B 为时段转播容量， τ_s 为 UTC 小时数， I_t^r 为球队 t 在第 r 轮的唯一比赛。同一候选 UTC

时刻的转播容量和同队休息约束为

$$\sum_i x_{is} \leq c_s^B, \quad \forall s,$$

$$\sum_s \tau_s x_{js} - \sum_s \tau_s x_{is} \geq 60, \quad i \in \mathcal{I}_t^r, j \in \mathcal{I}_t^{r+1}.$$

题目还要求同组轮次整体分离。对小组 g 的相邻轮次, 有:

$$\min_{i \in \mathcal{I}_{g,r+1}} t_i - \max_{j \in \mathcal{I}_{g,r}} t_j \geq 60 \text{ h}, \quad r \in \{1, 2\}.$$

时段 MILP 对同组上一轮与下一轮的每一对比赛加入同样的时间差约束, 从而显式实现上述最小值减最大值口径。设 $\mathcal{S}_d$ 为日期 d 的时段, $\mathcal{I}_{3,H}$ 为第三轮且最低安保需求不低于 3 的比赛, 则

$$\sum_{i \in \mathcal{I}_{3,H}} \sum_{s \in \mathcal{S}_d} x_{is} \leq c_d^H, \quad \forall d.$$

以 n_t^P 表示球队 t 的黄金时段次数, 用辅助变量 $p^{\max}, p^{\min}$ 线性化极差:

$$n_t^P \leq p^{\max}, \quad n_t^P \geq p^{\min}, \quad p^{\max} - p^{\min} \leq 2.$$

固定时段后, 记 $\bar{s}(i)$ 为比赛 i 已选时段, $a_{ivd} = 1$ 表示该时段在场馆 v 的当地日为 d 。场馆子问题的主要约束可紧凑写为

$$z_{iv} = 0 \quad (L_i^{\text{req}} > L_v^{\text{security}}),$$

$$\sum_{i: \bar{s}(i)=u} z_{iv} \leq 1, \quad \sum_i a_{ivd} z_{iv} \leq m_v^{\text{day}},$$

$$m_v^{\min} \leq \sum_i z_{iv} \leq m_v^{\max}.$$

大容量场馆次数 n_t^L 同样以 $l^{\max}, l^{\min}$ 线性化: $n_t^L \leq l^{\max}, n_t^L \geq l^{\min}$ 。这些式子与程序中的容量、安保、当地日和极差约束一一对应。

4.5 两阶段分解 MILP

完整三维指派规模较大, 且时段价值与场馆价值在题面指标中基本可分。本文采用两阶段方法:

1. 时段 MILP 决定 $\mathbf{x}_{is}$, 处理每场唯一、转播容量、球队休息、同组轮次边界、第三轮高安保日容量和黄金时段公平。实际变动目标是 $0.25\bar{B}' - 0.01(p^{\max} - p^{\min})$; U, H 因对阵固定而是常数。
2. 固定时段后, 场馆 MILP 决定 $\mathbf{z}_{iv}$, 处理场馆安保、场馆-UTC 冲突、当地日容量、总场次上下限和大容量场馆公平。实际变动目标是 $0.25\bar{T}' - 0.08\bar{C}' - 0.07\bar{D}' - 0.04\bar{R}' - 0.01(l^{\max} - l^{\min})$ 。

两个子问题通过 SciPy 的 milp 接口调用 HiGHS 混合整数分支割平面求解器 [6, 7]; HiGHS 的线性规划基础可参见文献 [5]。两个子问题均在设定 MIP 容差内求得最优解。表 6 给出实际模型规模与终止信息。该结果只说明两个条件子问题达到设定容差, 不构成原三维联合模型的最优性证明。

表 6 问题二两个分解 MILP 的规模与求解状态

子问题	变量数	约束数	时间/s	MIP gap	终止表述
时段 MILP	5762	461	23.2163	0.00047844	设定容差内最优
条件场馆 MILP	1073	888	0.4908	0.00035709	设定容差内最优

4.6 排程结果

八项指标的原始值、统一边界和目标贡献见表 7。

表 7 问题二指标分解

指标	原始值	Min-Max 边界	标准化值	加权贡献
T	285910470.000	[180224640.000, 462345840.000]	0.374611	0.093653
B	326546865.536	[86445484.184, 613637147.351]	0.455435	0.113859
U	0.461436	[0, 1]	0.461436	0.069215
H	0.639699	[0, 1]	0.639699	0.063970
C	53.055	[38.470, 71.590]	0.440368	-0.035229
D	0.189247	[0, 1]	0.189247	-0.013247
F	0.000000	[0, 1]	0.000000	0.000000
R	0.410364	[0.224709, 0.600402]	0.494167	-0.019767

由此得到：

$$Z_2 = 0.2724533543.$$

转播价值的正向贡献最大，其次为票务、悬念和吸引力；成本与风险是主要负向贡献。公平性惩罚为 0。排程中共有 48 场落入黄金时段，对应 96 个球队参赛席位，恰好可以为 48 支球队各分配 2 场；大容量场馆承办 24 场，对应 48 个球队席位，也恰好可以为每队各分配 1 场。在整除条件提供可均衡结构的基础上，黄金时段硬约束和公平性目标共同将两个极差压至 0。表 8 给出主要硬约束的独立复算结果。

表 8 问题二主要硬约束复核

检查项	结果	检查项	结果
比赛唯一安排	72 场均唯一	场馆-UTC 冲突	0
球队最小相邻休息	63 小时	同组相邻轮次最小边界	63 小时
同组轮次边界违规	0	场馆当地日容量违规	0
场馆总负荷违规	0	转播容量违规	0
安保能力违规	0	第三轮高安保日容量违规	0
黄金时段次数极差	0		

图 3 显示 72 场并未均匀地铺满赛程窗口，而是被转播容量、黄金时段和轮次间隔约束切分成多个波峰。全部同组相邻轮次边界的最小值为 63 小时，但 B 组第一、二轮为 303 小时，L 组对应边界为 231 小时，同队间隔最大为 312 小时。这些长空档的直接原因是目标中只惩罚旅行代理、成本、风险和公平性，没有对超过 60 小时的额外等待时间收费。因此长空档是目标设定允许的结果，不是约束违规，但在组织实务上会增加驻地和赛程紧凑性成本。

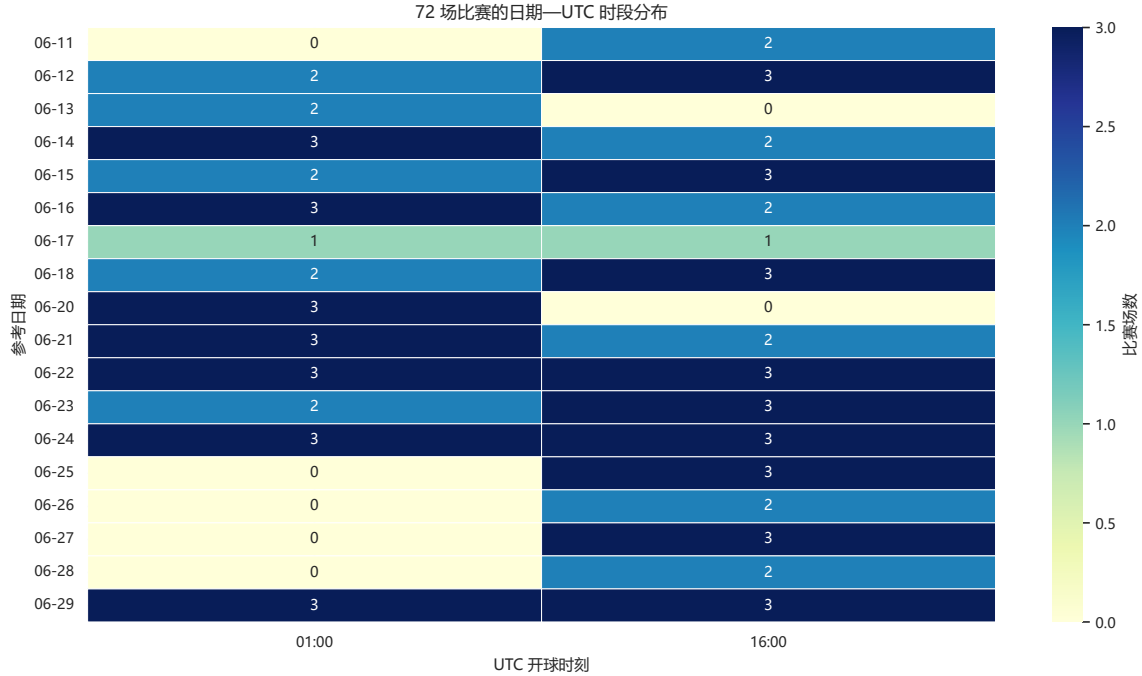


图 3 72 场比赛的日期—UTC 开球时刻热力图。格内数字为该日该 UTC 时刻的比赛数。

图 4 将场馆负荷、休息、目标贡献和轮次边界放在同一证据框架中。

场馆负荷同样显示目标与上下限的共同作用。V01–V05 和 V08 达到 8 场上限，V06 承办 6 场，其余 9 个场馆停在 2 场下限。前一类场馆通常在容量、运营成本、气候风险或安保能力的组合上更有利；后一类由每馆最少 2 场的硬约束保留。例如 V04 容量不大，但运营成本较低且气候风险小；V08 则兼具大容量和 4 级安保能力。因此“8 场与 2 场”不是单一容量排序，而是票务、成本、风险和硬下限的取舍。

4.7 权重扰动与近等价排程

为检查方案是否依赖单一权重组合，保持权重正负方向，分别将 T, B, C, D, F, R 的权重幅值上下扰动 20%，再将全部权重绝对值归一为 1，每个情景重新求解两个 MILP 并独立复核硬约束。 U, H 只由固定对阵决定，在排程中是常数，故不做重求解敏感性。十二个扰动情景全部通过硬约束， F 始终为 0； D 位于 0.187879–0.190562， R 位于 0.403309–0.411664， C 位于 53.015–53.435 百万美元。因此公平、旅行代理、风险和成本的总体结论没有因小范围权重变化而质变。

另一方面，每个情景有 73.6%–87.5% 的比赛更换了完整时段–场馆指派。将这些扰动方案统一代回原始权重评价， Z_2 仅在 0.272323–0.272503 之间变化，跨度约 1.8×10^{-4} 。这不能直接解释为相应权重对所有决策的因果作用：例如 T, D, R 不进入时段子问题，但时段仍可因大量近等价解和 MIP 容差而改变。故安全的解释是：汇总指标和硬约束较稳定，但具体指派并不唯一，组织者拥有多套近等价的可执行备选。

图 5 中大量指派替换与汇总指标的小幅变化共同说明，该问题存在多个指标接近的可行排程。

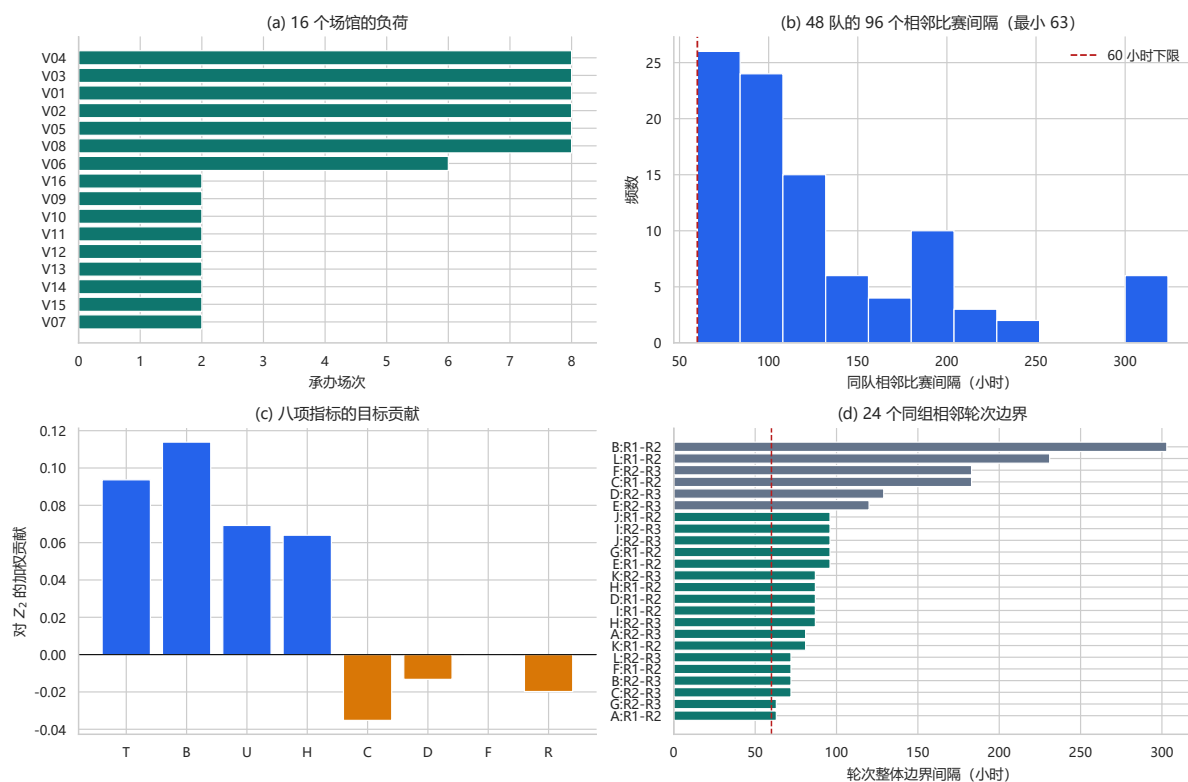


图 4 问题二的场馆负荷、休息间隔、目标贡献与同组轮次边界。红色虚线为 60 小时下限。

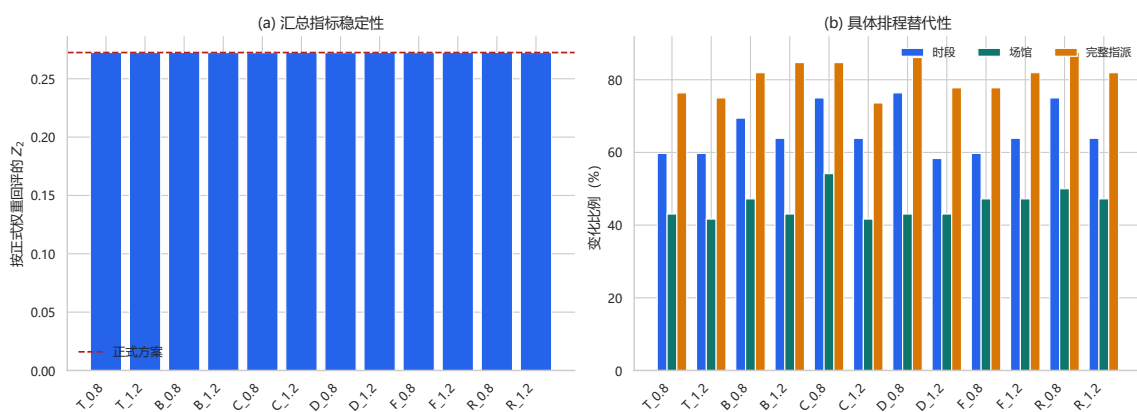


图 5 问题二单项权重 $\pm 20\%$ 重求解。左图为各方案在正式原权重下的统一回评；右图分别给出时段、场馆和完整指派的变化比例。

5 第三轮动态资源优化

5.1 统一信息截面与球队状态

问题三固定问题二确定的 24 场第三轮比赛、场馆和时段，只调整转播优先级、安保等级、交通保障等级和票价。全部参数在第三轮开始前一次性更新；任一第三轮比赛都不能使用其他第三轮比赛结果。

设球队 t 前两轮已赛场数、积分、累计预期进球、累计预期失球和红牌数分别为 G_t 、 P_t 、 xGF_t 、 xGA_t 、 RC_t 。按题面定义状态：

$$S_t = 0.45 \frac{P_t}{3G_t} + 0.35 \left[0.5 + 0.5 \tanh \left(\frac{(xGF_t - xGA_t)/G_t}{1.25} \right) \right] + 0.20 \exp \left(-0.55 \frac{RC_t}{G_t} \right).$$

伤病影响 h_t 取前两轮均值并截断至 0 到 3。第三轮比赛 i 的双方为 a 、 b ，定义：

$$\Delta_i = 0.32(S_a - S_b) - 0.055(h_a - h_b).$$

在赛前基础进球均值上更新：

$$\lambda_{i,a} = \text{clip}(\lambda_{i,a}^0 \exp(\Delta_i - 0.04h_a), 0.15, 4.50),$$

$$\lambda_{i,b} = \text{clip}(\lambda_{i,b}^0 \exp(-\Delta_i - 0.04h_b), 0.15, 4.50).$$

5.2 24 场联合模拟与同分处理

每次模拟同时从 24 组泊松分布生成第三轮比分 [4]，再与前两轮结果合并，统一计算 12 个小组排名和 12 个第三名中的最好 8 队。共模拟 20000 次。球队晋级概率为：

$$p_t = \frac{1}{20000} \sum_{m=1}^{20000} w_{tm}^{\text{adv}}.$$

排名只使用积分、净胜球和总进球。当这些指标完全相同且同分区间跨越晋级截止位置时，将剩余名额平均分给同分球队。例如三队完全相同而只剩两个名额，则三队在该次模拟中的晋级权重均为三分之二。该处理同时用于组内前二、第三名身份和最佳第三名第 8 位，不使用球队编号固定破同分。晋级累计矩阵允许浮点权重，每次模拟的晋级权重和严格为 32，20000 次平均晋级概率总和也为 32。

比赛晋级重要性定义为：

$$Q_i = \frac{4p_a(1 - p_a) + 4p_b(1 - p_b)}{2}.$$

当双方晋级概率接近 0.5 时， Q_i 较高；当双方几乎确定晋级或出局时， Q_i 较低。

5.3 反馈、需求和吸引力

前两轮现场和转播反馈先分别计算实际值与赛前预测值之比，对单场比例截断后取球队均值。比赛双方的反馈采用几何平均，记作 f_i^N 和 f_i^V 。定义：

$$S_i = \frac{S_a + S_b}{2}, \quad H_i = \frac{h_a + h_b}{6}, \quad A_i^0 = \frac{\text{attractiveness_index}_i}{100}.$$

反馈强度为：

$$\text{feedback}_i = \text{clip} \left(\frac{0.5f_i^N + 0.5f_i^V - 0.75}{0.50}, 0, 1 \right).$$

更新吸引力为:

$$A_i = \text{clip} [0.50A_i^0 + 0.25Q_i + 0.12\text{feedback}_i + 0.08S_i + 0.05(1 - H_i), 0, 1],$$

更新现场需求和转播需求为各反馈因子的连乘:

$$\tilde{N}_i = N_i^{\text{pre}} \cdot f_i^N \cdot (0.88 + 0.27Q_i) \cdot (0.94 + 0.12S_i) \cdot (1 - 0.10H_i),$$

$$\tilde{V}_i = V_i^{\text{pre}} \cdot f_i^V \cdot (0.87 + 0.30Q_i) \cdot (0.90 + 0.20A_i) \cdot (1 - 0.06H_i).$$

5.4 无激励风险、默契风险和安保风险

无激励风险为:

$$R_i^{\text{stake}} = 0.5(2p_a - 1)^2 + 0.5(2p_b - 1)^2.$$

在同一组联合模拟中, 分别统计平局、a 胜和 b 胜条件下双方的晋级概率。记:

$$D_i = \min(p_a^D, p_b^D),$$

$$G_i = 0.5 \max(p_a^W - p_a^D, 0) + 0.5 \max(p_b^W - p_b^D, 0),$$

则默契风险为:

$$R_i^{\text{coll}} = \text{clip} [D_i (1 - \text{clip}(G_i, 0, 1)) (1 - 0.35R_i^{\text{stake}}), 0, 1].$$

若某条件样本数为 0, 则以该队无条件晋级概率作为保守替代, 避免除以 0。动态安保占用率必须使用票价和交通动作之前的更新现场需求:

$$O_i = \text{clip} \left(\frac{\tilde{N}_i}{K_i}, 0, 1 \right).$$

安保需求和综合风险为:

$$d_i = \text{clip} \left[0.45d_i^0 + 0.25O_i + 0.15A_i + 0.15 \frac{R_i^{\text{stake}} + R_i^{\text{coll}}}{2}, 0, 1 \right],$$

$$R_i = 0.40R_i^{\text{stake}} + 0.40R_i^{\text{coll}} + 0.20d_i m_{q_i}^R.$$

5.5 资源动作与多重选择 MILP

每场选择转播优先级 b_i 、安保等级 q_i 、交通等级 l_i 和票价调整 delta_i 。安保等级不得低于比赛需求, 也不得超过场馆能力。对给定交通等级, 最终现场需求为:

$$N_i(\delta_i) = \min \left\{ K_i, \tilde{N}_i m_{l_i}^N (1 + \delta_i)^{\varepsilon_i} \right\}.$$

所有票价候选均用容量截断后的完整函数验证：

$$N_i(\delta_i) \geq 0.88N_i(0).$$

候选点包括最大折扣、0、最大涨价、需求保持边界和容量切换边界。票务与转播价值为：

$$TV_i = P_i^0(1 + \delta_i)N_i(\delta_i),$$

$$\hat{V}_i^{\text{upd}} = \tilde{V}_i m_{b_i}^V, \quad BV_i = u_i \hat{V}_i^{\text{upd}}.$$

枚举每场合法动作 $\mathbf{a}$ ，以 $\mathbf{w}_{\mathbf{ia}}$ 表示是否选择该动作。每场恰好选择一个动作，并限制每日高转播、高安保、强化交通和资源预算。为避免与转播观看人数符号冲突，记单场动作的归一化净贡献为

$$J_{ia} = 0.35TV'_{ia} + 0.35BV'_{ia} + 0.10A'_i - 0.10C'_{ia} - 0.10R'_{ia}.$$

动态目标为：

$$\max Z_3 = \sum_{i \in \mathcal{I}_3} \sum_a J_{ia} w_{ia}.$$

所有标准化边界由第三轮全部更新候选动作统一确定。动态方案的更新边界见表 9。

表 9 问题三更新环境的共同归一化边界

指标	下界	上界
票务价值	2981226.448	6558212.829
转播价值	4726317.999	8519872.166
吸引力	0.366896	0.686617
资源成本	3.000000	23.000000
风险	0.079917	0.745848

5.6 静态方案与动态方案

静态方案在赛前使用问题二预计现场观众、问题一预计收视、赛前吸引力、赛前无激励风险 1-uncertainty_index 和默契风险 0，在相同动作集合、每日资源上限和预算下优化。得到的四项动作随后固定。

前两轮结束后，静态方案不得重新选动作，而是在更新环境中按固定的转播、安保、交通和票价重新计算现场需求、票务价值、转播观看人数、转播价值、安保风险、资源成本和净值。动态方案则在更新信息下重新优化。二者最终使用表 9 的同一更新边界评价。

逐场改善率为：

$$I_i = \frac{J_i^{\text{dynamic}} - J_i^{\text{static}}}{|J_i^{\text{static}}|}.$$

静态净值为 0 时留空。摘要与总体结论使用总净值改善率：

$$I_{\text{total}} = \frac{\sum_i J_i^{\text{dynamic}} - \sum_i J_i^{\text{static}}}{|\sum_i J_i^{\text{static}}|}.$$

5.7 结果与随机种子稳定性

主种子 20260816 下动态总净值为 6.5838485986，静态总净值为 6.5783676577，因此：

$$I_{\text{total}} = 0.083318\%.$$

24 场中有 19 场的逐场净值基本不变，4 场正向、1 场负向。动态方案推荐 20 场使用转播优先级 3、4 场使用等级 2；安保等级 1 至 4 的场次数分别为 3、9、10、2；交通等级 1 至 3 的场次数分别为 3、4、17。图 6 表明晋级概率与风险不是单调对应：晋级重要性在概率接近 0.5 时高，而双方已确定晋级时无激励风险反而高。

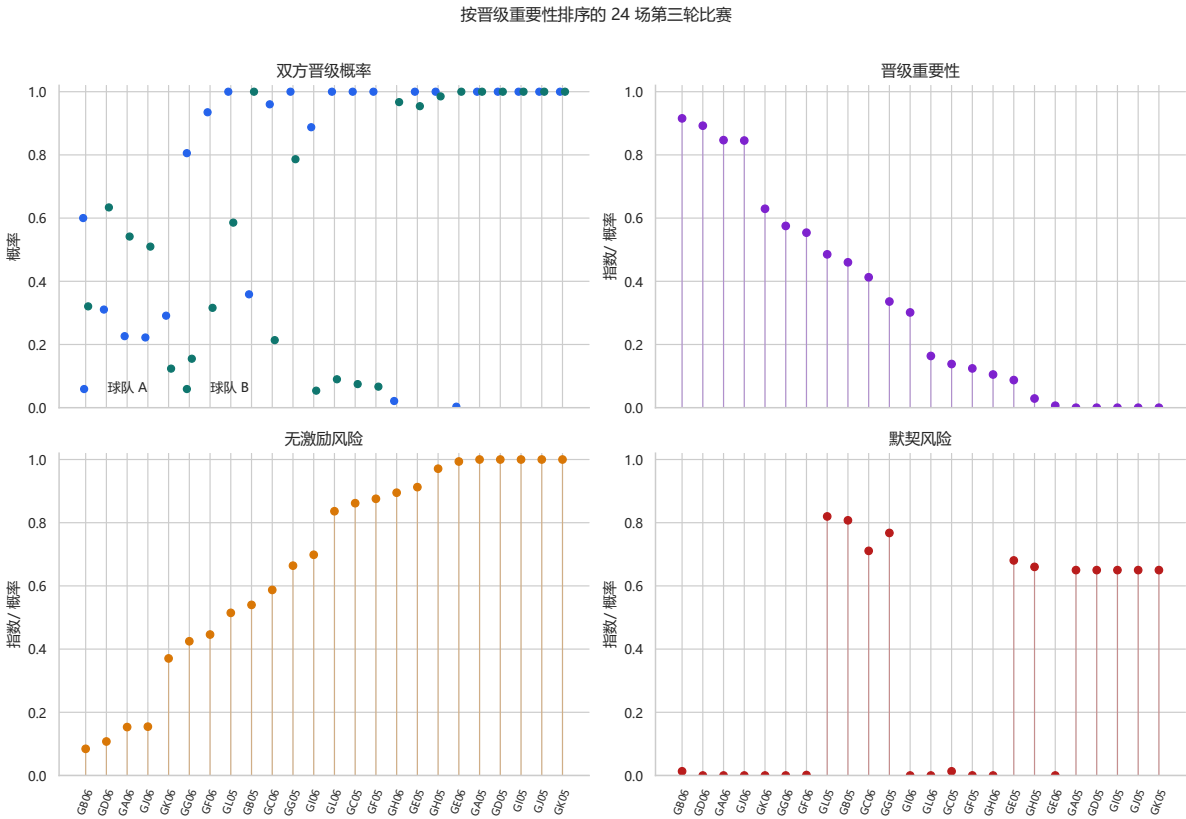


图 6 24 场第三轮比赛的晋级概率、晋级重要性、无激励风险与默契风险。比赛按 Q_i 从高到低排序。

5.8 五场动作改变的机制

表 10–11 只展示非零变化的五场，数据由论文分析脚本从正式输出和更新环境自动重算。动作缩写 B/S/T 依次表示转播、安保和交通等级。

表 10 五场变化比赛的晋级形势与风险

比赛	A 队晋级	B 队晋级	重要性	无激励	默契
GA05	1.000000	1.000000	0.000000	1.000000	0.650000
GF05	1.000000	0.066587	0.124307	0.875693	0.000207
GG05	1.000000	0.786392	0.335960	0.664040	0.767586
GJ05	1.000000	1.000000	0.000000	1.000000	0.650000
GL05	1.000000	0.585683	0.485317	0.514683	0.819861

GA05 的双方晋级概率均为 1， $Q_i = 0$ ，无激励风险和默契风险分别为 1 和 0.65。动态方案将

表 11 五场比赛的静态、动态动作与净值

比赛	静态动作	动态动作	静态净值	动态净值	改善率
GA05	B3/S2/T3/18.583%	B2/S2/T2/18.583%	0.011215	0.000257	-0.977054
GF05	B2/S2/T2/18.583%	B2/S2/T1/18.583%	0.048872	0.049466	0.012167
GG05	B3/S2/T3/18.583%	B3/S2/T2/18.583%	0.118202	0.118265	0.000533
GJ05	B2/S1/T2/18.583%	B2/S1/T1/18.583%	-0.073705	-0.072456	0.016943
GL05	B2/S1/T1/18.583%	B3/S1/T1/18.583%	0.040658	0.055190	0.357428

转播由 3 级降至 2 级、交通由 3 级降至 2 级，单场净值由 0.011215 降至 0.000257，是唯一负向变化。这不是求解器错误：多场 MILP 最大化 24 场总净值，并不要求逐场 Pareto 改善。日级高级资源容量存在绑定，但由于本文未计算对偶价格或逐场反事实，不把该资源的唯一受益对象归因于某场比赛。

GF05、GG05 和 GJ05 均将交通等级下调一级，改善率分别为 1.2167%、0.0533% 和 1.6943%；这些场次的更新需求与风险使较高交通等级的边际收益不再覆盖资源成本。GL05 的英格兰和巴拿马晋级概率分别为 1 和 0.585683， $Q_i = 0.485317$ ，且默契风险为 0.819861；动态方案将转播由 2 级提至 3 级，单场净值由 0.040658 增至 0.055190，是五场中绝对改善最大的比赛。

5.9 日容量、票价边界与模拟收敛

表 12 中的“用量/容量”显示，6 月 23 日和 29 日的三类高级资源同时达到上限；6 月 24 日和 28 日高转播容量绑定，6 月 20 日高安保容量绑定。预算使用率最高为 86%，故资源等级数量而非总成本是主要绑定条件。

表 12 第三轮每日资源使用量与容量

日期	高转播 _ 用量/容量	高安保 _ 用量/容量	强交通 _ 用量/容量	预算使用率%
06-20	3/4	3/3	3/4	47.500000
06-22	1/3	1/3	1/3	19.000000
06-23	3/3	3/3	3/3	86.000000
06-24	3/3	0/2	1/3	40.000000
06-27	3/4	1/3	3/4	42.500000
06-28	4/4	2/3	3/4	60.000000
06-29	3/3	2/2	3/3	54.000000

24 场票价调整均为 18.583183%。对未触发容量上限的情形，共同弹性 $\varepsilon = -0.75$ 下的 88% 需求保持边界为

$$\delta \leq 0.88^{1/\varepsilon} - 1 = 0.88^{-4/3} - 1 = 18.583183\%.$$

因为 $-1 < \varepsilon < 0$ ，未达容量时 $P_i^0(1 + \delta)N_i(\delta) \propto (1 + \delta)^{1+\varepsilon}$ 随 δ 增大；达容量后票务价值也随票价上升。程序对每个候选点使用容量截断后的完整 $N_i(\delta)$ 重新验证，因此统一涨幅是共同弹性和需求保持边界的解析结果，并非程序未优化票价。

动态改善较小有三方面原因。其一，静态方案本身已在同一动作集合和日容量下优化，并非最低资源基线；其二，价格弹性和需求保持约束使票价动作高度一致；其三，更新信息只改变少数比赛的边际资源排序，19 场在动态和静态评价中的净值基本相同。因此动态价值主要体现在少量比赛的重新分配，而不是总体资源水平的大幅提高。五随机种子的数值范围见表 13。

表 13 联合模拟的随机种子稳定性检验

指标	五种子范围
动态总净值	[6.572417, 6.644567]
静态总净值	[6.566840, 6.639229]
总体改善率	[0.080406%, 0.084923%]
动态动作与主种子一致率	100%

五个种子下动作完全一致，且改善率区间较窄，说明结论对 Monte Carlo 随机种子稳定。但该检验不能证明模型对目标权重、泊松独立假设或反馈参数整体稳健。

使用同一主种子，分别用 2000、5000、10000 和 20000 次联合模拟重新建立环境并求解。每个规模都重置随机数生成器，但主客队泊松数组分两次生成，因此这些实验不是严格共用同一随机序列前缀的嵌套收敛检查。表 14 显示，四种模拟量的 24 场动作与 20000 次基准全部一致，总改善率位于 0.078870%–0.083318%。但 2000 次时单队晋级概率最大绝对差仍为 0.027517，5000 次降至 0.007748，10000 次又因随机波动为 0.011833。因此 20000 次对本题的资源动作已足够稳定，但不应声称每支球队的概率估计已严格收敛。

表 14 主种子下的 Monte Carlo 模拟次数收敛检查

模拟次数	晋级概率最大差	动作一致率	动态总值	静态总值	总改善率
2000	0.027517	1.000000	6.629479	6.624254	0.000789
5000	0.007748	1.000000	6.621664	6.616262	0.000816
10000	0.011833	1.000000	6.605048	6.599617	0.000823
20000	0.000000	1.000000	6.583849	6.578368	0.000833

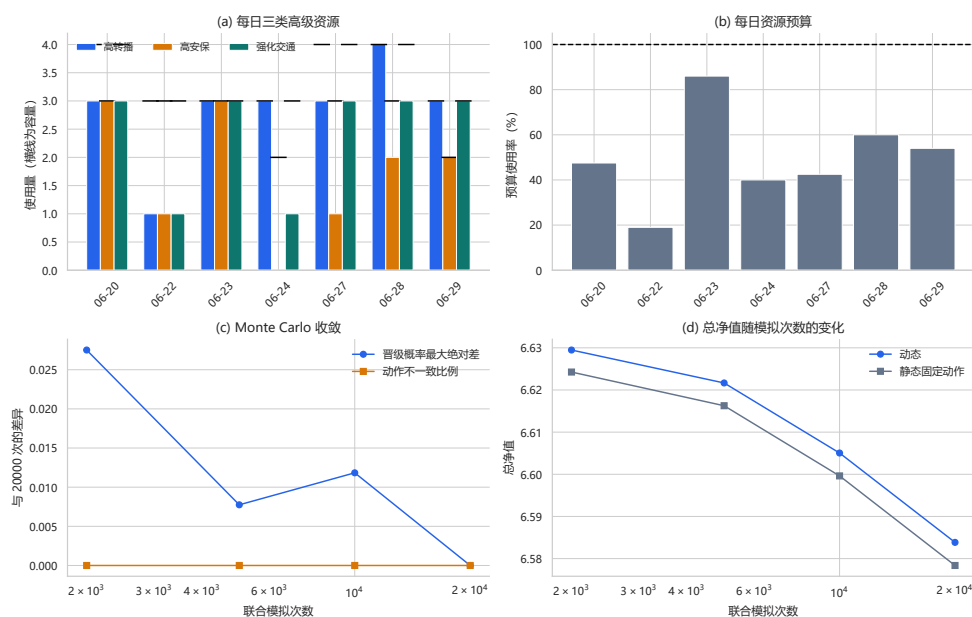


图 7 第三轮每日资源与联合模拟收敛。上左图黑色横线为各日容量；动作不一致比例在四个模拟量下均为 0。

图 7 将日资源绑定情况与动作收敛放在同一图中，说明动态调整的主要来源是离散资源数量

约束，而非预算耗尽。

6 与 2022 年世界杯实际赛程比较

6.1 数据来源与映射

实际赛程选取 2022 年卡塔尔世界杯。固定版本 Openfootball JSON 含 64 场比赛，其中小组赛 48 场 [8]；文件 SHA-256 见支撑材料校验记录。赛前 Elo 表覆盖 32 支球队，无缺失或重复 [10]。实体映射统一处理 USA 到 United States 等球队别名，并覆盖 8 个实际场馆的城市、容量、经纬度和时区。赛程时间与比分由 FIFA 比赛中心交叉核验 [9]。

模拟赛事有 48 支球队、72 场比赛、16 个场馆和 19 天赛程，2022 年小组赛则有 32 支球队、48 场比赛、8 个场馆和 13 天赛程。总量不能直接作为优劣依据，本文采用三层比较。

6.2 直接结构事实

第一层只使用赛程时间和场馆位置直接计算，不借用题目商业参数。旅行按球队赛事内相邻场馆的大圆距离累计，不计首场前往赛区的距离；休息按球队相邻比赛的 UTC 差值计算，结果见表 15。

表 15 问题四的直接结构事实

指标	2022 实际赛程	优化赛程
比赛数	48	72
球队数	32	48
场馆数	8	16
赛程跨度，天	13	19
每队平均旅行，km	37.965	3715.871
每场平均旅行，km	25.310	2477.247
跨时区转场次数	0	78
平均休息，h	97.813	128.563
最小休息，h	87	63
休息标准差，h	5.917	63.907
每场馆平均场次	6.000	4.500
每活跃场馆日平均场次	1.021	1.043
场馆活跃日利用率	0.4519	0.2270

优化赛程的平均休息更长，但最小休息更短、休息标准差明显更大，说明 19 天窗口带来的空闲并未均匀分配。2022 年赛程在旅行和跨时区指标上远低于优化赛程，但卡塔尔 8 个场馆集中于单一小范围、没有跨时区，而模拟赛事跨越多个国家和时区。即使换算为每队或每场，地理尺度仍然主导差异，因此不能将较低旅行量完全归因于 2022 排程方法更优。

6.3 映射辅助指标

第二层指标需要将实际比赛映射到题目候选对象，但尚未使用完整综合目标。实际 UTC 开球时间按环形小时差映射到同轮次候选时段，以对应时段是否位于 `global_prime_score` 前 20 名判断黄金时段；容量匹配和预计观众使用映射后的题内预计需求，不是 2022 年真实上座数据。

映射辅助指标见表 16。

表 16 问题四的映射辅助指标

指标	2022 映射值	优化赛程
黄金时段覆盖率	0.666667	0.666667
每队平均黄金时段场次	2.000	2.000
每队黄金时段次数极差	3	0
容量匹配率	0.832326	0.601677
题内预计场均观众	42051.000	41746.417

两套赛程的黄金时段覆盖率和队均次数相同，但优化赛程将每队次数极差从 3 降为 0，公平性更好。另一方面，2022 映射容量匹配率更高，题内预计场均观众略高约 304.6 人。这里的“预计观众”来自映射，不可解释为真实 2022 上座人数。

6.4 完整题内代理评价

对每场 2022 比赛，先在同轮次题目比赛中按双方赛前 Elo 均值和绝对差寻找最近邻。若实际比赛双方的 Elo 均值和绝对差分别为 $\bar{E}, \Delta E$ ，候选比赛为 $\bar{E}_i, \Delta E_i$ ，则缩放距离为

$$d_i^E = \frac{|\bar{E}_i - \bar{E}|}{200} + \frac{|\Delta E_i - \Delta E|}{200}.$$

然后在安保可行后按容量绝对差映射场馆，并按环形 UTC 小时差映射时段。由此借用题内商业、吸引力、安保和风险参数，按问题二的 T、B、U、H、C、D、F、R 定义、边界和权重计算综合值。为降低赛事规模影响，T、B、C 以场均值展示，综合标准化仍使用问题二候选边界。

映射质量是代理结论的前提，而不是可以省略的技术细节。最近邻的 d_i^E 均值和中位数分别为 0.3915 和 0.3550，90% 分位数为 0.7573；场馆容量差的中位数仅 1100 人，但 95% 分位数达 19972 人，说明少数大场馆难以找到容量接近的题内对象。UTC 小时差的中位数为 0，但 90% 分位数为 6 小时。因此映射并非普遍极远，但存在不可忽略的上尾样本。表 17 进一步给出统一边界下的完整代理分项。

表 17 最近邻映射下的完整题内代理指标

指标	2022 实际赛程代理值	优化赛程值	指标方向
场均票务价值, USD	4103559.688	3970978.750	越大越好
场均转播价值, USD	4377564.228	4535373.132	越大越好
悬念均值	0.473238	0.461436	越大越好
吸引力均值	0.673815	0.639699	越大越好
场均成本, 百万 USD	0.718750	0.736875	越小越好
旅行负担代理	0.208229	0.189247	越小越好
公平性惩罚	0.833333	0.000000	越小越好
执行风险	0.483899	0.410364	越小越好
综合值	0.196995	0.272453	越大越好

优化赛程的题内代理综合值较高，主要来自公平性、执行风险、转播价值和旅行负担代理；2022 映射结果在票务、悬念、吸引力和成本上占优。这说明综合分差异不是所有分项同向变化，而是题目权重下的取舍结果。

将最近邻改为排序后的第二近邻，2022 代理综合值由 0.196995 变为 0.210331，仍低于优化赛程的 0.272453，综合方向未反转。由于“第二近邻”本身强制选择排名第二的对象，对象 ID 改变不能解释为自然数据扰动；尤其两种时段映射的小时差分布完全一致，一些 slot_id 变化只是同一 UTC 小时下的候选 ID 替换。故这项检查只称为局部映射敏感性。

为检查综合方向是否由某一组权重巧合造成，用固定种子 20260816 对八项权重幅值独立乘以 $U(0.8, 1.2)$ ，保持正负方向并重新归一化，共生成 2000 组固定赛程的回评权重。最近邻下 $Z_2^{\text{opt}} - Z_2^{2022}$ 位于 0.05602–0.09722，第二近邻下位于 0.04677–0.07941；两组各 2000 次均未反转。这一结论仅支持：在固定赛程、固定映射、固定指标和小范围权重扰动下，题内代理综合方向保持不变；它不是重新排程后的全模型稳健性，也不是对映射假设、Elo 数据源或真实世界杯价值的全面证明。

图 8 对应前述三层口径，图 9 解释映射距离与代理分差的来源，图 10 则显示权重扰动下的方向边界。

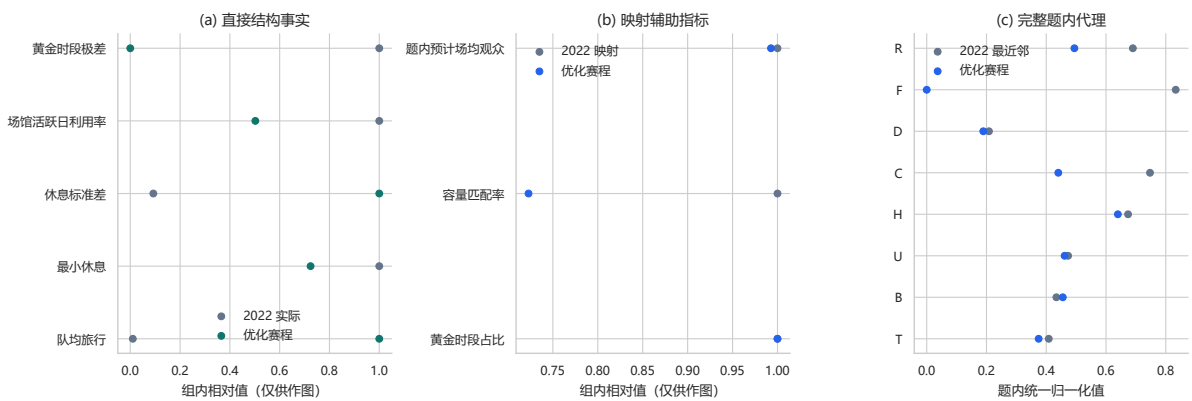


图 8 问题四的三层比较。前两个分图为解决量纲差异而对每对数值按该对最大值缩放，只比较同一指标内的相对位置。

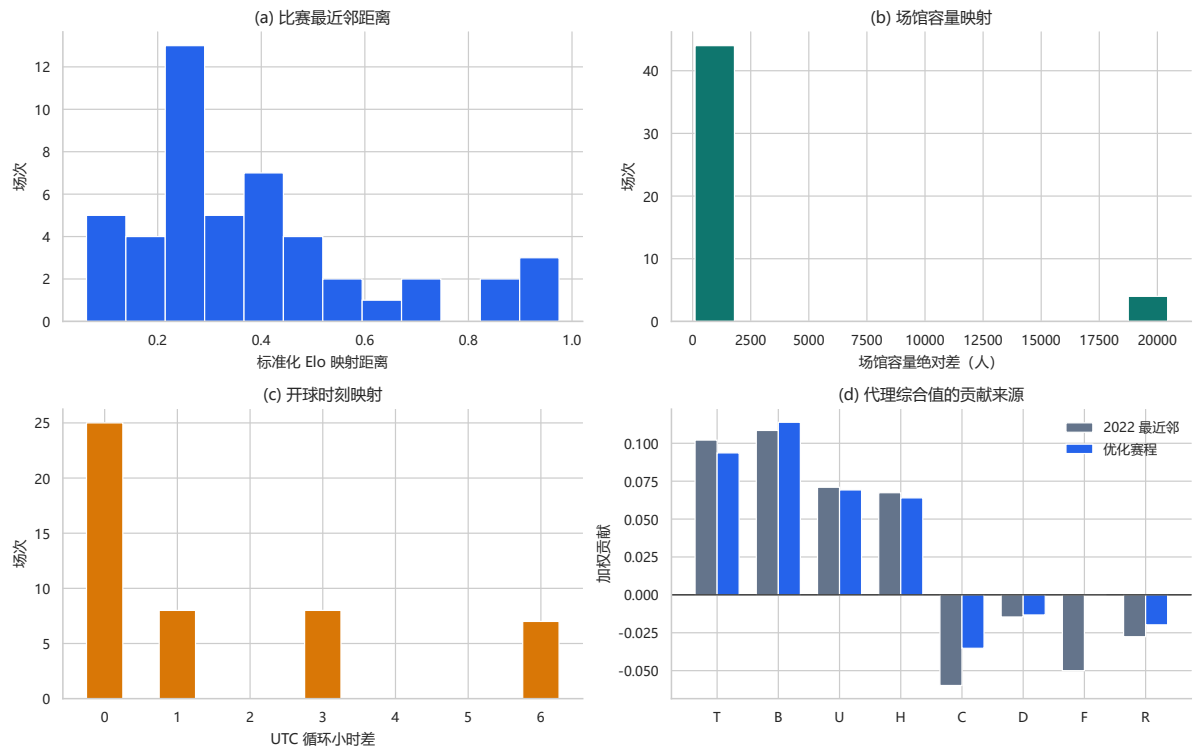


图 9 最近邻映射质量与代理综合值的分项贡献。容量差存在少数近 2 万人的上尾映射。

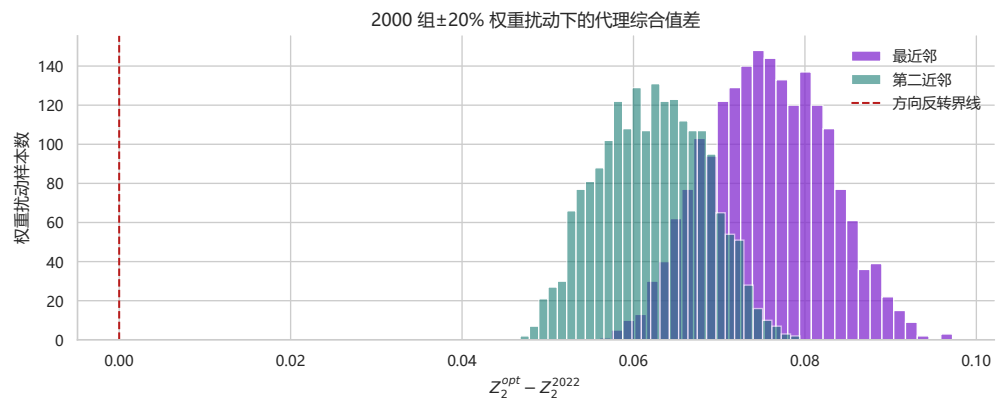


图 10 2000 组 $\pm 20\%$ 权重扰动下的代理综合值差分布。零点左侧表示综合方向反转。

7 模型评价与局限

7.1 模型特点

本文首先用统一信息时点连接四问，使问题一预测、问题二排程和问题三更新之间不存在未来信息倒流。问题一的对称特征保证球队记录顺序不影响预测，时间扩窗验证比随机划分更接近真实外推。问题二将全部硬约束转化为可独立复算的 UTC、场馆和资源条件，并报告标准化边界和目标贡献。问题三按题面同时模拟 24 场，使用浮点晋级权重处理完全同分，并把静态动作固定后置于同一更新环境评价。问题四没有将代理指标伪装为真实经营数据，而是把直接事实、映射辅助量和完整代理量分层解释。

7.2 局限性

问题一的 Elastic Net 不能充分表示复杂非线性，也无法吸收临赛新闻等附件外信息。对数逆变换直接使用 `expm1`，没有额外估计对数正态偏差修正；时间外残差也显示极端热门与冷门场次存在向中心收缩。因此 72 场预测更适合作附件信息口径下的相对转播需求，不能替代临赛实时修正。

问题二采用时段与场馆分解，不能证明三维联合模型全局最优。旅行负担又是上一轮全部安保管合格候选场馆下的均值代理，没有用转移变量描述实际连续路线；部分小组出现较长轮次空档，表明目标中缺少对过度空闲的直接惩罚。

问题三的泊松模型假定给定状态后进球独立，未描述比赛中红牌、领先后的战术变化和双方进球相关性。五种子检验只覆盖 Monte Carlo 随机性；约 0.083% 的改善较小，其实际管理价值还取决于题目资源成本指数与真实成本的对应程度。

问题四的 2022 比较受赛事规模和举办区域影响。卡塔尔的紧凑地理条件天然降低旅行和时区成本；完整代理层还依赖 Elo 最近邻、场馆容量和时段映射，无法覆盖转播合同、球队基地、城市治理、政治和实际安保安排。

8 结论

问题一以对称赛前特征和对数目标建立收视预测模型。Elastic Net 在扩窗验证中的 MSE 为 188.978683，优于 Extra Trees 和均值基线；统一特征函数将世界杯同义赛事名称归一，测试集和未来 72 场的未见赛事类别数均为 0。72 场预测均值为 161.599758 百万人，其高于训练拟合均值的主要原因是世界杯赛事指示量的正向贡献。

问题二采用时段 MILP 与固定时段后的场馆 MILP 分解求解，得到 $z_2=0.272453$ 的可行排程。两个子问题均在设定 MIP 容差内求得最优解，但不据此声明三维联合模型的最优性。最终每队获得 2 场黄金时段和 1 场大容量场馆比赛，公平性惩罚为 0，球队及同组轮次最小间隔均为 63 小时，全部硬约束复算通过。

问题三在第三轮开始前的统一信息截面上进行 20000 次 24 场联合泊松模拟，完全同分时采用等比例晋级权重。静态固定方案和动态方案在同一更新边界下的总净值分别为 6.578368 和 6.583849，总体改善率为 0.083318%。变化集中 4 场正向、1 场负向，负向场次体现了联合资源优化允许单场牺牲的取舍。五个随机种子与四种模拟次数下动作选择一致，说明这一小幅边际改善对 Monte Carlo 设置稳定，但不构成对其他模型假设的全面稳健性证明。

问题四显示，优化赛程在题内代理口径下的综合值高于 2022 实际赛程的两种邻近映射结果，且黄金时段分配更均衡；但 2022 赛程在最小休息、休息波动、场馆活跃日利用率、旅行和容量匹配等方面表现更好或更紧凑。旅行差异主要受到举办区域尺度影响，因此合理结论是两套赛程在不同组织条件下呈现指标取舍，而不是本文方案对真实世界杯安排的全面替代。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于建模方案讨论、程序调试、结果复核与语言表达整理，详细使用情况见支撑材料。

参考文献

- [1] 2026 年度“策联杯”数学建模精英联赛 C 题及数据附件，2026。
- [2] Zou H, Hastie T. Regularization and Variable Selection via the Elastic Net. *Journal of the Royal Statistical Society: Series B*, 2005, 67(2): 301–320.
- [3] Geurts P, Ernst D, Wehenkel L. Extremely Randomized Trees. *Machine Learning*, 2006, 63: 3–42.
- [4] Maher M J. Modelling Association Football Scores. *Statistica Neerlandica*, 1982, 36(3): 109–118.
- [5] Huangfu Q, Hall J A J. Parallelizing the Dual Revised Simplex Method. *Mathematical Programming Computation*, 2018, 10: 119–142.
- [6] SciPy Developers. `scipy.optimize.milp`—SciPy Reference Guide. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html>, accessed 2026-08-16.
- [7] HiGHS Development Team. HiGHS: High-performance software for linear optimization. <https://highs.dev/>, accessed 2026-08-16.
- [8] Openfootball. World Cup 2022 JSON, commit 516d3825c3bd23fdc298c4014e84bde78f2d4965. <https://github.com/openfootball/worldcup.json/blob/516d3825c3bd23fdc298c4014e84bde78f2d4965/2022/worldcup.json>.
- [9] FIFA. FIFA World Cup Qatar 2022 Match Centre. <https://www.fifa.com/tournaments/mens/worldcup/qatar2022/match-center>.
- [10] International Football. World Football Elo Ratings Table, 2022-11-12. <https://www.international-football.net/elo-ratings-table?confed=&day=12&month=11&old-team=&year=2022>.

A 支撑材料文件列表

支撑材料与论文采用相同的最终计算版本，主要文件如下：

1. code/run_all.py: 四问计算流水线入口；
2. code/pipeline_core.py: 特征、预测、MILP、联合模拟和比较指标；
3. code/check_outputs.py: 公式、约束、输出和可复现性验收；
4. code/paper_analysis.py: 论文专用的扩窗、系数、敏感性、收敛与映射质量分析；
5. requirements.txt: 固定 Python 依赖版本；
6. requirements-paper.txt: 论文作图与 PDF 质检的附加依赖；
7. output/: 四问最终 CSV 与 diagnostics.json；
8. paper/generated/: 由论文分析脚本自动生成的图、表和可追溯结果；
9. material/user_data/worldcup_2022_openfootball.json: 2022 年赛程原始数据；
10. material/user_data/worldcup_2022_pre_tournament_elo.csv: 2022 年赛前 Elo；
11. material/user_data/entity_mappings.json: 球队与场馆实体映射；
12. AI 工具使用详情.pdf: AI 工具、用途、交互与人工核验说明。

B 复现命令与算法摘要

完整源程序作为独立支撑材料提交，不在论文附录重复排印。在项目根目录执行：

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python code/run_all.py --output-dir output
python code/check_outputs.py --output-dir output
python code/paper_analysis.py --output-dir output --generated-dir paper/generated
```

问题一使用单一特征函数完成赛事名称规范化、对称特征构造、时间扩窗与全量重训练。问题二先解时段 MILP，再在时段固定后解场馆 MILP；问题三先生成 20000 次联合比分并计算浮点晋级权重，再枚举合法动作并解多重选择 MILP。问题四在固定实体映射上计算结构层与题内代理层指标。

C 输出校验摘要

自动验收重新训练问题一并逐值比对预测，复算问题二的全部指标、UTC 容量、同组轮次边界和场馆约束，验证问题三晋级权重总和、票价边界、静态动作固定复算和五种子结果，并检查问题四的三份来源材料、赛程结构与 30 项比较指标。正式结果的最终状态为 CHECK PASSED。

D 完整源程序

根据论文格式规范第三条，附录给出全部完整、可运行源程序；附录页数不计入正文 30 页限制。

D.1 流水线入口

```
1  #!/usr/bin/env python3
2  """Run all four revised models and write a complete, reproducible result set."""
3
4  from __future__ import annotations
5
6  import argparse
7  import hashlib
8  import importlib.metadata
9  import json
10 import platform
11 from pathlib import Path
12
13 from pipeline_core import (
14     RANDOM_SEEDS,
15     ROOT,
16     read_data,
17     solve_problem1,
18     solve_problem2,
19     solve_problem3,
20     solve_problem4,
21 )
22
23
24 OUTPUT_FILES = [
25     "result_1_test_prediction.csv",
26     "result_1_match_prediction.csv",
27     "result_2_group_schedule.csv",
28     "result_3_dynamic_strategy.csv",
29     "actual_schedule_2022.csv",
30     "result_4_schedule_comparison.csv",
31 ]
32
33
34 def parse_args() -> argparse.Namespace:
35     parser = argparse.ArgumentParser()
36     parser.add_argument(
37         "--output-dir", default="output_v2",
38         help="Output directory relative to C题, or an absolute path (default: output_v2).",
39     )
40     return parser.parse_args()
41
42
43 def resolve_output_dir(value: str) -> Path:
44     path = Path(value)
45     return path if path.is_absolute() else ROOT / path
46
47
48 def write_csv(frame, output_dir: Path, name: str) -> None:
49     frame.to_csv(output_dir / name, index=False, encoding="utf-8-sig")
50
51
52 def environment_diagnostics() -> dict:
53     requirements = ROOT / "requirements.txt"
54     packages = [
55         "numpy", "pandas", "scipy", "scikit-learn",
56         "openpyxl", "joblib", "threadpoolctl",
57     ]
58     return {
59         "python": platform.python_version(),
60         "python_implementation": platform.python_implementation(),
61         "platform": platform.platform(),
62         "packages": {name: importlib.metadata.version(name) for name in packages},
```

```

63     "requirements_sha256": hashlib.sha256(requirements.read_bytes()).hexdigest(),
64 }
65
66
67 def main() -> None:
68     args = parse_args()
69     output_dir = resolve_output_dir(args.output_dir)
70     output_dir.mkdir(parents=True, exist_ok=True)
71     data = read_data()
72
73     p1_test, p1_match, diagnostics1 = solve_problem1(data)
74     write_csv(p1_test, output_dir, OUTPUT_FILES[0])
75     write_csv(p1_match, output_dir, OUTPUT_FILES[1])
76
77     p2, diagnostics2, p2_context = solve_problem2(data, p1_match)
78     write_csv(p2, output_dir, OUTPUT_FILES[2])
79
80     p3, diagnostics3 = solve_problem3(data, p2)
81     write_csv(p3, output_dir, OUTPUT_FILES[3])
82
83     actual, p4, diagnostics4 = solve_problem4(data, p1_match, p2, p2_context)
84     write_csv(actual, output_dir, OUTPUT_FILES[4])
85     write_csv(p4, output_dir, OUTPUT_FILES[5])
86
87     diagnostics = {
88         "pipeline_version": "2026-08-16-v3-competition-normalized",
89         "output_directory": str(output_dir),
90         "environment": environment_diagnostics(),
91         "random_seeds": RANDOM_SEEDS,
92         **diagnostics1, **diagnostics2, **diagnostics3, **diagnostics4,
93     }
94     (output_dir / "diagnostics.json").write_text(
95         json.dumps(diagnostics, ensure_ascii=False, indent=2, allow_nan=False),
96         encoding="utf-8",
97     )
98     print(f"Wrote revised outputs to {output_dir}")
99     print(f"P1 model: {diagnostics1['problem1_selected_model']}")
100    print(f"P2 Z2: {diagnostics2['problem2_total_objective']:.10f}")
101    print(
102        "P3 dynamic/static: "
103        f"{diagnostics3['problem3_dynamic_total_net']:.10f}/"
104        f"{diagnostics3['problem3_static_total_net']:.10f}"
105    )
106
107
108 if __name__ == "__main__":
109     main()

```

D.2 模型与计算核心

```

1  """Reproducible models and independent metric recomputation for Problem C."""
2
3  from __future__ import annotations
4
5  import json
6  import math
7  import re
8  import time
9  from dataclasses import dataclass
10 from datetime import date, datetime, timezone
11 from pathlib import Path
12 from zoneinfo import ZoneInfo
13

```

```

14 import numpy as np
15 import pandas as pd
16 from scipy.optimize import Bounds, LinearConstraint, milp
17 from sklearn.compose import ColumnTransformer
18 from sklearn.dummy import DummyRegressor
19 from sklearn.ensemble import ExtraTreesRegressor
20 from sklearn.impute import SimpleImputer
21 from sklearn.linear_model import ElasticNet
22 from sklearn.metrics import mean_absolute_error, mean_squared_error
23 from sklearn.pipeline import Pipeline
24 from sklearn.preprocessing import OneHotEncoder, StandardScaler
25
26
27 ROOT = Path(__file__).resolve().parents[1]
28 ATTACH_DIR = ROOT / "2026年度“策联杯”数学建模精英联赛-C题-附件" / "2026年度“策联杯”数学建模精英联赛-C题-附件"
29 INPUT_XLSX = ATTACH_DIR / "input_data" / "C题_数据附件.xlsx"
30 TEMPLATE_DIR = ATTACH_DIR / "output_result"
31 MATERIAL_DIR = ROOT / "material" / "user_data"
32 P1_MODEL_SEED = 20260816
33 P3_PRIMARY_SEED = 20260816
34 P3_STABILITY_SEEDS = tuple(P3_PRIMARY_SEED + offset for offset in range(5))
35 P3_CONVERGENCE_SEED = 20260816
36 P4_WEIGHT_SENSITIVITY_SEED = 20260816
37 RNG_SEED = P3_PRIMARY_SEED
38 RANDOM_SEEDS = {
39     "problem1_random_models": P1_MODEL_SEED,
40     "problem3_primary_simulation": P3_PRIMARY_SEED,
41     "problem3_five_seed_stability": list(P3_STABILITY_SEEDS),
42     "problem3_convergence": P3_CONVERGENCE_SEED,
43     "problem4_weight_perturbation": P4_WEIGHT_SENSITIVITY_SEED,
44 }
45 P1_NUMERIC_FEATURES = [
46     "neutral", "rank_mean", "rank_min", "rank_max", "rank_absdiff",
47     "elo_mean", "elo_min", "elo_max", "elo_absdiff",
48     "p_draw", "win_prob_min", "win_prob_max", "win_prob_absdiff",
49     "prob_entropy",
50     "market_value_musd_mean", "market_value_musd_min", "market_value_musd_max", "market_value_musd_absdiff",
51     "avg_age_mean", "avg_age_min", "avg_age_max", "avg_age_absdiff",
52     "star_index_mean", "star_index_min", "star_index_max", "star_index_absdiff",
53     "fan_base_index_mean", "fan_base_index_min", "fan_base_index_max", "fan_base_index_absdiff",
54     "style_attack_mean", "style_attack_min", "style_attack_max", "style_attack_absdiff",
55     "style_defense_mean", "style_defense_min", "style_defense_max", "style_defense_absdiff",
56     "host_flag_mean", "host_flag_min", "host_flag_max", "host_flag_absdiff",
57     "strength_score_mean", "strength_score_min", "strength_score_max", "strength_score_absdiff",
58     "month", "weekday_num",
59 ]
60 P1_CATEGORICAL_FEATURES = ["competition", "stage_group"]
61 P2_WEIGHTS = {"T": 0.25, "B": 0.25, "U": 0.15, "H": 0.10, "C": -0.08, "D": -0.07, "F": -0.06, "R": -0.04}
62 P3_FIELDS = ["ticket_value", "broadcast_value", "attractiveness", "resource_cost", "risk"]
63
64
65 def read_data() -> dict[str, pd.DataFrame]:
66     book = pd.ExcelFile(INPUT_XLSX)
67     return {sheet: pd.read_excel(INPUT_XLSX, sheet_name=sheet) for sheet in book.sheet_names}
68
69
70 def clip(value, low=0.0, high=1.0):
71     return np.minimum(np.maximum(value, low), high)
72
73
74 def minmax_value(value: float, bounds: tuple[float, float] | list[float]) -> float:
75     low, high = map(float, bounds)

```

```

76     return 0.0 if abs(high - low) < 1e-12 else float((value - low) / (high - low))
77
78
79 def utc_timestamp(value) -> pd.Timestamp:
80     stamp = pd.to_datetime(value)
81     return stamp.tz_localize("UTC") if stamp.tzinfo is None else stamp.tz_convert("UTC")
82
83
84 def stage_key(round_in_group: int) -> str:
85     return f"Group_Match_R{int(round_in_group)}"
86
87
88 def distribution(values: pd.Series | np.ndarray) -> dict[str, float]:
89     array = np.asarray(values, dtype=float)
90     return {
91         "mean": float(np.mean(array)),
92         "std": float(np.std(array, ddof=1)) if len(array) > 1 else 0.0,
93         "min": float(np.min(array)),
94         "max": float(np.max(array)),
95     }
96
97
98 # -----
99 # Problem 1
100
101
102 def _stage_group(value: object) -> str:
103     text = str(value).lower()
104     if "group" in text:
105         return "Group"
106     if "friendly" in text:
107         return "Friendly"
108     if any(word in text for word in ["final", "quarter", "semi", "round of", "knockout"]):
109         return "Knockout"
110     return "Other"
111
112
113 def normalize_competition_name(value: object) -> str:
114     """Return a deterministic competition category shared by every P1 split."""
115     if pd.isna(value):
116         return "Other"
117     text = re.sub(r"\s+", " ", re.sub(r"[_-]+", " ", str(value).strip()))
118     key = text.casefold()
119     world_cup_key = re.sub(r"\b(?:19|20)\d{2}\b", "", key)
120     world_cup_key = re.sub(r"\s+", " ", world_cup_key).strip()
121     if re.fullmatch(r"(?:fifa\s+)?world cup", world_cup_key):
122         return "World Cup"
123     canonical = {
124         "world cup qualifier": "World Cup Qualifier",
125         "fifa world cup qualifier": "World Cup Qualifier",
126         "friendly": "Friendly",
127         "continental cup": "Continental Cup",
128         "nations league": "Nations League",
129         "other": "Other",
130     }
131     return canonical.get(key, text.title())
132
133
134 def raw_competition_series(matches: pd.DataFrame) -> pd.Series:
135     default = pd.Series("FIFA World Cup", index=matches.index, dtype=object)
136     return matches["competition"].copy() if "competition" in matches else default
137
138

```

```

139 def competition_normalization_diagnostics(
140     train: pd.DataFrame, test: pd.DataFrame, future: pd.DataFrame,
141 ) -> dict:
142     frames = {"train": train, "test": test, "future72": future}
143     normalized: dict[str, pd.Series] = {}
144     result: dict[str, dict] = {}
145     for name, frame in frames.items():
146         raw = raw_competition_series(frame).fillna("Other").astype(str)
147         canonical = raw.map(normalize_competition_name)
148         normalized[name] = canonical
149         result[name] = {
150             "before_categories": sorted(raw.unique().tolist()),
151             "after_categories": sorted(canonical.unique().tolist()),
152             "before_counts": {str(k): int(v) for k, v in raw.value_counts().sort_index().items()},
153             "after_counts": {str(k): int(v) for k, v in canonical.value_counts().sort_index().items()},
154         }
155     train_categories = set(normalized["train"])
156     for name in ("test", "future72"):
157         unseen_mask = ~normalized[name].isin(train_categories)
158         result[name]["unseen_categories"] = sorted(normalized[name][unseen_mask].unique().tolist())
159         result[name]["unseen_unique_count"] = len(result[name]["unseen_categories"])
160         result[name]["unseen_row_count"] = int(unseen_mask.sum())
161     result["normalizer"] = "normalize_competition_name_v1"
162     result["world_cup_canonical_category"] = "World Cup"
163     return result
164
165
166 def build_prediction_features(matches: pd.DataFrame, teams: pd.DataFrame) -> pd.DataFrame:
167     """One symmetric pre-match feature builder used for train, test and future."""
168     frame = matches.copy()
169     team_index = teams.set_index("team_name")
170     for side in ("a", "b"):
171         rank_col = f"strength_rank_{side}"
172         elo_col = f"elo_{side}"
173         names = frame[f"team_{side}"]
174         if rank_col not in frame:
175             frame[rank_col] = names.map(team_index["strength_rank"])
176         if elo_col not in frame:
177             frame[elo_col] = names.map(team_index["elo_rating"])
178
179     if {"p_a_win", "p_draw", "p_b_win"}.issubset(frame.columns):
180         probs = frame[["p_a_win", "p_draw", "p_b_win"]].astype(float).copy()
181         probs = probs.div(probs.sum(axis=1), axis=0)
182     else:
183         inverse = 1.0 / frame[["odds_a", "odds_draw", "odds_b"]].astype(float).clip(lower=1.000001)
184         probs = inverse.div(inverse.sum(axis=1), axis=0)
185         probs.columns = ["p_a_win", "p_draw", "p_b_win"]
186
187     out = pd.DataFrame(index=frame.index)
188     out["neutral"] = frame.get("neutral", pd.Series(True, index=frame.index)).astype(int)
189     for source, target in [("strength_rank", "rank"), ("elo", "elo")]:
190         a = frame[f"{source}_a"].astype(float)
191         b = frame[f"{source}_b"].astype(float)
192         out[f"{target}_mean"] = (a + b) / 2
193         out[f"{target}_min"] = np.minimum(a, b)
194         out[f"{target}_max"] = np.maximum(a, b)
195         out[f"{target}_absdiff"] = np.abs(a - b)
196     for attribute in (
197         "market_value_musd", "avg_age", "star_index", "fan_base_index",
198         "style_attack", "style_defense", "host_flag", "strength_score",
199     ):
200         a = frame["team_a"].map(team_index[attribute]).astype(float)
201         b = frame["team_b"].map(team_index[attribute]).astype(float)

```

```

202     out[f"{attribute}_mean"] = (a + b) / 2
203     out[f"{attribute}_min"] = np.minimum(a, b)
204     out[f"{attribute}_max"] = np.maximum(a, b)
205     out[f"{attribute}_absdiff"] = np.abs(a - b)
206 pa, pd_, pb = probs["p_a_win"], probs["p_draw"], probs["p_b_win"]
207 out["p_draw"] = pd_
208 out["win_prob_min"] = np.minimum(pa, pb)
209 out["win_prob_max"] = np.maximum(pa, pb)
210 out["win_prob_absdiff"] = np.abs(pa - pb)
211 out["prob_entropy"] = -(probs * np.log(probs.clip(lower=1e-12))).sum(axis=1) / math.log(3)
212 dates = pd.to_datetime(frame.get("date", pd.Series(pd.NaT, index=frame.index)), errors="coerce")
213 out["month"] = dates.dt.month.fillna(6).astype(int)
214 out["weekday_num"] = dates.dt.dayofweek.fillna(0).astype(int)
215 out["competition"] = raw_competition_series(frame).fillna("Other").map(normalize_competition_name)
216 if "stage" in frame:
217     out["stage_group"] = frame["stage"].map(_stage_group)
218 else:
219     out["stage_group"] = "Group"
220 return out[P1_NUMERIC_FEATURES + P1_CATEGORICAL_FEATURES]
221
222
223 def swap_match_sides(matches: pd.DataFrame) -> pd.DataFrame:
224     swapped = matches.copy()
225     pairs = [
226         ("team_a", "team_b"), ("team_a_id", "team_b_id"),
227         ("strength_rank_a", "strength_rank_b"), ("elo_a", "elo_b"),
228         ("odds_a", "odds_b"), ("p_a_win", "p_b_win"),
229         ("expected_goals_a", "expected_goals_b"),
230     ]
231     for left, right in pairs:
232         if left in swapped and right in swapped:
233             swapped[left], swapped[right] = matches[right].copy(), matches[left].copy()
234     return swapped
235
236
237 def make_p1_model(name: str) -> Pipeline:
238     preprocess = ColumnTransformer([
239         ("numeric", Pipeline([
240             ("impute", SimpleImputer(strategy="median")),
241             ("scale", StandardScaler()),
242         ]), P1_NUMERIC_FEATURES),
243         ("categorical", Pipeline([
244             ("impute", SimpleImputer(strategy="most_frequent")),
245             ("onehot", OneHotEncoder(handle_unknown="ignore")),
246         ]), P1_CATEGORICAL_FEATURES),
247     ])
248     if name == "mean":
249         regressor = DummyRegressor(strategy="mean")
250     elif name == "elastic_net":
251         regressor = ElasticNet(alpha=0.02, l1_ratio=0.10, max_iter=20000, random_state=P1_MODEL_SEED)
252     elif name == "extra_trees":
253         regressor = ExtraTreesRegressor(
254             n_estimators=500, min_samples_leaf=3, random_state=P1_MODEL_SEED, n_jobs=-1
255         )
256     else:
257         raise ValueError(name)
258     return Pipeline([("preprocess", preprocess), ("regressor", regressor)])
259
260
261 def expanding_validation(features: pd.DataFrame, target_log: np.ndarray, dates: pd.Series) -> pd.DataFrame:
262     order = np.argsort(pd.to_datetime(dates).to_numpy())
263     folds: list[tuple[np.ndarray, np.ndarray]] = []
264     for fraction in (0.60, 0.70, 0.80):

```

```

265     cut = int(len(order) * fraction)
266     end = min(len(order), cut + max(30, int(len(order) * 0.10)))
267     folds.append((order[:cut], order[cut:end]))
268     rows = []
269     for name in ("mean", "elastic_net", "extra_trees"):
270         fold_metrics = []
271         for train_idx, valid_idx in folds:
272             model = make_p1_model(name)
273             model.fit(features.iloc[train_idx], target_log[train_idx])
274             prediction = np.expm1(model.predict(features.iloc[valid_idx])).clip(min=0)
275             truth = np.expm1(target_log[valid_idx])
276             mse = mean_squared_error(truth, prediction)
277             fold_metrics.append((mse, math.sqrt(mse), mean_absolute_error(truth, prediction)))
278         values = np.asarray(fold_metrics)
279         rows.append({
280             "model": name,
281             "mse": float(values[:, 0].mean()),
282             "rmse": float(values[:, 1].mean()),
283             "mae": float(values[:, 2].mean()),
284         })
285     return pd.DataFrame(rows).sort_values("mse").reset_index(drop=True)
286
287
288 def feature_distribution_diagnostics(train: pd.DataFrame, test: pd.DataFrame, future: pd.DataFrame) -> dict:
289     result: dict[str, dict] = {}
290     for column in P1_NUMERIC_FEATURES:
291         train_values = train[column].astype(float)
292         mean = float(train_values.mean())
293         std = float(train_values.std(ddof=1))
294         limits = (mean - 4 * std, mean + 4 * std) if std > 1e-12 else (mean, mean)
295         result[column] = {
296             "train": distribution(train_values),
297             "test": distribution(test[column]),
298             "future72": distribution(future[column]),
299             "train_mean_minus_4sd": limits[0],
300             "train_mean_plus_4sd": limits[1],
301             "test_outside_4sd_count": int(((test[column] < limits[0]) | (test[column] > limits[1])).sum()),
302             "future72_outside_4sd_count": int(((future[column] < limits[0]) | (future[column] > limits[1])).
303             sum()),
304         }
305     return result
306
307 def solve_problem1(data: dict[str, pd.DataFrame]) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
308     historical = data["historical_matches"].copy()
309     train = historical[historical["dataset_split"].eq("train") & historical["tv_viewers"].notna()].copy()
310     test = historical[historical["dataset_split"].eq("test")].copy()
311     teams = data["teams"]
312     train_features = build_prediction_features(train, teams)
313     test_features = build_prediction_features(test, teams)
314     target_log = np.log1p(train["tv_viewers"].to_numpy(float) / 1_000_000)
315     metrics = expanding_validation(train_features, target_log, train["date"])
316     selected = str(metrics.iloc[0]["model"])
317     elastic_mse = float(metrics.loc[metrics["model"].eq("elastic_net"), "mse"].iloc[0])
318     if elastic_mse <= float(metrics.iloc[0]["mse"]) * 1.02:
319         selected = "elastic_net"
320     model = make_p1_model(selected)
321     model.fit(train_features, target_log)
322     test_prediction = np.expm1(model.predict(test_features)).clip(min=0)
323
324     future_raw = data["groups_matches"].merge(
325         data["base_predictions"].drop(columns=["group_id", "round_in_group", "team_a", "team_b"], errors="
ignore"),

```

```

326         on="match_id", how="left",
327     )
328     future_features = build_prediction_features(future_raw, teams)
329     future_prediction_million = np.expm1(model.predict(future_features)).clip(min=0)
330     swapped_features = build_prediction_features(swap_match_sides(future_raw), teams)
331     swapped_prediction = np.expm1(model.predict(swapped_features)).clip(min=0)
332
333     result_test = pd.DataFrame({
334         "match_id_test": test["match_id"].to_numpy(),
335         "predicted_test_tv_viewers": np.round(test_prediction, 6),
336     })
337     result_match = data["groups_matches"][["match_id", "team_a", "team_b"]].copy()
338     result_match["predicted_tv_viewers"] = np rint(future_prediction_million * 1_000_000).astype(np.int64)
339     train_fitted = np.expm1(model.predict(train_features)).clip(min=0)
340     diagnostics = {
341         "problem1_model_metrics": metrics.to_dict(orient="records"),
342         "problem1_selected_model": selected,
343         "problem1_feature_columns": {"numeric": P1_NUMERIC_FEATURES, "categorical": P1_CATEGORICAL_FEATURES
344     },
345         "problem1_feature_builder": "build_prediction_features_v3_symmetric_competition_normalized",
346         "problem1_model_seed": P1_MODEL_SEED,
347         "problem1_competition_normalization": competition_normalization_diagnostics(train, test, future_raw)
348     ,
349         "problem1_prediction_distributions": {
350             "train_fitted_million": distribution(train_fitted),
351             "test_prediction_million": distribution(test_prediction),
352             "future72_prediction_million": distribution(future_prediction_million),
353         },
354         "problem1_swap_invariance_max_abs_error_million": float(np.max(np.abs(future_prediction_million -
355             swapped_prediction))),
356         "problem1_feature_distributions": feature_distribution_diagnostics(train_features, test_features,
357             future_features),
358         "problem1_forbidden_features": [
359             "attractiveness_index", "commercial_value_index",
360             "expected_attendance_base", "uncertainty_index",
361         ],
362     }
363     return result_test, result_match, diagnostics
364
365 # -----
366 # Problem 2
367
368 def exact_top_quartile_ids(frame: pd.DataFrame, score: str, id_column: str) -> set[str]:
369     count = int(math.ceil(len(frame) * 0.25))
370     ordered = frame.sort_values([score, id_column], ascending=[False, True], kind="mergesort")
371     return set(ordered.head(count)[id_column].astype(str))
372
373 def distance_lookup(distance: pd.DataFrame) -> dict[tuple[str, str], tuple[float, float, float]]:
374     return {
375         (str(r.origin_id), str(r.destination_id)): (
376             float(r.distance_km), float(r.travel_time_hour), abs(float(r.timezone_diff))
377         )
378         for r in distance.itertuples(index=False)
379     }
380
381 @dataclass
382 class P2Context:
383     matches: pd.DataFrame
384     venue_candidates: pd.DataFrame

```

```

385 slot_candidates: pd.DataFrame
386 prime_slots: set[str]
387 large_venues: set[str]
388 bounds: dict[str, list[float]]
389
390
391 def build_p2_context(data: dict[str, pd.DataFrame], p1_match: pd.DataFrame) -> P2Context:
392     gm = data["groups_matches"].copy()
393     matches = gm.merge(
394         data["base_predictions"].drop(columns=["group_id", "round_in_group", "team_a", "team_b"], errors="
395         ignore"),
396         on="match_id", how="left",
397     ).merge(
398         data["security_requirements"][["match_id", "required_security_level", "security_demand_score"]],
399         on="match_id", how="left",
400     ).merge(p1_match[["match_id", "predicted_tv_viewers"]], on="match_id", how="left")
401     venues = data["venues"]
402     slots = data["time_slots"]
403     ticket = data["ticket_broadcast"].set_index("match_stage")
404     distance = distance_lookup(data["distance_matrix"])
405     venue_rows = []
406
407     previous_match: dict[tuple[str, int], pd.Series] = {}
408     for row in matches.itertuples(index=False):
409         previous_match[(row.team_a, int(row.round_in_group))] = pd.Series(row._asdict())
410         previous_match[(row.team_b, int(row.round_in_group))] = pd.Series(row._asdict())
411
412     def travel_components(match_row, venue_row, team_name: str, team_id: str) -> tuple[float, float, float]:
413         round_no = int(match_row.round_in_group)
414         if round_no == 1:
415             return distance[(str(team_id), str(venue_row.venue_id))]
416         prior = previous_match[(team_name, round_no - 1)]
417         required = int(prior["required_security_level"])
418         eligible = venues.loc[venues["security_level"] >= required, "venue_id"].astype(str)
419         values = [distance[(origin, str(venue_row.venue_id))] for origin in eligible]
420         return tuple(np.mean(np.asarray(values, dtype=float), axis=0))
421
422     for match in matches.itertuples(index=False):
423         tb = ticket.loc[stage_key(match.round_in_group)]
424         for venue in venues.itertuples(index=False):
425             if int(venue.security_level) < int(match.required_security_level):
426                 continue
427             ta = travel_components(match, venue, match.team_a, match.team_a_id)
428             tb_team = travel_components(match, venue, match.team_b, match.team_b_id)
429             components = (np.asarray(ta) + np.asarray(tb_team)) / 2
430             attendance = min(float(match.expected_attendance_base), float(venue.capacity))
431             venue_rows.append({
432                 "match_id": match.match_id,
433                 "venue_id": venue.venue_id,
434                 "expected_attendance": attendance,
435                 "ticket_revenue_usd": float(tb["base_ticket_price_usd"]) * attendance,
436                 "match_cost_musd": float(venue.operation_cost_musd_per_match)
437                     + 0.1 * int(match.required_security_level) * float(venue.security_cost_index),
438                 "distance_km": float(components[0]),
439                 "travel_time_hour": float(components[1]),
440                 "timezone_diff": float(components[2]),
441                 "risk_index": 0.5 * float(venue.climate_risk)
442                     + 0.3 * attendance / float(venue.capacity)
443                     + 0.2 * int(match.required_security_level) / float(venue.security_level),
444             })
445     venue_candidates = pd.DataFrame(venue_rows)
446     component_bounds = {}
447     for column in ("distance_km", "travel_time_hour", "timezone_diff"):

```

```

447     low, high = float(venue_candidates[column].min()), float(venue_candidates[column].max())
448     component_bounds[column] = [low, high]
449     venue_candidates[column + "_norm"] = venue_candidates[column].map(lambda value: minmax_value(value,
450 (low, high)))
451 venue_candidates["travel_burden"] = (
452     0.5 * venue_candidates["distance_km_norm"]
453     + 0.3 * venue_candidates["travel_time_hour_norm"]
454     + 0.2 * venue_candidates["timezone_diff_norm"]
455 )
456 slot_rows = []
457 for match in matches.itertuples(index=False):
458     tb = ticket.loc[stage_key(match.round_in_group)]
459     for slot in slots.itertuples(index=False):
460         slot_rows.append({
461             "match_id": match.match_id,
462             "slot_id": slot.slot_id,
463             "broadcast_value_usd": float(match.predicted_tv_viewers)
464                 * float(tb["broadcast_unit_value_usd"])
465                 * float(slot.global_prime_score)
466                 * float(tb["sponsor_weight"]),
467         })
468 slot_candidates = pd.DataFrame(slot_rows)
469 bounds = {
470     "ticket_revenue_usd": [float(venue_candidates["ticket_revenue_usd"].min()), float(venue_candidates["
471 ticket_revenue_usd"].max())],
472     "broadcast_value_usd": [float(slot_candidates["broadcast_value_usd"].min()), float(slot_candidates["
473 broadcast_value_usd"].max())],
474     "match_cost_musd": [float(venue_candidates["match_cost_musd"].min()), float(venue_candidates["
475 match_cost_musd"].max())],
476     "travel_burden": [0.0, 1.0],
477     "risk_index": [float(venue_candidates["risk_index"].min()), float(venue_candidates["risk_index"].max
478 ())],
479     "travel_components": component_bounds,
480 }
481 for column in ("ticket_revenue_usd", "match_cost_musd", "risk_index"):
482     venue_candidates[column + "_norm"] = venue_candidates[column].map(lambda value: minmax_value(value,
483 bounds[column]))
484 slot_candidates["broadcast_value_usd_norm"] = slot_candidates["broadcast_value_usd"].map(
485     lambda value: minmax_value(value, bounds["broadcast_value_usd"])
486 )
487 return P2Context(
488     matches=matches,
489     venue_candidates=venue_candidates,
490     slot_candidates=slot_candidates,
491     prime_slots=exact_top_quartile_ids(slots, "global_prime_score", "slot_id"),
492     large_venues=exact_top_quartile_ids(venues, "capacity", "venue_id"),
493     bounds=bounds,
494 )
495
496 def _milp_result_diagnostics(result, started: float) -> dict:
497     gap = float(getattr(result, "mip_gap", np.nan))
498     if result.status == 0:
499         status = "optimal_within_mip_tolerance"
500         interpretation = "在设定 MIP 容差内求得最优解"
501     elif result.status == 1:
502         status = "limit_reached"
503         interpretation = "达到求解限制, 保留当前可行解与实际 gap"
504     else:
505         status = "failed"
506         interpretation = "求解失败"
507     return {

```

```

504     "status_code": int(result.status),
505     "solver_status": "optimal" if result.status == 0 else "limit_reached" if result.status == 1 else "
failed",
506     "status": status,
507     "interpretation": interpretation,
508     "message": str(result.message),
509     "runtime_seconds": float(time.perf_counter() - started),
510     "best_bound": float(getattr(result, "mip_dual_bound", np.nan)),
511     "mip_gap": gap,
512     "node_count": int(getattr(result, "mip_node_count", 0)),
513 }
514
515
516 def solve_p2_slots(
517     ctx: P2Context, data: dict[str, pd.DataFrame],
518     weights: dict[str, float] | None = None,
519 ) -> tuple[pd.DataFrame, dict]:
520     weights = P2_WEIGHTS if weights is None else weights
521     matches, slots = ctx.matches.reset_index(drop=True), data["time_slots"].reset_index(drop=True)
522     n_match, n_slot = len(matches), len(slots)
523     n_x = n_match * n_slot
524     pmax, pmin, n_var = n_x, n_x + 1, n_x + 2
525     slot_index = {str(row.slot_id): i for i, row in slots.iterrows()}
526     match_index = {str(row.match_id): i for i, row in matches.iterrows()}
527     score = ctx.slot_candidates.pivot(index="match_id", columns="slot_id", values="broadcast_value_usd_norm"
)
528     c = np.zeros(n_var)
529     for match_id, i in match_index.items():
530         for slot_id, j in slot_index.items():
531             c[i * n_slot + j] = -weights["B"] * float(score.loc[match_id, slot_id]) / n_match
532     c[pmax], c[pmin] = -weights["F"] / 6, weights["F"] / 6
533     rows, lower, upper = [], [], []
534
535     def add(entries: dict[int, float], low: float, high: float):
536         row = np.zeros(n_var)
537         for index, value in entries.items():
538             row[index] = value
539         rows.append(row); lower.append(low); upper.append(high)
540
541     for i in range(n_match):
542         add({i * n_slot + j: 1 for j in range(n_slot)}, 1, 1)
543     for j, slot in slots.iterrows():
544         add({i * n_slot + j: 1 for i in range(n_match)}, 0, int(slot.broadcast_capacity))
545     times = np.array([(utc_timestamp(value) - utc_timestamp(slots["reference_utc_time"].iloc[0]))
546         .total_seconds() / 3600 for value in slots["reference_utc_time"]])
547     all_teams = sorted(set(matches["team_a"]) | set(matches["team_b"]))
548     for team in all_teams:
549         team_matches = matches[(matches["team_a"].eq(team)) | (matches["team_b"].eq(team))].sort_values("
550 round_in_group")
551         ids = [match_index[mid] for mid in team_matches["match_id"]]
552         for first, second in zip(ids, ids[1:]):
553             entries = {first * n_slot + j: -times[j] for j in range(n_slot)}
554             entries.update({second * n_slot + j: times[j] for j in range(n_slot)})
555             add(entries, 60, np.inf)
556     # Stronger group boundary: every match in round r+1 starts at least
557     # 60 hours after every match in round r for the same group.
558     for _, group_matches in matches.groupby("group_id"):
559         for round_no in (1, 2):
560             previous_ids = [
561                 match_index[mid]
562                 for mid in group_matches.loc[group_matches["round_in_group"].eq(round_no), "match_id"]
563             ]
564             next_ids = [

```

```

563         match_index[mid]
564         for mid in group_matches.loc[group_matches["round_in_group"].eq(round_no + 1), "match_id"]
565     ]
566     for previous in previous_ids:
567         for following in next_ids:
568             entries = {previous * n_slot + j: -times[j] for j in range(n_slot)}
569             entries.update({following * n_slot + j: times[j] for j in range(n_slot)})
570             add(entries, 60, np.inf)
571 limits = data["dynamic_resource_limits"].set_index("reference_date")
572 for day, day_slots in slots.groupby("date"):
573     high_matches = matches.index[matches["round_in_group"].eq(3) & matches["required_security_level"].ge
574 (3)]
575     entries = {i * n_slot + j: 1 for i in high_matches for j in day_slots.index}
576     add(entries, 0, int(limits.loc[day, "high_security_capacity"]))
577 for team in all_teams:
578     team_ids = matches.index[(matches["team_a"].eq(team)) | (matches["team_b"].eq(team))]
579     entries = {
580         i * n_slot + j: 1
581         for i in team_ids for j, slot in slots.iterrows()
582         if str(slot.slot_id) in ctx.prime_slots
583     }
584     add({**entries, pmax: -1}, -np.inf, 0)
585     add({**entries, pmin: -1}, 0, np.inf)
586 add({pmax: 1, pmin: -1}, -np.inf, 2)
587 lower_bounds = np.zeros(n_var)
588 upper_bounds = np.r_[np.ones(n_x), 3, 3]
589 started = time.perf_counter()
590 result = milp(
591     c=c, integrality=np.ones(n_var),
592     bounds=Bounds(lower_bounds, upper_bounds),
593     constraints=LinearConstraint(np.vstack(rows), np.asarray(lower), np.asarray(upper)),
594     options={"time_limit": 180, "mip_rel_gap": 0.0005, "presolve": True},
595 )
596 if result.x is None:
597     raise RuntimeError(f"Problem 2 slot MILP failed: {result.message}")
598 chosen = []
599 for i, match in matches.iterrows():
600     values = result.x[i * n_slot:(i + 1) * n_slot]
601     chosen.append({"match_id": match.match_id, "slot_id": slots.iloc[int(np.argmax(values))].slot_id})
602 diagnostics = _milp_result_diagnostics(result, started)
603 diagnostics.update({"variable_count": n_var, "constraint_count": len(rows)})
604 return pd.DataFrame(chosen), diagnostics
605
606 def solve_p2_venues(
607     ctx: P2Context, data: dict[str, pd.DataFrame], match_slots: pd.DataFrame,
608     weights: dict[str, float] | None = None,
609 ) -> tuple[pd.DataFrame, float]:
610     weights = P2_WEIGHTS if weights is None else weights
611     matches = ctx.matches.merge(match_slots, on="match_id").reset_index(drop=True)
612     venues = data["venues"].reset_index(drop=True)
613     slots = data["time_slots"].set_index("slot_id")
614     pairs = ctx.venue_candidates[["match_id", "venue_id"]].drop_duplicates().reset_index(drop=True)
615     pairs = pairs.merge(match_slots, on="match_id", how="left")
616     pair_metric = ctx.venue_candidates.set_index(["match_id", "venue_id"])
617     n_pair = len(pairs)
618     lmax, lmin, n_var = n_pair, n_pair + 1, n_pair + 2
619     c = np.zeros(n_var)
620     for k, pair in pairs.iterrows():
621         metric = pair_metric.loc[(pair.match_id, pair.venue_id)]
622         value = (
623             weights["T"] * metric.ticket_revenue_usd_norm
624             + weights["C"] * metric.match_cost_musd_norm

```

```

625         + weights["D"] * metric.travel_burden
626         + weights["R"] * metric.risk_index_norm
627     ) / len(matches)
628     c[k] = -float(value)
629 c[lmax], c[lmin] = -weights["F"] / 6, weights["F"] / 6
630 rows, lower, upper = [], [], []
631
632 def add(indices: list[int], coefficients: list[float] | None, low: float, high: float, extras: dict[int,
float] | None = None):
633     row = np.zeros(n_var)
634     row[indices] = 1 if coefficients is None else coefficients
635     for index, value in (extras or {}).items():
636         row[index] = value
637     rows.append(row); lower.append(low); upper.append(high)
638
639 for match_id, indices in pairs.groupby("match_id").groups.items():
640     add(list(indices), None, 1, 1)
641 venue_index = venues.set_index("venue_id")
642 for venue_id, indices in pairs.groupby("venue_id").groups.items():
643     venue = venue_index.loc[venue_id]
644     add(list(indices), None, int(venue.min_total_matches), int(venue.max_total_matches))
645 local_groups: dict[tuple[str, str], list[int]] = {}
646 utc_groups: dict[tuple[str, str], list[int]] = {}
647 for k, pair in pairs.iterrows():
648     venue = venue_index.loc[pair.venue_id]
649     utc = utc_timestamp(slots.loc[pair.slot_id, "reference_utc_time"])
650     local_day = utc.tz_convert(ZoneInfo(str(venue.timezone))).date().isoformat()
651     local_groups.setdefault((pair.venue_id, local_day), []).append(k)
652     utc_groups.setdefault((pair.venue_id, utc.isoformat()), []).append(k)
653 for (venue_id, _), indices in local_groups.items():
654     add(indices, None, 0, int(venue_index.loc[venue_id, "max_matches_per_day"]))
655 for indices in utc_groups.values():
656     if len(indices) > 1:
657         add(indices, None, 0, 1)
658 all_teams = sorted(set(matches["team_a"]) | set(matches["team_b"]))
659 match_team = matches.set_index("match_id")[["team_a", "team_b"]]
660 for team in all_teams:
661     indices = [
662         k for k, pair in pairs.iterrows()
663         if pair.venue_id in ctx.large_venues
664         and team in (match_team.loc[pair.match_id, "team_a"], match_team.loc[pair.match_id, "team_b"])
665     ]
666     add(indices, None, -np.inf, 0, {lmax: -1})
667     add(indices, None, 0, np.inf, {lmin: -1})
668 started = time.perf_counter()
669 result = milp(
670     c=c, integrality=np.ones(n_var),
671     bounds=Bounds(np.zeros(n_var), np.r_[np.ones(n_pair), 3, 3]),
672     constraints=LinearConstraint(np.vstack(rows), np.asarray(lower), np.asarray(upper)),
673     options={"time_limit": 180, "mip_rel_gap": 0.0005, "presolve": True},
674 )
675 if result.x is None:
676     raise RuntimeError(f"Problem 2 venue MILP failed: {result.message}")
677 chosen = pairs.loc[np.where(result.x[:n_pair] > 0.5)[0], ["match_id", "venue_id", "slot_id']].copy()
678 diagnostics = _milp_result_diagnostics(result, started)
679 diagnostics.update({"variable_count": n_var, "constraint_count": len(rows)})
680 return chosen, diagnostics
681
682
683 def p2_fairness(schedule: pd.DataFrame, prime_slots: set[str], large_venues: set[str]) -> tuple[float, dict
]:
684     teams = sorted(set(schedule["team_a"]) | set(schedule["team_b"]))
685     prime = {team: 0 for team in teams}

```

```

686     large = {team: 0 for team in teams}
687     for row in schedule.itertuples(index=False):
688         for team in (row.team_a, row.team_b):
689             prime[team] += int(str(row.slot_id) in prime_slots)
690             large[team] += int(str(row.venue_id) in large_venues)
691     prime_range = max(prime.values()) - min(prime.values())
692     large_range = max(large.values()) - min(large.values())
693     value = 0.5 * prime_range / 3 + 0.5 * large_range / 3
694     return float(value), {
695         "prime_counts": prime,
696         "large_venue_counts": large,
697         "prime_range": int(prime_range),
698         "large_venue_range": int(large_range),
699     }
700
701
702 def p2_metrics(schedule: pd.DataFrame, ctx: P2Context, data: dict[str, pd.DataFrame]) -> dict:
703     base = data["base_predictions"].set_index("match_id")
704     venues = data["venues"]
705     selected_venues = schedule[["match_id", "venue_id"]].merge(
706         ctx.venue_candidates, on=["match_id", "venue_id"], how="left"
707     )
708     selected_slots = schedule[["match_id", "slot_id"]].merge(
709         ctx.slot_candidates, on=["match_id", "slot_id"], how="left"
710     )
711     fairness, fairness_detail = p2_fairness(schedule, ctx.prime_slots, ctx.large_venues)
712     setup = float(venues.loc[venues["venue_id"].isin(schedule["venue_id"].unique()), "setup_cost_musd"].sum())
713     raw = {
714         "T": float(selected_venues["ticket_revenue_usd"].sum()),
715         "B": float(selected_slots["broadcast_value_usd"].sum()),
716         "U": float(base.loc[schedule["match_id"], "uncertainty_index"].mean()),
717         "H": float((base.loc[schedule["match_id"], "attractiveness_index"] / 100).mean()),
718         "C": setup + float(selected_venues["match_cost_musd"].sum()),
719         "D": float(selected_venues["travel_burden"].mean()),
720         "F": fairness,
721         "R": float(selected_venues["risk_index"].mean()),
722     }
723     n = len(schedule)
724     bounds = {
725         "T": [n * ctx.bounds["ticket_revenue_usd"][0], n * ctx.bounds["ticket_revenue_usd"][1]],
726         "B": [n * ctx.bounds["broadcast_value_usd"][0], n * ctx.bounds["broadcast_value_usd"][1]],
727         "U": [0.0, 1.0],
728         "H": [0.0, 1.0],
729         "C": [setup + n * ctx.bounds["match_cost_musd"][0], setup + n * ctx.bounds["match_cost_musd"][1]],
730         "D": [0.0, 1.0],
731         "F": [0.0, 1.0],
732         "R": ctx.bounds["risk_index"],
733     }
734     normalized = {key: minmax_value(raw[key], bounds[key]) for key in raw}
735     contributions = {key: P2_WEIGHTS[key] * normalized[key] for key in raw}
736     return {
737         "raw": raw,
738         "bounds": bounds,
739         "normalized": normalized,
740         "weighted_contributions": contributions,
741         "Z2": float(sum(contributions.values())),
742         "fairness_detail": fairness_detail,
743         "setup_cost_musd": setup,
744     }
745
746
747 def p2_hard_constraint_audit(schedule: pd.DataFrame, ctx: P2Context, data: dict[str, pd.DataFrame]) -> dict:

```

```

748 venues = data["venues"].set_index("venue_id")
749 slots = data["time_slots"].set_index("slot_id")
750 limits = data["dynamic_resource_limits"].set_index("reference_date")
751 results: dict[str, object] = {}
752 results["unique_matches"] = int(schedule["match_id"].nunique()) == 72
753 results["venue_utc_conflicts"] = int(schedule.duplicated(["venue_id", "utc_datetime"]).sum())
754 rest_values = []
755 order_ok = True
756 for team in sorted(set(schedule["team_a"]) | set(schedule["team_b"])):
757     subset = schedule[(schedule["team_a"].eq(team)) | (schedule["team_b"].eq(team))].sort_values("
round_in_group")
758     order_ok &= subset["round_in_group"].tolist() == [1, 2, 3]
759     differences = pd.to_datetime(subset["utc_datetime"], utc=True).diff().dropna().dt.total_seconds() /
3600
760     rest_values.extend(differences.tolist())
761 results["team_round_order"] = bool(order_ok)
762 results["minimum_rest_hours"] = float(min(rest_values))
763 group_round_gaps = {}
764 for group_id, group_matches in schedule.groupby("group_id"):
765     group_times = group_matches.assign(_utc=pd.to_datetime(group_matches["utc_datetime"], utc=True))
766     for round_no in (1, 2):
767         previous_max = group_times.loc[group_times["round_in_group"].eq(round_no), "_utc"].max()
768         next_min = group_times.loc[group_times["round_in_group"].eq(round_no + 1), "_utc"].min()
769         key = f"{group_id}:R{round_no}-R{round_no + 1}"
770         group_round_gaps[key] = float((next_min - previous_max).total_seconds() / 3600)
771 results["group_round_gaps_hours"] = group_round_gaps
772 results["group_round_minimum_gap_hours"] = float(min(group_round_gaps.values()))
773 results["group_round_violation_count"] = int(sum(gap < 60 - 1e-9 for gap in group_round_gaps.values()))
774 results["group_round_boundary_passed"] = results["group_round_violation_count"] == 0
775 local_counts = schedule.assign(local_day=pd.to_datetime(schedule["local_datetime"]).dt.date).groupby(["
venue_id", "local_day"]).size()
776 results["venue_local_day_max_violation"] = int(sum(
777     count > int(venues.loc[venue_id, "max_matches_per_day"])
778     for (venue_id, _), count in local_counts.items()
779 ))
780 total_counts = schedule.groupby("venue_id").size()
781 results["venue_total_violation"] = int(sum(
782     not (int(row.min_total_matches) <= int(total_counts.get(venue_id, 0)) <= int(row.max_total_matches))
783     for venue_id, row in venues.iterrows()
784 ))
785 results["broadcast_capacity_violation"] = int(sum(
786     count > int(slots.loc[slot_id, "broadcast_capacity"])
787     for slot_id, count in schedule.groupby("slot_id").size().items()
788 ))
789 results["security_violation"] = int(sum(
790     int(row.required_security_level) > int(venues.loc[row.venue_id, "security_level"])
791     for row in schedule.itertuples(index=False)
792 ))
793 third_high = schedule[schedule["round_in_group"].eq(3) & schedule["required_security_level"].ge(3)].
groupby("reference_date").size()
794 results["third_round_high_security_violation"] = int(sum(
795     count > int(limits.loc[day, "high_security_capacity"]) for day, count in third_high.items()
796 ))
797 results["prime_range"] = p2_fairness(schedule, ctx.prime_slots, ctx.large_venues)[1]["prime_range"]
798 results["all_passed"] = bool(
799     results["unique_matches"] and results["team_round_order"]
800     and results["minimum_rest_hours"] >= 60 - 1e-9
801     and results["group_round_boundary_passed"]
802     and all(results[key] == 0 for key in [
803         "venue_utc_conflicts", "venue_local_day_max_violation", "venue_total_violation",
804         "broadcast_capacity_violation", "security_violation", "third_round_high_security_violation",
805     ])
806     and results["prime_range"] <= 2

```

```

807 )
808 return results
809
810
811 def solve_problem2(data: dict[str, pd.DataFrame], p1_match: pd.DataFrame) -> tuple[pd.DataFrame, dict,
    P2Context]:
812     context = build_p2_context(data, p1_match)
813     match_slots, slot_solver = solve_p2_slots(context, data)
814     assignments, venue_solver = solve_p2_venues(context, data, match_slots)
815     venues = data["venues"].set_index("venue_id")
816     slots = data["time_slots"].set_index("slot_id")
817     schedule = context.matches.merge(assignments, on="match_id", how="left")
818     venue_metric = context.venue_candidates.set_index(["match_id", "venue_id"])
819     slot_metric = context.slot_candidates.set_index(["match_id", "slot_id"])
820     rows = []
821     for row in schedule.itertuples(index=False):
822         venue = venues.loc[row.venue_id]
823         slot = slots.loc[row.slot_id]
824         vm = venue_metric.loc[(row.match_id, row.venue_id)]
825         sm = slot_metric.loc[(row.match_id, row.slot_id)]
826         utc = utc_timestamp(slot.reference_utc_time)
827         local = utc.tz_convert(ZoneInfo(str(venue.timezone)))
828         rows.append({
829             "match_id": row.match_id, "group_id": row.group_id,
830             "round_in_group": int(row.round_in_group), "team_a": row.team_a, "team_b": row.team_b,
831             "venue_id": row.venue_id, "slot_id": row.slot_id,
832             "city": venue.city, "country": venue.country,
833             "reference_date": pd.to_datetime(slot.date).date().isoformat(),
834             "reference_kickoff_time": str(slot.reference_kickoff_time),
835             "local_datetime": local.strftime("%Y-%m-%d %H:%M"),
836             "utc_datetime": utc.strftime("%Y-%m-%d %H:%M"),
837             "required_security_level": int(row.required_security_level),
838             "expected_attendance": round(float(vm.expected_attendance), 2),
839             "expected_tv_viewers": int(row.predicted_tv_viewers),
840             "ticket_revenue_usd": round(float(vm.ticket_revenue_usd), 2),
841             "broadcast_value_usd": round(float(sm.broadcast_value_usd), 2),
842             "travel_cost_index": round(float(vm.travel_burden), 8),
843             "fairness_penalty": np.nan,
844             "risk_index": round(float(vm.risk_index), 8),
845             "total_objective_value": "",
846         })
847     output = pd.DataFrame(rows).sort_values("match_id").reset_index(drop=True)
848     fairness = p2_fairness(output, context.prime_slots, context.large_venues)[0]
849     output["fairness_penalty"] = round(fairness, 8)
850     metrics = p2_metrics(output, context, data)
851     output.loc[0, "total_objective_value"] = f"{metrics['Z2']:.10f}"
852     audit = p2_hard_constraint_audit(output, context, data)
853     diagnostics = {
854         "problem2_total_objective": metrics["Z2"],
855         "problem2_indicator_details": metrics,
856         "problem2_candidate_normalization": context.bounds,
857         "problem2_prime_slots": sorted(context.prime_slots),
858         "problem2_large_capacity_venues": sorted(context.large_venues),
859         "problem2_solver": {
860             "method": "decomposed_slot_MILP_then_conditional_venue_MILP",
861             "global_optimality_claim": False,
862             "slot_subproblem": slot_solver,
863             "venue_subproblem_given_slots": venue_solver,
864         },
865         "problem2_hard_constraint_audit": audit,
866     }
867     return output, diagnostics, context
868

```

```

869
870 # -----
871 # Problem 3
872
873
874 def standings_from_live(data: dict[str, pd.DataFrame]) -> pd.DataFrame:
875     members = data["group_membership"]
876     stats = {
877         row.team_name: {
878             "team_id": row.team_id, "group_id": row.group_id, "G": 0, "P": 0,
879             "GF": 0, "GA": 0, "xGF": 0.0, "xGA": 0.0, "RC": 0, "injuries": [],
880         }
881         for row in members.itertuples(index=False)
882     }
883     for row in data["live_group_results"].itertuples(index=False):
884         for side, opponent in (("a", "b"), ("b", "a")):
885             team = getattr(row, f"team_{side}")
886             goals_for = int(getattr(row, f"goals_{side}"))
887             goals_against = int(getattr(row, f"goals_{opponent}"))
888             item = stats[team]
889             item["G"] += 1
890             item["P"] += 3 if goals_for > goals_against else 1 if goals_for == goals_against else 0
891             item["GF"] += goals_for; item["GA"] += goals_against
892             item["xGF"] += float(getattr(row, f"xg_{side}")); item["xGA"] += float(getattr(row, f"xg_{opponent}"))
893             item["RC"] += int(getattr(row, f"red_cards_{side}"))
894             item["injuries"].append(float(row.injury_impact_level))
895     rows = []
896     for team, item in stats.items():
897         games = item["G"]
898         state = (
899             0.45 * item["P"] / (3 * games)
900             + 0.35 * (0.5 + 0.5 * math.tanh(((item["xGF"] - item["xGA"]) / games) / 1.25))
901             + 0.20 * math.exp(-0.55 * item["RC"] / games)
902         )
903         rows.append({
904             **{key: value for key, value in item.items() if key != "injuries"},
905             "team_name": team, "state": float(clip(state)),
906             "injury": float(clip(np.mean(item["injuries"]), 0, 3)),
907         })
908     return pd.DataFrame(rows)
909
910
911 def _rank_key(team_index: int, points: np.ndarray, goals_for: np.ndarray, goals_against: np.ndarray) ->
912     tuple[int, int, int]:
913     return int(points[team_index]), int(goals_for[team_index] - goals_against[team_index]), int(goals_for[
914         team_index])
915
916
917 def fractional_advancement_weights(
918     points: np.ndarray, goals_for: np.ndarray, goals_against: np.ndarray,
919     groups: dict[str, list[int]],
920 ) -> np.ndarray:
921     """Exact equal weights at group top-two/third and best-third cutoffs."""
922     advancement = np.zeros(len(points), dtype=float)
923     third_candidates: list[tuple[int, tuple[int, int, int], float]] = []
924     for team_ids in groups.values():
925         blocks: dict[tuple[int, int, int], list[int]] = {}
926         for team_id in team_ids:
927             blocks.setdefault(_rank_key(team_id, points, goals_for, goals_against), []).append(team_id)
928         position = 0
929         for key in sorted(blocks, reverse=True):
930             block = blocks[key]

```

```

929         block_size = len(block)
930         top_slots = max(0, min(position + block_size, 2) - position)
931         if top_slots:
932             advancement[block] += top_slots / block_size
933         if position <= 2 < position + block_size:
934             third_candidates.extend((team_id, key, 1.0 / block_size) for team_id in block)
935         position += block_size
936
937     by_key: dict[tuple[int, int, int], list[tuple[int, float]]] = {}
938     for team_id, key, identity_weight in third_candidates:
939         by_key.setdefault(key, []).append((team_id, identity_weight))
940     remaining = 8.0
941     for key in sorted(by_key, reverse=True):
942         candidates = by_key[key]
943         mass = sum(weight for _, weight in candidates)
944         accepted = min(remaining, mass)
945         if accepted > 0:
946             ratio = accepted / mass
947             for team_id, identity_weight in candidates:
948                 advancement[team_id] += identity_weight * ratio
949             remaining -= accepted
950             if remaining <= 1e-12:
951                 break
952     if abs(float(advancement.sum()) - 32.0) > 1e-9:
953         raise AssertionError(f"Advancement weights sum to {advancement.sum()}, expected 32")
954     return advancement
955
956 def simulate_advancement(
957     data: dict[str, pd.DataFrame], standings: pd.DataFrame,
958     n_sim: int = 20000, seed: int = RNG_SEED,
959 ) -> tuple[dict, pd.DataFrame]:
960     rng = np.random.default_rng(seed)
961     groups_frame = data["group_membership"].sort_values("team_id")
962     team_names = groups_frame["team_name"].tolist()
963     index = {team: i for i, team in enumerate(team_names)}
964     groups = {group: [index[name] for name in names] for group, names in groups_frame.groupby("group_id")["team_name"]}
965     state = standings.set_index("team_name").reindex(team_names)
966     third = data["groups_matches"].query("round_in_group == 3").reset_index(drop=True)
967     base = data["base_predictions"].set_index("match_id")
968     lambda_a, lambda_b = [], []
969     for row in third.itertuples(index=False):
970         sa, sb = float(state.loc[row.team_a, "state"]), float(state.loc[row.team_b, "state"])
971         ha, hb = float(state.loc[row.team_a, "injury"]), float(state.loc[row.team_b, "injury"])
972         delta = 0.32 * (sa - sb) - 0.055 * (ha - hb)
973         base_row = base.loc[row.match_id]
974         lambda_a.append(float(clip(float(base_row.expected_goals_a) * math.exp(delta - 0.04 * ha), 0.15, 4.50)))
975         lambda_b.append(float(clip(float(base_row.expected_goals_b) * math.exp(-delta - 0.04 * hb), 0.15, 4.50)))
976     goals_a = rng.poisson(np.asarray(lambda_a), size=(n_sim, len(third)))
977     goals_b = rng.poisson(np.asarray(lambda_b), size=(n_sim, len(third)))
978     points = np.tile(state["P"].to_numpy(int), (n_sim, 1))
979     goals_for = np.tile(state["GF"].to_numpy(int), (n_sim, 1))
980     goals_against = np.tile(state["GA"].to_numpy(int), (n_sim, 1))
981     outcomes = np.empty((n_sim, len(third)), dtype=np.int8)
982     for j, row in enumerate(third.itertuples(index=False)):
983         ia, ib = index[row.team_a], index[row.team_b]
984         ga, gb = goals_a[:, j], goals_b[:, j]
985         goals_for[:, ia] += ga; goals_against[:, ia] += gb
986         goals_for[:, ib] += gb; goals_against[:, ib] += ga
987         a_win, b_win = ga > gb, gb > ga

```

```

989     draw = ~(a_win | b_win)
990     points[a_win, ia] += 3; points[b_win, ib] += 3
991     points[draw, ia] += 1; points[draw, ib] += 1
992     outcomes[:, j] = np.where(a_win, 1, np.where(b_win, -1, 0))
993     advancement = np.empty((n_sim, len(team_names)), dtype=float)
994     for simulation in range(n_sim):
995         advancement[simulation] = fractional_advancement_weights(
996             points[simulation], goals_for[simulation], goals_against[simulation], groups
997         )
998     probabilities = advancement.mean(axis=0)
999     match_rows = []
1000     for j, row in enumerate(third.itertuples(index=False)):
1001         ia, ib = index[row.team_a], index[row.team_b]
1002         pa, pb = float(probabilities[ia]), float(probabilities[ib])
1003         masks = {"draw": outcomes[:, j] == 0, "a_win": outcomes[:, j] == 1, "b_win": outcomes[:, j] == -1}
1004
1005         def conditional(team_id: int, key: str) -> float:
1006             mask = masks[key]
1007             return float(advancement[mask, team_id].mean()) if mask.any() else float(probabilities[team_id])
1008
1009         pa_draw, pb_draw = conditional(ia, "draw"), conditional(ib, "draw")
1010         pa_win, pb_win = conditional(ia, "a_win"), conditional(ib, "b_win")
1011         stakeless = 0.5 * (2 * pa - 1) ** 2 + 0.5 * (2 * pb - 1) ** 2
1012         draw_benefit = min(pa_draw, pb_draw)
1013         gain = 0.5 * max(pa_win - pa_draw, 0) + 0.5 * max(pb_win - pb_draw, 0)
1014         collusion = float(clip(draw_benefit * (1 - clip(gain)) * (1 - 0.35 * stakeless)))
1015         match_rows.append({
1016             "match_id": row.match_id, "lambda_a": lambda_a[j], "lambda_b": lambda_b[j],
1017             "p_team_a_advance": pa, "p_team_b_advance": pb,
1018             "stakeless_risk": float(stakeless), "collusion_risk": collusion,
1019         })
1020     diagnostics = {
1021         "seed": int(seed), "simulations": int(n_sim),
1022         "tie_method": "exact_equal_fractional_weights_without_team_id_tiebreak",
1023         "advancement_probs": {team: float(probabilities[index[team]]) for team in team_names},
1024         "advancement_probability_sum": float(probabilities.sum()),
1025         "per_simulation_weight_sum_min": float(advancement.sum(axis=1).min()),
1026         "per_simulation_weight_sum_max": float(advancement.sum(axis=1).max()),
1027     }
1028     return diagnostics, pd.DataFrame(match_rows)
1029
1030
1031 def dynamic_p3_environment(
1032     data: dict[str, pd.DataFrame], p2: pd.DataFrame, simulation: pd.DataFrame,
1033     standings: pd.DataFrame,
1034 ) -> pd.DataFrame:
1035     third = p2[p2["round_in_group"].eq(3)].merge(simulation, on="match_id", how="left")
1036     base = data["base_predictions"].set_index("match_id")
1037     live = data["live_group_results"]
1038     state = standings.set_index("team_name")
1039     p2_index = p2.set_index("match_id")
1040     feedback = {}
1041     for team in sorted(set(third["team_a"]) | set(third["team_b"])):
1042         prior = live[(live["team_a"].eq(team)) | (live["team_b"].eq(team))]
1043         attendance_ratios, viewer_ratios = [], []
1044         for row in prior.itertuples(index=False):
1045             attendance_ratios.append(float(clip(
1046                 float(row.attendance) / float(p2_index.loc[row.match_id, "expected_attendance"]), 0.60, 1.50
1047             )))
1048             viewer_ratios.append(float(clip(
1049                 float(row.tv_viewers) / float(p2_index.loc[row.match_id, "expected_tv_viewers"]), 0.60, 1.50
1050             )))
1051         feedback[team] = {

```

```

1052         "attendance": float(clip(np.mean(attendance_ratios), 0.75, 1.25)),
1053         "viewers": float(clip(np.mean(viewer_ratios), 0.75, 1.25)),
1054     }
1055     rows = []
1056     for row in third.itertuples(index=False):
1057         base_row = base.loc[row.match_id]
1058         sa, sb = float(state.loc[row.team_a, "state"]), float(state.loc[row.team_b, "state"])
1059         ha, hb = float(state.loc[row.team_a, "injury"]), float(state.loc[row.team_b, "injury"])
1060         state_mean, injury_scaled = (sa + sb) / 2, (ha + hb) / 6
1061         attendance_feedback = math.sqrt(feedback[row.team_a]["attendance"] * feedback[row.team_b]["attendance"])
1062         viewer_feedback = math.sqrt(feedback[row.team_a]["viewers"] * feedback[row.team_b]["viewers"])
1063         importance = (
1064             4 * row.p_team_a_advance * (1 - row.p_team_a_advance)
1065             + 4 * row.p_team_b_advance * (1 - row.p_team_b_advance)
1066         ) / 2
1067         base_attractiveness = float(base_row.attractiveness_index) / 100
1068         feedback_strength = float(clip((0.5 * attendance_feedback + 0.5 * viewer_feedback - 0.75) / 0.50))
1069         attractiveness = float(clip(
1070             0.50 * base_attractiveness + 0.25 * importance + 0.12 * feedback_strength
1071             + 0.08 * state_mean + 0.05 * (1 - injury_scaled)
1072         ))
1073         n_tilde = (
1074             float(row.expected_attendance) * attendance_feedback * (0.88 + 0.27 * importance)
1075             * (0.94 + 0.12 * state_mean) * (1 - 0.10 * injury_scaled)
1076         )
1077         v_tilde = (
1078             float(row.expected_tv_viewers) * viewer_feedback * (0.87 + 0.30 * importance)
1079             * (0.90 + 0.20 * attractiveness) * (1 - 0.06 * injury_scaled)
1080         )
1081         rows.append({
1082             **row._asdict(), "Q_i": float(importance), "S_i": state_mean, "H_i": injury_scaled,
1083             "feedback_i": feedback_strength, "attractiveness": attractiveness,
1084             "n_base": float(n_tilde), "v_base": float(v_tilde),
1085             "information_state": "after_round2_common_snapshot",
1086         })
1087     return pd.DataFrame(rows)
1088
1089
1090 def static_p3_environment(data: dict[str, pd.DataFrame], p2: pd.DataFrame) -> pd.DataFrame:
1091     base = data["base_predictions"].set_index("match_id")
1092     rows = []
1093     for row in p2[p2["round_in_group"].eq(3)].itertuples(index=False):
1094         base_row = base.loc[row.match_id]
1095         rows.append({
1096             **row._asdict(),
1097             "p_team_a_advance": np.nan, "p_team_b_advance": np.nan,
1098             "stakeless_risk": float(1 - base_row.uncertainty_index),
1099             "collusion_risk": 0.0,
1100             "attractiveness": float(base_row.attractiveness_index) / 100,
1101             "n_base": float(row.expected_attendance),
1102             "v_base": float(row.expected_tv_viewers),
1103             "information_state": "pre_tournament_static",
1104         })
1105     return pd.DataFrame(rows)
1106
1107
1108 def p3_resource_tables(data: dict[str, pd.DataFrame]) -> dict[tuple[str, int], dict[str, float]]:
1109     return {
1110         (row.resource_type, int(row.resource_level)): {
1111             "cost": float(row.unit_cost_index),
1112             "demand": float(row.demand_multiplier),
1113             "risk": float(row.risk_multiplier),

```

```

1114     }
1115     for row in data["dynamic_resource_costs"].itertuples(index=False)
1116 }
1117
1118
1119 def p3_attendance(n_base: float, capacity: float, transport_multiplier: float, elasticity: float, delta:
1120     float) -> float:
1121     return float(min(capacity, n_base * transport_multiplier * (1 + delta) ** elasticity))
1122
1123 def p3_price_candidates(
1124     n_base: float, capacity: float, transport_multiplier: float, elasticity: float,
1125     max_discount: float, max_increase: float,
1126 ) -> list[float]:
1127     low, high = -float(max_discount), float(max_increase)
1128     candidates = {low, 0.0, high}
1129     if elasticity != 0 and n_base * transport_multiplier > 0:
1130         capacity_boundary = (capacity / (n_base * transport_multiplier)) ** (1 / elasticity) - 1
1131         if low <= capacity_boundary <= high:
1132             candidates.add(float(capacity_boundary))
1133     n0 = p3_attendance(n_base, capacity, transport_multiplier, elasticity, 0.0)
1134     # Bisection locates the exact truncated-demand boundary N(delta)=0.88N(0).
1135     if p3_attendance(n_base, capacity, transport_multiplier, elasticity, high) < 0.88 * n0:
1136         left, right = 0.0, high
1137         for _ in range(80):
1138             middle = (left + right) / 2
1139             if p3_attendance(n_base, capacity, transport_multiplier, elasticity, middle) >= 0.88 * n0:
1140                 left = middle
1141             else:
1142                 right = middle
1143             candidates.add(float(left))
1144     return sorted(
1145         delta for delta in candidates
1146         if low - 1e-12 <= delta <= high + 1e-12
1147         and 1 + delta > 0
1148         and p3_attendance(n_base, capacity, transport_multiplier, elasticity, delta) >= 0.88 * n0 - 1e-8
1149     )
1150
1151
1152 def evaluate_p3_action(
1153     data: dict[str, pd.DataFrame], environment_row: pd.Series | object,
1154     broadcast_level: int, security_level: int, transport_level: int,
1155     delta: float,
1156 ) -> dict:
1157     row = environment_row if isinstance(environment_row, pd.Series) else pd.Series(environment_row._asdict()
1158     )
1159     venues = data["venues"].set_index("venue_id")
1160     ticket = data["ticket_broadcast"].set_index("match_stage")
1161     resources = p3_resource_tables(data)
1162     security = data["security_requirements"].set_index("match_id")
1163     venue = venues.loc[row["venue_id"]]
1164     ticket_row = ticket.loc[stage_key(int(row["round_in_group"]))]
1165     broadcast = resources[("broadcast", int(broadcast_level))]
1166     security_resource = resources[("security", int(security_level))]
1167     transport = resources[("transport", int(transport_level))]
1168     attendance = p3_attendance(
1169         float(row["n_base"]), float(venue.capacity), transport["demand"],
1170         float(ticket_row.price_elasticity), float(delta),
1171     )
1172     attendance_zero = p3_attendance(
1173         float(row["n_base"]), float(venue.capacity), transport["demand"],
1174         float(ticket_row.price_elasticity), 0.0,
1175     )

```

```

1175 if attendance < 0.88 * attendance_zero - 1e-7:
1176     raise ValueError(f"Ticket demand retention violated for {row['match_id']}")
1177 ticket_value = float(ticket_row.base_ticket_price_usd) * (1 + float(delta)) * attendance
1178 viewers = float(row["v_base"]) * broadcast["demand"]
1179 broadcast_value = viewers * float(ticket_row.broadcast_unit_value_usd)
1180 pre_resource_occupancy = float(clip(float(row["n_base"]) / float(venue.capacity)))
1181 base_security = float(security.loc[row["match_id"], "security_demand_score"])
1182 security_demand = float(clip(
1183     0.45 * base_security + 0.25 * pre_resource_occupancy
1184     + 0.15 * float(row["attractiveness"])
1185     + 0.15 * (float(row["stakeless_risk"]) + float(row["collusion_risk"])) / 2
1186 ))
1187 risk = (
1188     0.40 * float(row["stakeless_risk"]) + 0.40 * float(row["collusion_risk"])
1189     + 0.20 * security_demand * security_resource["risk"]
1190 )
1191 return {
1192     "match_id": row["match_id"], "reference_date": row["reference_date"],
1193     "broadcast": int(broadcast_level), "security": int(security_level), "transport": int(transport_level
1194 ),
1195     "delta": float(delta), "attendance": attendance, "attendance_at_zero_delta": attendance_zero,
1196     "broadcast_viewers": viewers, "ticket_value": ticket_value, "broadcast_value": broadcast_value,
1197     "attractiveness": float(row["attractiveness"]),
1198     "resource_cost": broadcast["cost"] + security_resource["cost"] + transport["cost"],
1199     "security_demand": security_demand, "pre_resource_occupancy": pre_resource_occupancy,
1200     "risk": float(risk), "information_state": row["information_state"],
1201 }
1202
1203 def build_p3_actions(data: dict[str, pd.DataFrame], environment: pd.DataFrame) -> pd.DataFrame:
1204     venues = data["venues"].set_index("venue_id")
1205     ticket = data["ticket_broadcast"].set_index("match_stage")
1206     limits = data["dynamic_resource_limits"].set_index("reference_date")
1207     resources = p3_resource_tables(data)
1208     rows = []
1209     for environment_row in environment.itertuples(index=False):
1210         venue = venues.loc[environment_row.venue_id]
1211         ticket_row = ticket.loc[stage_key(int(environment_row.round_in_group))]
1212         limit = limits.loc[environment_row.reference_date]
1213         for broadcast_level in (1, 2, 3):
1214             for security_level in range(int(environment_row.required_security_level), int(venue.
1215 security_level) + 1):
1216                 for transport_level in (1, 2, 3):
1217                     transport_multiplier = resources[("transport", transport_level)]["demand"]
1218                     deltas = p3_price_candidates(
1219                         float(environment_row.n_base), float(venue.capacity), transport_multiplier,
1220                         float(ticket_row.price_elasticity), float(limit.max_ticket_discount_rate),
1221                         float(limit.max_ticket_increase_rate),
1222                     )
1223                     evaluated = [
1224                         evaluate_p3_action(
1225                             data, environment_row, broadcast_level, security_level, transport_level, delta
1226                         )
1227                     for delta in deltas
1228                 ]
1229                 rows.append(max(evaluated, key=lambda item: item["ticket_value"]))
1230     return pd.DataFrame(rows)
1231
1232 def p3_normalize(actions: pd.DataFrame, bounds: dict[str, list[float]] | None = None) -> tuple[pd.DataFrame,
1233 dict[str, list[float]]]:
1234     output = actions.copy()
1235     if bounds is None:

```

```

1235     bounds = {field: [float(output[field].min()), float(output[field].max())] for field in P3_FIELDS}
1236     for field in P3_FIELDS:
1237         output[field + "_norm"] = output[field].map(lambda value: minmax_value(value, bounds[field]))
1238     output["net_value"] = (
1239         0.35 * output["ticket_value_norm"] + 0.35 * output["broadcast_value_norm"]
1240         + 0.10 * output["attractiveness_norm"] - 0.10 * output["resource_cost_norm"]
1241         - 0.10 * output["risk_norm"]
1242     )
1243     return output, bounds
1244
1245
1246 def solve_p3_actions(actions: pd.DataFrame, limits: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
1247     actions = actions.reset_index(drop=True)
1248     n = len(actions)
1249     rows, lower, upper = [], [], []
1250
1251     def add(indices: list[int], values: np.ndarray | None, low: float, high: float):
1252         row = np.zeros(n)
1253         row[indices] = 1 if values is None else values
1254         rows.append(row); lower.append(low); upper.append(high)
1255
1256     for indices in actions.groupby("match_id").groups.values():
1257         add(list(indices), None, 1, 1)
1258     limit_index = limits.set_index("reference_date")
1259     for day, indices in actions.groupby("reference_date").groups.items():
1260         indices = list(indices)
1261         limit = limit_index.loc[day]
1262         for field, capacity in [
1263             ("broadcast", "high_broadcast_capacity"),
1264             ("security", "high_security_capacity"),
1265             ("transport", "enhanced_transport_capacity"),
1266         ]:
1267             add(indices, actions.loc[indices, field].ge(3).to_numpy(float), 0, float(limit[capacity]))
1268             add(indices, actions.loc[indices, "resource_cost"].to_numpy(float), 0, float(limit[
1269                 daily_resource_budget_index]))
1270         started = time.perf_counter()
1271         result = milp(
1272             c=-actions["net_value"].to_numpy(float), integrality=np.ones(n),
1273             bounds=Bounds(np.zeros(n), np.ones(n)),
1274             constraints=LinearConstraint(np.vstack(rows), np.asarray(lower), np.asarray(upper)),
1275             options={"time_limit": 120, "mip_rel_gap": 0.0001},
1276         )
1277         if result.x is None:
1278             raise RuntimeError(f"Problem 3 MILP failed: {result.message}")
1279         return actions.loc[np.where(result.x > 0.5)[0]].copy(), _milp_result_diagnostics(result, started)
1280
1281 def evaluate_fixed_p3(
1282     data: dict[str, pd.DataFrame], updated_environment: pd.DataFrame,
1283     fixed_decisions: pd.DataFrame, updated_bounds: dict[str, list[float]],
1284 ) -> pd.DataFrame:
1285     environment = updated_environment.set_index("match_id")
1286     rows = []
1287     for decision in fixed_decisions.itertuples(index=False):
1288         environment_row = environment.loc[decision.match_id].copy()
1289         environment_row["match_id"] = decision.match_id
1290         evaluated = evaluate_p3_action(
1291             data, environment_row, int(decision.broadcast), int(decision.security),
1292             int(decision.transport), float(decision.delta),
1293         )
1294         evaluated["decision_information_state"] = "pre_tournament_static"
1295         rows.append(evaluated)
1296     result, _ = p3_normalize(pd.DataFrame(rows), updated_bounds)

```

```

1297     return result
1298
1299
1300 def p3_constraints_audit(selected: pd.DataFrame, data: dict[str, pd.DataFrame], p2: pd.DataFrame) -> dict:
1301     limits = data["dynamic_resource_limits"].set_index("reference_date")
1302     p2_index = p2.set_index("match_id")
1303     venues = data["venues"].set_index("venue_id")
1304     violations = []
1305     for row in selected.itertuples(index=False):
1306         p2_row = p2_index.loc[row.match_id]
1307         venue = venues.loc[p2_row.venue_id]
1308         if not (int(p2_row.required_security_level) <= int(row.security) <= int(venue.security_level)):
1309             violations.append(f"security:{row.match_id}")
1310         if float(row.attendance) > float(venue.capacity) + 1e-7:
1311             violations.append(f"capacity:{row.match_id}")
1312         if float(row.attendance) < 0.88 * float(row.attendance_at_zero_delta) - 1e-7:
1313             violations.append(f"ticket:{row.match_id}")
1314     for day, subset in selected.groupby("reference_date"):
1315         limit = limits.loc[day]
1316         if subset["broadcast"].ge(3).sum() > int(limit.high_broadcast_capacity):
1317             violations.append(f"broadcast:{day}")
1318         if subset["security"].ge(3).sum() > int(limit.high_security_capacity):
1319             violations.append(f"security_day:{day}")
1320         if subset["transport"].ge(3).sum() > int(limit.enhanced_transport_capacity):
1321             violations.append(f"transport:{day}")
1322         if subset["resource_cost"].sum() > float(limit.daily_resource_budget_index) + 1e-7:
1323             violations.append(f"budget:{day}")
1324     return {"all_passed": not violations, "violations": violations}
1325
1326
1327 def solve_problem3(data: dict[str, pd.DataFrame], p2: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
1328     standings = standings_from_live(data)
1329     static_environment = static_p3_environment(data, p2)
1330     static_actions, static_bounds = p3_normalize(build_p3_actions(data, static_environment))
1331     static_decisions, static_solver = solve_p3_actions(static_actions, data["dynamic_resource_limits"])
1332
1333     seeds = list(P3_STABILITY_SEEDS)
1334     runs = []
1335     primary = None
1336     for seed in seeds:
1337         simulation_diagnostics, simulation = simulate_advancement(data, standings, 20000, seed)
1338         dynamic_environment = dynamic_p3_environment(data, p2, simulation, standings)
1339         dynamic_actions, updated_bounds = p3_normalize(build_p3_actions(data, dynamic_environment))
1340         dynamic_decisions, dynamic_solver = solve_p3_actions(dynamic_actions, data["dynamic_resource_limits"])
1341     ]
1342     static_evaluation = evaluate_fixed_p3(data, dynamic_environment, static_decisions, updated_bounds)
1343     dynamic_total = float(dynamic_decisions["net_value"].sum())
1344     static_total = float(static_evaluation["net_value"].sum())
1345     signature = {
1346         row.match_id: [int(row.broadcast), int(row.security), int(row.transport), round(float(row.delta)
1347 , 8)]
1348     }
1349     for row in dynamic_decisions.itertuples(index=False):
1350         runs.append({
1351             "seed": seed, "advancement_probs": simulation_diagnostics["advancement_probs"],
1352             "dynamic_total_net": dynamic_total, "static_total_net": static_total,
1353             "overall_improvement_rate": None if abs(static_total) < 1e-12 else (dynamic_total - static_total
1354 ) / abs(static_total),
1355             "decision_signature": signature,
1356         })
1357     if primary is None:
1358         primary = (
1359             simulation_diagnostics, dynamic_environment, dynamic_decisions,

```

```

1357         static_evaluation, updated_bounds, dynamic_solver,
1358     )
1359     assert primary is not None
1360     simulation_diagnostics, environment, dynamic, static_eval, updated_bounds, dynamic_solver = primary
1361     dynamic_index, static_index, environment_index = (
1362         dynamic.set_index("match_id"), static_eval.set_index("match_id"), environment.set_index("match_id")
1363     )
1364     rows = []
1365     for match_id in sorted(dynamic_index.index):
1366         dyn, sta, env = dynamic_index.loc[match_id], static_index.loc[match_id], environment_index.loc[
1367             match_id]
1368         static_net = float(sta.net_value)
1369         improvement = np.nan if abs(static_net) < 1e-12 else (float(dyn.net_value) - static_net) / abs(
1370             static_net)
1371         rows.append({
1372             "match_id": match_id, "group_id": env.group_id, "team_a": env.team_a, "team_b": env.team_b,
1373             "updated_p_team_a_advance": round(float(env.p_team_a_advance), 8),
1374             "updated_p_team_b_advance": round(float(env.p_team_b_advance), 8),
1375             "updated_expected_attendance": round(float(dyn.attendance), 2),
1376             "updated_expected_tv_viewers": round(float(dyn.broadcast_viewers), 2),
1377             "stakeless_risk": round(float(env.stakeless_risk), 8),
1378             "collusion_risk": round(float(env.collusion_risk), 8),
1379             "updated_attractiveness": round(float(env.attractiveness), 8),
1380             "recommended_broadcast_priority": int(dyn.broadcast),
1381             "recommended_security_level": int(dyn.security),
1382             "recommended_transport_level": int(dyn.transport),
1383             "recommended_ticket_adjustment": round(float(dyn.delta), 8),
1384             "updated_ticket_revenue_usd": round(float(dyn.ticket_value), 2),
1385             "updated_broadcast_value_usd": round(float(dyn.broadcast_value), 2),
1386             "resource_cost_index": round(float(dyn.resource_cost), 6),
1387             "risk_exposure_index": round(float(dyn.risk), 8),
1388             "static_net_value": round(static_net, 10),
1389             "dynamic_net_value": round(float(dyn.net_value), 10),
1390             "improvement_rate": np.nan if pd.isna(improvement) else round(float(improvement), 10),
1391         })
1392     primary_signature = runs[0]["decision_signature"]
1393     teams = sorted(runs[0]["advancement_probs"])
1394     probability_stability = {
1395         team: {
1396             "mean": float(np.mean([run["advancement_probs"][team] for run in runs])),
1397             "std": float(np.std([run["advancement_probs"][team] for run in runs], ddof=1)),
1398             "min": float(np.min([run["advancement_probs"][team] for run in runs])),
1399             "max": float(np.max([run["advancement_probs"][team] for run in runs])),
1400         }
1401         for team in teams
1402     }
1403     decision_stability = [
1404         float(np.mean([run["decision_signature"][match_id] == primary_signature[match_id] for match_id in
1405             primary_signature]))
1406         for run in runs
1407     ]
1408     output = pd.DataFrame(rows)
1409     diagnostics = {
1410         **simulation_diagnostics,
1411         "problem3_normalization_bounds": updated_bounds,
1412         "problem3_static_pre_normalization_bounds": static_bounds,
1413         "problem3_dynamic_total_net": float(dynamic["net_value"].sum()),
1414         "problem3_static_total_net": float(static_eval["net_value"].sum()),
1415         "problem3_overall_improvement_rate": (
1416             float(dynamic["net_value"].sum() - static_eval["net_value"].sum()) / abs(float(static_eval["
1417             net_value"].sum()))
1418             if abs(float(static_eval["net_value"].sum())) > 1e-12 else None
1419         ),

```

```

1416         "problem3_static_decisions": {
1417             row.match_id: {
1418                 "broadcast": int(row.broadcast), "security": int(row.security), "transport": int(row.
transport),
1419                 "delta": float(row.delta), "selected_using": "pre_tournament_static_information_only",
1420             }
1421             for row in static_decisions.itertuples(index=False)
1422         },
1423         "problem3_solver": {"dynamic": dynamic_solver, "static_pre": static_solver},
1424         "problem3_constraint_audit": {
1425             "dynamic": p3_constraints_audit(dynamic, data, p2),
1426             "static_fixed_in_updated_environment": p3_constraints_audit(static_eval, data, p2),
1427         },
1428         "problem3_robustness": {
1429             "seeds": seeds, "runs": runs,
1430             "advancement_probability_by_team": probability_stability,
1431             "dynamic_decision_match_fraction_vs_primary": decision_stability,
1432             "dynamic_total_net_range": [min(run["dynamic_total_net"] for run in runs), max(run["
dynamic_total_net"] for run in runs)],
1433             "static_total_net_range": [min(run["static_total_net"] for run in runs), max(run["
static_total_net"] for run in runs)],
1434             "overall_improvement_rate_range": [
1435                 min(run["overall_improvement_rate"] for run in runs),
1436                 max(run["overall_improvement_rate"] for run in runs),
1437             ],
1438         },
1439     }
1440     return output, diagnostics
1441
1442
1443     # -----
1444     # Problem 4
1445
1446
1447     def read_entity_mappings() -> dict:
1448         return json.loads((MATERIAL_DIR / "entity_mappings.json").read_text(encoding="utf-8"))
1449
1450
1451     def actual_schedule_2022() -> pd.DataFrame:
1452         source = json.loads((MATERIAL_DIR / "worldcup_2022_openfootball.json").read_text(encoding="utf-8"))
1453         mappings = read_entity_mappings()
1454         aliases = mappings["team_aliases"]
1455         stadium_aliases = mappings["stadium_aliases"]
1456         stadiums = mappings["stadiums"]
1457         source_url = mappings["sources"]["openfootball_file"]
1458         matches = [match for match in source["matches"] if "group" in match]
1459         rows = []
1460         team_round_count: dict[str, int] = {}
1461         for sequence, match in enumerate(sorted(matches, key=lambda item: (item["date"], item["time"], item["
group"]))) :
1462             team_a = aliases.get(match["team1"], match["team1"])
1463             team_b = aliases.get(match["team2"], match["team2"])
1464             round_no = max(team_round_count.get(team_a, 0), team_round_count.get(team_b, 0)) + 1
1465             team_round_count[team_a] = round_no; team_round_count[team_b] = round_no
1466             raw_stadium = match["ground"].split(",")[0].strip()
1467             stadium = stadium_aliases.get(raw_stadium, raw_stadium)
1468             metadata = stadiums[stadium]
1469             local = datetime.fromisoformat(f"{match['date']}T{match['time']}:00").replace(tzinfo=ZoneInfo("Asia/
Qatar"))
1470             utc = local.astimezone(timezone.utc)
1471             score_value = match["score"]
1472             score = score_value["ft"] if isinstance(score_value, dict) else score_value
1473             rows.append({

```

```

1474         "match_id": f"WC2022-{sequence + 1:02d}", "competition": "FIFA World Cup 2022",
1475         "stage": "Group", "group_id": match["group"].replace("Group ", ""),
1476         "round_in_group": round_no, "team_a": team_a, "team_b": team_b,
1477         "venue": stadium, "city": metadata["city"], "country": "Qatar",
1478         "date": local.date().isoformat(), "local_kickoff_time": local.strftime("%H:%M"),
1479         "utc_datetime": utc.strftime("%Y-%m-%d %H:%M"),
1480         "goals_a": int(score[0]), "goals_b": int(score[1]),
1481         "source_url": source_url, "retrieval_date": mappings["retrieval_date"],
1482     })
1483     return pd.DataFrame(rows).sort_values(["group_id", "round_in_group", "utc_datetime"]).reset_index(drop=
1484     True)
1485
1486 def haversine_km(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
1487     radius = 6371.0088
1488     phi1, phi2 = math.radians(lat1), math.radians(lat2)
1489     dphi, dlambda = math.radians(lat2 - lat1), math.radians(lon2 - lon1)
1490     value = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlambda / 2) ** 2
1491     return 2 * radius * math.asin(math.sqrt(value))
1492
1493
1494 def map_actual_matches(
1495     actual: pd.DataFrame, data: dict[str, pd.DataFrame], context: P2Context,
1496     neighbour_rank: int,
1497 ) -> pd.DataFrame:
1498     mappings = read_entity_mappings()
1499     stadiums = mappings["stadiums"]
1500     elo = pd.read_csv(MATERIAL_DIR / "worldcup_2022_pre_tournament_elo.csv").set_index("team_name")
1501     if "USA" in elo.index and "United States" not in elo.index:
1502         elo.loc["United States"] = elo.loc["USA"]
1503     teams = data["teams"].set_index("team_name")
1504     template = context.matches.copy()
1505     template["elo_mean"] = template.apply(lambda row: (teams.loc[row.team_a, "elo_rating"] + teams.loc[row.
1506         team_b, "elo_rating"]) / 2, axis=1)
1507     template["elo_absdiff"] = template.apply(lambda row: abs(teams.loc[row.team_a, "elo_rating"] - teams.loc
1508         [row.team_b, "elo_rating"]), axis=1)
1509     venues = data["venues"]
1510     slots = data["time_slots"].copy()
1511     slot_hours = pd.to_datetime(slots["reference_utc_time"], utc=True).dt.hour + pd.to_datetime(slots["
1512         reference_utc_time"], utc=True).dt.minute / 60
1513     rows = []
1514     for row in actual.itertuples(index=False):
1515         elo_a = float(elo.loc[row.team_a, "elo_rating"])
1516         elo_b = float(elo.loc[row.team_b, "elo_rating"])
1517         mean_elo, diff_elo = (elo_a + elo_b) / 2, abs(elo_a - elo_b)
1518         candidates = template[template["round_in_group"].eq(int(row.round_in_group))].copy()
1519         candidates["mapping_distance"] = abs(candidates["elo_mean"] - mean_elo) / 200 + abs(candidates["
1520             elo_absdiff"] - diff_elo) / 200
1521         matched = candidates.sort_values(["mapping_distance", "match_id", kind="mergesort"]).iloc[min(
1522             neighbour_rank, len(candidates) - 1)]
1523         capacity = float(stadiums[row.venue]["capacity"])
1524         venue_candidates = venues[venues["security_level"].ge(int(matched.required_security_level))].copy()
1525         venue_candidates = venue_candidates.assign(
1526             mapping_distance=(venue_candidates["capacity"] - capacity).abs()
1527         ).sort_values(["mapping_distance", "venue_id", kind="mergesort"])
1528         mapped_venue = venue_candidates.iloc[min(neighbour_rank, len(venue_candidates) - 1)]
1529         actual_utc = pd.to_datetime(row.utc_datetime, utc=True)
1530         actual_hour = actual_utc.hour + actual_utc.minute / 60
1531         circular = np.minimum(abs(slot_hours - actual_hour), 24 - abs(slot_hours - actual_hour))
1532         slot_candidates = slots.assign(mapping_distance=circular).sort_values(
1533             ["mapping_distance", "global_prime_score", "slot_id", ascending=[True, False, True], kind="
1534             mergesort"
1535         )

```

```

1530 mapped_slot = slot_candidates.iloc[min(neighbour_rank, len(slot_candidates) - 1)]
1531 rows.append({
1532     "actual_match_id": row.match_id, "mapped_match_id": matched.match_id,
1533     "mapped_venue_id": mapped_venue.venue_id, "mapped_slot_id": mapped_slot.slot_id,
1534     "match_mapping_distance": float(matched.mapping_distance),
1535     "venue_capacity_difference": float(mapped_venue.mapping_distance),
1536     "slot_hour_difference": float(mapped_slot.mapping_distance),
1537 })
1538 return pd.DataFrame(rows)
1539
1540
1541 def actual_proxy_metrics(
1542     actual: pd.DataFrame, mapping: pd.DataFrame, p1_match: pd.DataFrame,
1543     context: P2Context, data: dict[str, pd.DataFrame],
1544 ) -> dict:
1545     mapped = actual.merge(mapping, left_on="match_id", right_on="actual_match_id")
1546     base = data["base_predictions"].set_index("match_id")
1547     ticket = data["ticket_broadcast"].set_index("match_stage")
1548     venues = data["venues"].set_index("venue_id")
1549     slots = data["time_slots"].set_index("slot_id")
1550     security = data["security_requirements"].set_index("match_id")
1551     viewers = p1_match.set_index("match_id")
1552     venue_metric = context.venue_candidates.set_index(["match_id", "venue_id"])
1553     rows = []
1554     for row in mapped.itertuples(index=False):
1555         matched_id, venue_id, slot_id = row.mapped_match_id, row.mapped_venue_id, row.mapped_slot_id
1556         match = context.matches.set_index("match_id").loc[matched_id]
1557         base_row, venue, slot = base.loc[matched_id], venues.loc[venue_id], slots.loc[slot_id]
1558         ticket_row = ticket.loc[stage_key(int(match.round_in_group))]
1559         vm = venue_metric.xs((matched_id, venue_id))
1560         rows.append({
1561             "actual_match_id": row.match_id, "team_a": row.team_a, "team_b": row.team_b,
1562             "T": float(vm.ticket_revenue_usd),
1563             "B": float(viewers.loc[matched_id, "predicted_tv_viewers"]) * float(ticket_row.
broadcast_unit_value_usd)
1564                 * float(slot.global_prime_score) * float(ticket_row.sponsor_weight),
1565             "U": float(base_row.uncertainty_index), "H": float(base_row.attractiveness_index) / 100,
1566             "C_match": float(venue.operation_cost_musd_per_match)
1567                 + 0.1 * int(security.loc[matched_id, "required_security_level"]) * float(venue.
security_cost_index),
1568             "D": float(vm.travel_burden), "R": float(vm.risk_index),
1569             "mapped_venue_id": venue_id, "mapped_slot_id": slot_id,
1570         })
1571     detail = pd.DataFrame(rows)
1572     fairness, fairness_detail = p2_fairness(
1573         detail.rename(columns={"mapped_venue_id": "venue_id", "mapped_slot_id": "slot_id"}),
1574         context.prime_slots, context.large_venues,
1575     )
1576     setup = float(venues.loc[detail["mapped_venue_id"].unique(), "setup_cost_musd"].sum())
1577     n = len(detail)
1578     raw = {
1579         "T": float(detail["T"].sum()), "B": float(detail["B"].sum()),
1580         "U": float(detail["U"].mean()), "H": float(detail["H"].mean()),
1581         "C": setup + float(detail["C_match"].sum()), "D": float(detail["D"].mean()),
1582         "F": fairness, "R": float(detail["R"].mean()),
1583     }
1584     bounds = {
1585         "T": [n * context.bounds["ticket_revenue_usd"][0], n * context.bounds["ticket_revenue_usd"][1]],
1586         "B": [n * context.bounds["broadcast_value_usd"][0], n * context.bounds["broadcast_value_usd"][1]],
1587         "U": [0, 1], "H": [0, 1],
1588         "C": [setup + n * context.bounds["match_cost_musd"][0], setup + n * context.bounds["match_cost_musd"
] [1]],
1589         "D": [0, 1], "F": [0, 1], "R": context.bounds["risk_index"],

```

```

1590     }
1591     normalized = {key: minmax_value(raw[key], bounds[key]) for key in raw}
1592     contributions = {key: P2_WEIGHTS[key] * normalized[key] for key in raw}
1593     return {
1594         "raw": raw, "per_match": {"T": raw["T"] / n, "B": raw["B"] / n, "C": raw["C"] / n},
1595         "bounds": bounds, "normalized": normalized, "weighted_contributions": contributions,
1596         "Z2": float(sum(contributions.values())), "fairness_detail": fairness_detail,
1597         "detail": detail.to_dict(orient="records"),
1598     }
1599
1600
1601 def schedule_structure_metrics(
1602     schedule: pd.DataFrame, venue_column: str, date_column: str, utc_column: str,
1603     venue_metadata: dict[str, dict], prime_flags: dict[str, bool],
1604 ) -> dict[str, float]:
1605     frame = schedule.copy()
1606     frame["_utc"] = pd.to_datetime(frame[utc_column], utc=True)
1607     rests, travel, timezone_transitions = [], 0.0, 0
1608     teams = sorted(set(frame["team_a"]) | set(frame["team_b"]))
1609     for team in teams:
1610         subset = frame[(frame["team_a"].eq(team)) | (frame["team_b"].eq(team))].sort_values("_utc")
1611         rests.extend((subset["_utc"].diff().dropna().dt.total_seconds() / 3600).tolist())
1612         previous = None
1613         for row in subset.itertuples(index=False):
1614             metadata = venue_metadata[getattr(row, venue_column)]
1615             if previous is not None:
1616                 travel += haversine_km(previous["latitude"], previous["longitude"], metadata["latitude"],
1617                                         metadata["longitude"])
1618                 timezone_transitions += int(previous["timezone"] != metadata["timezone"])
1619             previous = metadata
1620     dates = pd.to_datetime(frame[date_column])
1621     span = int((dates.max() - dates.min()).days + 1)
1622     venue_count = frame[venue_column].nunique()
1623     active_venue_days = frame.groupby([venue_column, date_column]).ngroups
1624     prime_counts = {team: 0 for team in teams}
1625     for row in frame.itertuples(index=False):
1626         if prime_flags[getattr(row, "match_id")]:
1627             prime_counts[row.team_a] += 1; prime_counts[row.team_b] += 1
1628     return {
1629         "match_count": float(len(frame)), "team_count": float(len(teams)), "venue_count": float(venue_count)
1630     },
1631     "schedule_span_days": float(span), "total_travel_km": float(travel),
1632     "avg_travel_per_team_km": float(travel / len(teams)),
1633     "avg_travel_per_match_km": float(travel / len(frame)),
1634     "cross_timezone_transitions": float(timezone_transitions),
1635     "average_rest_hours": float(np.mean(rests)), "minimum_rest_hours": float(np.min(rests)),
1636     "rest_hours_std": float(np.std(rests, ddof=1)),
1637     "matches_per_venue": float(len(frame) / venue_count),
1638     "matches_per_active_venue_day": float(len(frame) / active_venue_days),
1639     "venue_active_day_utilization": float(active_venue_days / (venue_count * span)),
1640     "prime_time_share": float(np.mean([prime_flags[mid] for mid in frame["match_id"]])),
1641     "prime_matches_per_team": float(sum(prime_counts.values()) / len(teams)),
1642     "prime_count_range": float(max(prime_counts.values()) - min(prime_counts.values())),
1643 }
1644
1645 def _comparison_row(name: str, category: str, actual_value: float, optimized_value: float, direction: str)
1646     -> dict:
1647     difference = float(optimized_value) - float(actual_value)
1648     if direction == "larger_better":
1649         relative = np.nan if abs(actual_value) < 1e-12 else difference / abs(actual_value)
1650         result = "优化赛程更优" if difference > 1e-12 else "实际赛程更优" if difference < -1e-12 else "基本相同"

```

```

1649 elif direction == "smaller_better":
1650     relative = np.nan if abs(actual_value) < 1e-12 else -difference / abs(actual_value)
1651     result = "优化赛程更优" if difference < -1e-12 else "实际赛程更优" if difference > 1e-12 else "基本相
        同"
1652 else:
1653     relative, result = np.nan, "规模或描述指标, 不作直接优劣判定"
1654 return {
1655     "indicator_name": name, "indicator_category": category,
1656     "actual_schedule_value": actual_value, "optimized_schedule_value": optimized_value,
1657     "absolute_difference": difference, "relative_improvement": relative,
1658     "preferred_direction": direction, "evaluation_result": result,
1659 }
1660
1661
1662 def solve_problem4(
1663     data: dict[str, pd.DataFrame], p1_match: pd.DataFrame, p2: pd.DataFrame,
1664     context: P2Context,
1665 ) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
1666     actual = actual_schedule_2022()
1667     first_mapping = map_actual_matches(actual, data, context, 0)
1668     second_mapping = map_actual_matches(actual, data, context, 1)
1669     first_proxy = actual_proxy_metrics(actual, first_mapping, p1_match, context, data)
1670     second_proxy = actual_proxy_metrics(actual, second_mapping, p1_match, context, data)
1671     optimized_proxy = p2_metrics(p2, context, data)
1672     mappings = read_entity_mappings()
1673     actual_venues = mappings["stadiums"]
1674     optimized_venues = {
1675         row.venue_id: {
1676             "latitude": float(row.latitude), "longitude": float(row.longitude), "timezone": str(row.timezone)
1677         },
1678         "capacity": int(row.capacity),
1679     }
1680     for row in data["venues"].itertuples(index=False)
1681 }
1682 first_slot_flags = dict(zip(
1683     first_mapping["actual_match_id"], first_mapping["mapped_slot_id"].astype(str).isin(context.
1684     prime_slots)
1685 ))
1686 second_slot_flags = dict(zip(
1687     second_mapping["actual_match_id"], second_mapping["mapped_slot_id"].astype(str).isin(context.
1688     prime_slots)
1689 ))
1690 optimized_slot_flags = dict(zip(p2["match_id"], p2["slot_id"].astype(str).isin(context.prime_slots)))
1691 actual_structure = schedule_structure_metrics(
1692     actual, "venue", "date", "utc_datetime", actual_venues, first_slot_flags
1693 )
1694 optimized_structure = schedule_structure_metrics(
1695     p2, "venue_id", "reference_date", "utc_datetime", optimized_venues, optimized_slot_flags
1696 )
1697 base = data["base_predictions"].set_index("match_id")
1698 actual_meta = actual.set_index("match_id")
1699 actual_capacity_ratios, actual_attendance = [], []
1700 for mapping in first_mapping.itertuples(index=False):
1701     expected = float(base.loc[mapping.mapped_match_id, "expected_attendance_base"])
1702     capacity = float(actual_venues[actual_meta.loc[mapping.actual_match_id, "venue"]]["capacity"])
1703     actual_attendance.append(min(expected, capacity))
1704     actual_capacity_ratios.append(min(expected, capacity) / capacity)
1705 optimized_capacity = p2.merge(data["venues"][["venue_id", "capacity"]], on="venue_id")
1706 actual_structure["capacity_match_ratio"] = float(np.mean(actual_capacity_ratios))
1707 actual_structure["expected_attendance_per_match"] = float(np.mean(actual_attendance))
1708 optimized_structure["capacity_match_ratio"] = float((optimized_capacity["expected_attendance"] /
1709     optimized_capacity["capacity"]).mean())
1710 optimized_structure["expected_attendance_per_match"] = float(p2["expected_attendance"].mean())

```

```

1707 directions = {
1708     "match_count": "neutral", "team_count": "neutral", "venue_count": "neutral", "schedule_span_days": "
neutral",
1709     "total_travel_km": "smaller_better", "avg_travel_per_team_km": "smaller_better",
1710     "avg_travel_per_match_km": "smaller_better", "cross_timezone_transitions": "smaller_better",
1711     "average_rest_hours": "neutral", "minimum_rest_hours": "larger_better", "rest_hours_std": "
smaller_better",
1712     "matches_per_venue": "neutral", "matches_per_active_venue_day": "neutral",
1713     "venue_active_day_utilization": "larger_better", "prime_time_share": "larger_better",
1714     "prime_matches_per_team": "neutral", "prime_count_range": "smaller_better",
1715     "capacity_match_ratio": "larger_better", "expected_attendance_per_match": "larger_better",
1716 }
1717 rows = [
1718     _comparison_row(name, "structure", actual_structure[name], optimized_structure[name], directions[
name])
1719     for name in directions
1720 ]
1721 proxy_names = {
1722     "T": "proxy_ticket_value_per_match_usd", "B": "proxy_broadcast_value_per_match_usd",
1723     "U": "proxy_uncertainty_mean", "H": "proxy_attractiveness_mean",
1724     "C": "proxy_cost_per_match_musd", "D": "proxy_travel_burden",
1725     "F": "proxy_fairness_penalty", "R": "proxy_execution_risk",
1726 }
1727 for key, name in proxy_names.items():
1728     if key in ("T", "B", "C"):
1729         actual_value = first_proxy["per_match"][key]
1730         optimized_value = optimized_proxy["raw"][key] / len(p2)
1731     else:
1732         actual_value = first_proxy["raw"][key]
1733         optimized_value = optimized_proxy["raw"][key]
1734     direction = "larger_better" if key in ("T", "B", "U", "H") else "smaller_better"
1735     rows.append(_comparison_row(name, "proxy_nearest", actual_value, optimized_value, direction))
1736 rows.append(_comparison_row("proxy_Z2", "proxy_nearest", first_proxy["Z2"], optimized_proxy["Z2"], "
larger_better"))
1737 rows.append(_comparison_row("proxy_Z2_second_nearest", "proxy_sensitivity", second_proxy["Z2"],
optimized_proxy["Z2"], "larger_better"))
1738 second_prime_share = float(np.mean([second_slot_flags[mid] for mid in actual["match_id"]]))
1739 rows.append(_comparison_row(
1740     "prime_time_share_second_nearest", "proxy_sensitivity",
1741     second_prime_share, optimized_structure["prime_time_share"], "larger_better",
1742 ))
1743 diagnostics = {
1744     "problem4_sources": mappings["sources"],
1745     "problem4_entity_mapping_file": str(MATERIAL_DIR / "entity_mappings.json"),
1746     "problem4_mapping_method": "same-round nearest Elo mean/absolute-difference; venue by capacity; slot
by circular UTC hour",
1747     "problem4_first_neighbour_mapping": first_mapping.to_dict(orient="records"),
1748     "problem4_second_neighbour_mapping": second_mapping.to_dict(orient="records"),
1749     "problem4_structure": {"actual": actual_structure, "optimized": optimized_structure},
1750     "problem4_proxy": {
1751         "actual_nearest": first_proxy, "actual_second_nearest": second_proxy,
1752         "optimized": optimized_proxy,
1753         "direction_reversal": bool(
1754             np.sign(optimized_proxy["Z2"] - first_proxy["Z2"])
1755             != np.sign(optimized_proxy["Z2"] - second_proxy["Z2"])
1756         ),
1757     },
1758     "problem4_scale_basis": {
1759         "T_B_C": "per_match_for_cross_size_display; each schedule normalized with the same candidate
bounds",
1760         "travel": "within-tournament venue-to-venue transitions, excluding travel to first match",
1761         "rest": "per-team consecutive-match UTC differences",
1762     },

```

```

1763     }
1764     return actual, pd.DataFrame(rows), diagnostics

```

D.3 自动验收程序

```

1  #!/usr/bin/env python3
2  """Formula-level acceptance checks for the revised Problem C pipeline."""
3
4  from __future__ import annotations
5
6  import argparse
7  import hashlib
8  import importlib.metadata
9  import json
10 import math
11 import platform
12 import re
13 from pathlib import Path
14 from zoneinfo import ZoneInfo
15
16 import numpy as np
17 import pandas as pd
18
19 from pipeline_core import (
20     MATERIAL_DIR,
21     P1_NUMERIC_FEATURES,
22     RANDOM_SEEDS,
23     ROOT,
24     TEMPLATE_DIR,
25     actual_schedule_2022,
26     build_p2_context,
27     build_p3_actions,
28     build_prediction_features,
29     competition_normalization_diagnostics,
30     dynamic_p3_environment,
31     evaluate_p3_action,
32     exact_top_quartile_ids,
33     p2_hard_constraint_audit,
34     p2_metrics,
35     p3_constraints_audit,
36     p3_normalize,
37     read_data,
38     normalize_competition_name,
39     simulate_advancement,
40     solve_p3_actions,
41     solve_problem1,
42     standings_from_live,
43     static_p3_environment,
44     swap_match_sides,
45     utc_timestamp,
46 )
47
48
49 TEMPLATE_NAMES = {
50     "result_1_test_prediction.csv": "result_1_test_prediction_template.csv",
51     "result_1_match_prediction.csv": "result_1_match_prediction_template.csv",
52     "result_2_group_schedule.csv": "result_2_template.csv",
53     "result_3_dynamic_strategy.csv": "result_3_template.csv",
54     "actual_schedule_2022.csv": "actual_schedule_template.csv",
55     "result_4_schedule_comparison.csv": "result_4_template.csv",
56 }
57
58

```

```

59 def parse_args() -> argparse.Namespace:
60     parser = argparse.ArgumentParser()
61     parser.add_argument("--output-dir", default="output_v2")
62     return parser.parse_args()
63
64
65 def fail(message: str) -> None:
66     raise AssertionError(message)
67
68
69 def assert_true(condition: bool, message: str) -> None:
70     if not condition:
71         fail(message)
72
73
74 def assert_close(actual: float, expected: float, message: str, atol=1e-6, rtol=1e-8) -> None:
75     if not np.isclose(float(actual), float(expected), atol=atol, rtol=rtol):
76         fail(f"{message}: actual={actual}, expected={expected}")
77
78
79 def finite_nonnegative(series: pd.Series, name: str) -> None:
80     values = pd.to_numeric(series, errors="coerce")
81     assert_true(values.notna().all() and np.isfinite(values).all(), f"{name} contains non-finite values")
82     assert_true((values >= 0).all(), f"{name} contains negative values")
83
84
85 class Acceptance:
86     def __init__(self, output_dir: Path):
87         self.output_dir = output_dir
88         self.data = read_data()
89         self.diagnostics = json.loads((output_dir / "diagnostics.json").read_text(encoding="utf-8"))
90         self.output: dict[str, pd.DataFrame] = {}
91
92     def check_environment(self) -> None:
93         requirements = ROOT / "requirements.txt"
94         assert_true(requirements.is_file(), "requirements.txt is missing")
95         expected_packages = {
96             "numpy": "2.5.2",
97             "pandas": "3.0.5",
98             "scipy": "1.18.0",
99             "scikit-learn": "1.9.0",
100             "openpyxl": "3.1.5",
101             "joblib": "1.5.3",
102             "threadpoolctl": "3.6.0",
103         }
104         actual_packages = {
105             name: importlib.metadata.version(name) for name in expected_packages
106         }
107         assert_true(actual_packages == expected_packages, f"Runtime package versions differ: {
actual_packages}")
108         environment = self.diagnostics["environment"]
109         assert_true(environment["python"] == platform.python_version(), "Recorded Python version mismatch")
110         assert_true(environment["packages"] == actual_packages, "Recorded dependency versions mismatch")
111         assert_true(
112             environment["requirements_sha256"] == hashlib.sha256(requirements.read_bytes()).hexdigest(),
113             "Recorded requirements.txt hash mismatch",
114         )
115         assert_true(self.diagnostics["random_seeds"] == RANDOM_SEEDS, "Recorded random seeds mismatch")
116
117     def read_outputs(self) -> None:
118         for output_name, template_name in TEMPLATE_NAMES.items():
119             output = pd.read_csv(self.output_dir / output_name)
120             template = pd.read_csv(TEMPLATE_DIR / template_name)

```

```

121         assert_true(
122             list(output.columns) == list(template.columns),
123             f"{output_name} columns differ from official template",
124         )
125         self.output[output_name] = output
126
127     def check_problem1(self) -> None:
128         test = self.output["result_1_test_prediction.csv"]
129         future = self.output["result_1_match_prediction.csv"]
130         historical_test = self.data["historical_matches"].query("dataset_split == 'test'")
131         groups = self.data["groups_matches"]
132         assert_true(len(test) == 140 and test["match_id_test"].is_unique, "P1 test must have 140 unique IDs"
133     )
134         assert_true(test["match_id_test"].tolist() == historical_test["match_id"].tolist(), "P1 test IDs/
135         order mismatch")
136         assert_true(len(future) == 72 and future["match_id"].is_unique, "P1 future must have 72 unique
137         matches")
138         assert_true(
139             future[["match_id", "team_a", "team_b"]].reset_index(drop=True).equals(
140                 groups[["match_id", "team_a", "team_b"]].reset_index(drop=True)
141             ),
142             "P1 future match/team identity mismatch",
143         )
144         finite_nonnegative(test["predicted_test_tv_viewers"], "P1 test predictions (million)")
145         finite_nonnegative(future["predicted_tv_viewers"], "P1 future predictions (persons)")
146         assert_true(test["predicted_test_tv_viewers"].mean() < 1000, "P1 test output is not in millions")
147         assert_true(future["predicted_tv_viewers"].mean() > 1_000_000, "P1 future output is not in persons")
148
149         forbidden = {"attractiveness_index", "commercial_value_index", "expected_attendance_base", "
150         uncertainty_index"}
151         recorded = self.diagnostics["problem1_feature_columns"]
152         assert_true(not forbidden.intersection(recorded["numeric"] + recorded["categorical"]), "P1 forbidden
153         features remain")
154         assert_true(recorded["numeric"] == P1_NUMERIC_FEATURES, "P1 numeric feature dictionary mismatch")
155         assert_true(
156             self.diagnostics["problem1_feature_builder"]
157             == "build_prediction_features_v3_symmetric_competition_normalized",
158             "P1 feature builder signature mismatch",
159         )
160
161         regenerated_test, regenerated_future, regenerated_diag = solve_problem1(self.data)
162         assert_true(
163             np.allclose(test["predicted_test_tv_viewers"], regenerated_test["predicted_test_tv_viewers"],
164                 atol=5e-7),
165             "P1 test CSV differs from regenerated model output",
166         )
167         assert_true(
168             np.allclose(future["predicted_tv_viewers"], regenerated_future["predicted_tv_viewers"], atol=0),
169             "P1 future CSV differs from regenerated model output",
170         )
171         assert_true(regenerated_diag["problem1_swap_invariance_max_abs_error_million"] <= 1e-10, "P1 side-
172         swap invariance failed")
173         assert_close(
174             self.diagnostics["problem1_swap_invariance_max_abs_error_million"],
175             regenerated_diag["problem1_swap_invariance_max_abs_error_million"],
176             "P1 recorded swap error mismatch", atol=1e-12,
177         )
178         recorded_metrics = {
179             row["model"]: row for row in self.diagnostics["problem1_model_metrics"]
180         }
181         regenerated_metrics = {
182             row["model"]: row for row in regenerated_diag["problem1_model_metrics"]
183         }

```

```

177     assert_true(recorded_metrics.keys() == regenerated_metrics.keys(), "P1 model metric names mismatch")
178     for model_name in recorded_metrics:
179         for metric in ("mse", "rmse", "mae"):
180             assert_close(
181                 recorded_metrics[model_name][metric],
182                 regenerated_metrics[model_name][metric],
183                 f"P1 {model_name} {metric} diagnostic mismatch",
184                 atol=1e-10,
185             )
186     train = self.data["historical_matches"].query("dataset_split == 'train' and tv_viewers == tv_viewers
187 ")
188     raw_future = groups.merge(
189         self.data["base_predictions"].drop(columns=["group_id", "round_in_group", "team_a", "team_b"],
190 errors="ignore"),
191         on="match_id", how="left",
192     )
193     train_features = build_prediction_features(train, self.data["teams"])
194     historical_test_raw = self.data["historical_matches"].query("dataset_split == 'test'").copy()
195     category_diagnostics = competition_normalization_diagnostics(train, historical_test_raw, raw_future)
196     assert_true(
197         category_diagnostics == self.diagnostics["problem1_competition_normalization"],
198         "P1 competition normalization diagnostics mismatch",
199     )
200     assert_true(
201         {normalize_competition_name(value) for value in (
202             "World Cup", "world cup", "FIFA World Cup", "FIFA World Cup 2026", "WORLD-CUP 2026",
203         )} == {"World Cup"},
204         "P1 World Cup aliases do not normalize to one category",
205     )
206     assert_true(
207         normalize_competition_name("World Cup Qualifier") == "World Cup Qualifier",
208         "P1 qualifier was incorrectly merged with World Cup",
209     )
210     assert_true(
211         category_diagnostics["future72"]["after_categories"] == ["World Cup"],
212         "P1 future World Cup category does not match training category",
213     )
214     assert_true(
215         category_diagnostics["test"]["unseen_row_count"] == 0
216         and category_diagnostics["future72"]["unseen_row_count"] == 0,
217         "P1 normalized test/future contains unseen competition rows",
218     )
219     swapped = build_prediction_features(swap_match_sides(raw_future), self.data["teams"])
220     normal = build_prediction_features(raw_future, self.data["teams"])
221     assert_true(np.allclose(normal[P1_NUMERIC_FEATURES], swapped[P1_NUMERIC_FEATURES], atol=1e-12), "P1
222 symmetric features change after side swap")
223     for column in P1_NUMERIC_FEATURES:
224         stats = self.diagnostics["problem1_feature_distributions"][column]
225         mean, std = train_features[column].mean(), train_features[column].std(ddof=1)
226         assert_close(stats["train"]["mean"], mean, f"P1 {column} train mean diagnostic mismatch")
227         assert_close(stats["train_mean_minus_4sd"], mean - 4 * std, f"P1 {column} lower 4sd bound
228 mismatch")
229
230 def check_problem2(self) -> None:
231     output = self.output["result_2_group_schedule.csv"]
232     groups = self.data["groups_matches"]
233     venues = self.data["venues"].set_index("venue_id")
234     slots = self.data["time_slots"].set_index("slot_id")
235     assert_true(len(output) == 72 and output["match_id"].is_unique, "P2 must contain 72 unique matches")
236     identity_columns = ["match_id", "group_id", "round_in_group", "team_a", "team_b"]
237     assert_true(
238         output[identity_columns].reset_index(drop=True).equals(groups[identity_columns].reset_index(drop
239 =True))),

```

```

235     "P2 fixed match identity/order differs from groups_matches",
236 )
237 p1 = self.output["result_1_match_prediction.csv"]
238 context = build_p2_context(self.data, p1)
239 for row in output.itertuples(index=False):
240     venue, slot = venues.loc[row.venue_id], slots.loc[row.slot_id]
241     assert_true(row.city == venue.city and row.country == venue.country, f"P2 venue city/country
mismatch {row.match_id}")
242     assert_true(str(row.reference_date) == pd.to_datetime(slot.date).date().isoformat(), f"P2
reference date mismatch {row.match_id}")
243     assert_true(str(row.reference_kickoff_time)[:5] == str(slot.reference_kickoff_time)[:5], f"P2
kickoff mismatch {row.match_id}")
244     utc = utc_timestamp(slot.reference_utc_time)
245     local = utc.tz_convert(ZoneInfo(str(venue.timezone)))
246     assert_true(str(row.utc_datetime) == utc.strftime("%Y-%m-%d %H:%M"), f"P2 UTC mismatch {row.
match_id}")
247     assert_true(str(row.local_datetime) == local.strftime("%Y-%m-%d %H:%M"), f"P2 local datetime
mismatch {row.match_id}")
248     vm = context.venue_candidates.set_index(["match_id", "venue_id"]).xs((row.match_id, row.venue_id
))
249     sm = context.slot_candidates.set_index(["match_id", "slot_id"]).xs((row.match_id, row.slot_id))
250     assert_close(row.expected_attendance, vm.expected_attendance, f"P2 attendance formula mismatch {
row.match_id}", atol=0.011)
251     assert_close(row.ticket_revenue_usd, vm.ticket_revenue_usd, f"P2 T formula mismatch {row.
match_id}", atol=0.011)
252     assert_close(row.broadcast_value_usd, sm.broadcast_value_usd, f"P2 B formula mismatch {row.
match_id}", atol=0.011)
253     assert_close(row.travel_cost_index, vm.travel_burden, f"P2 D formula mismatch {row.match_id}",
atol=5e-9)
254     assert_close(row.risk_index, vm.risk_index, f"P2 R formula mismatch {row.match_id}", atol=5e-9)
255     audit = p2_hard_constraint_audit(output, context, self.data)
256     assert_true(audit["all_passed"], f"P2 hard constraints failed: {audit}")
257     independent_group_gaps = {}
258     schedule_times = output.assign(_utc=pd.to_datetime(output["utc_datetime"], utc=True))
259     for group_id, group_matches in schedule_times.groupby("group_id"):
260         for round_no in (1, 2):
261             previous_max = group_matches.loc[
262                 group_matches["round_in_group"].eq(round_no), "_utc"
263             ].max()
264             next_min = group_matches.loc[
265                 group_matches["round_in_group"].eq(round_no + 1), "_utc"
266             ].min()
267             key = f"{group_id}:R{round_no}-R{round_no + 1}"
268             independent_group_gaps[key] = float((next_min - previous_max).total_seconds() / 3600)
269     group_violation_count = sum(gap < 60 - 1e-9 for gap in independent_group_gaps.values())
270     group_minimum = min(independent_group_gaps.values())
271     assert_true(group_violation_count == 0, f"P2 group round boundary violations: {group_violation_count
}")
272     assert_true(
273         independent_group_gaps == audit["group_round_gaps_hours"],
274         "P2 group round gaps differ from hard-constraint audit",
275     )
276     assert_close(
277         audit["group_round_minimum_gap_hours"], group_minimum,
278         "P2 group round minimum gap mismatch",
279     )
280     assert_true(
281         audit["group_round_violation_count"] == group_violation_count
282         and audit["group_round_boundary_passed"],
283         "P2 group round boundary audit state mismatch",
284     )
285     self.group_round_summary = {
286         "gaps": independent_group_gaps,

```

```

287         "minimum": group_minimum,
288         "violations": int(group_violation_count),
289         "all_passed": bool(audit["all_passed"]),
290     }
291     assert_true(output.duplicated(["venue_id", "utc_datetime"]).sum() == 0, "P2 venue-UTC conflict")
292     for utc_value, subset in output.groupby("utc_datetime"):
293         capacities = [int(slots.loc[slot_id, "broadcast_capacity"]) for slot_id in subset["slot_id"]]
294         assert_true(len(set(capacities)) == 1 and len(subset) <= capacities[0], f"P2 real UTC broadcast
capacity exceeded {utc_value}")
295         prime_slots = exact_top_quartile_ids(self.data["time_slots"], "global_prime_score", "slot_id")
296         assert_true(len(prime_slots) == 20 and prime_slots == set(self.diagnostics["problem2_prime_slots"]),
"P2 prime slots are not exact top 20")
297         metrics = p2_metrics(output, context, self.data)
298         recorded = self.diagnostics["problem2_indicator_details"]
299         for section in ("raw", "normalized", "weighted_contributions"):
300             for key, expected in metrics[section].items():
301                 assert_close(recorded[section][key], expected, f"P2 diagnostic {section}.{key} mismatch",
atol=1e-9)
302         objective_values = pd.to_numeric(output["total_objective_value"], errors="coerce").dropna()
303         assert_true(len(objective_values) == 1, "P2 objective must appear only in the first row")
304         assert_close(objective_values.iloc[0], metrics["Z2"], "P2 Z2 mismatch", atol=5e-10)
305         solver = self.diagnostics["problem2_solver"]
306         assert_true(solver["global_optimality_claim"] is False, "P2 incorrectly claims global 3D optimum")
307         valid_statuses = {"optimal_within_mip_tolerance", "limit_reached"}
308         assert_true(solver["slot_subproblem"]["status"] in valid_statuses, "P2 slot solver status invalid")
309         assert_true(solver["venue_subproblem_given_slots"]["status"] in valid_statuses, "P2 venue solver
status invalid")
310         for subproblem in ("slot_subproblem", "venue_subproblem_given_slots"):
311             detail = solver[subproblem]
312             if detail["solver_status"] == "optimal":
313                 assert_true(
314                     detail["interpretation"] == "在设定 MIP 容差内求得最优解",
315                     f"P2 {subproblem} optimal status wording mismatch",
316                 )
317         self.p2_context = context
318
319     def check_problem3(self) -> None:
320         output = self.output["result_3_dynamic_strategy.csv"]
321         p2 = self.output["result_2_group_schedule.csv"]
322         third_ids = set(p2.loc[p2["round_in_group"].eq(3), "match_id"])
323         assert_true(len(output) == 24 and set(output["match_id"]) == third_ids, "P3 third-round match set
mismatch")
324         for column in [
325             "updated_p_team_a_advance", "updated_p_team_b_advance", "stakeless_risk",
326             "collusion_risk", "updated_attractiveness", "risk_exposure_index",
327         ]:
328             values = pd.to_numeric(output[column], errors="coerce")
329             assert_true(values.notna().all() and values.between(0, 1).all(), f"P3 {column} outside [0,1]")
330             source = (ROOT / "code" / "pipeline_core.py").read_text(encoding="utf-8")
331             assert_true("-team_index" not in source and "team_index" not in source, "P3 contains team-index tie
break")
332             assert_true(
333                 self.diagnostics["tie_method"] == "exact_equal_fractional_weights_without_team_id_tiebreak",
334                 "P3 tie method diagnostic mismatch",
335             )
336             assert_close(self.diagnostics["advancement_probability_sum"], 32, "P3 advancement probability sum",
atol=1e-10)
337             assert_close(self.diagnostics["per_simulation_weight_sum_min"], 32, "P3 per-simulation min weight",
atol=1e-10)
338             assert_close(self.diagnostics["per_simulation_weight_sum_max"], 32, "P3 per-simulation max weight",
atol=1e-10)
339
340         standings = standings_from_live(self.data)

```

```

341     seed = int(self.diagnostics["seed"])
342     simulation_diag, simulation = simulate_advancement(self.data, standings, 20000, seed)
343     assert_close(sum(simulation_diag["advancement_probs"].values()), 32, "P3 regenerated advancement sum
", atol=1e-10)
344     environment = dynamic_p3_environment(self.data, p2, simulation, standings)
345     environment_index = environment.set_index("match_id")
346     bounds = self.diagnostics["problem3_normalization_bounds"]
347     dynamic_rows = []
348     for row in output.itertuples(index=False):
349         env = environment_index.loc[row.match_id].copy(); env["match_id"] = row.match_id
350         evaluated = evaluate_p3_action(
351             self.data, env, int(row.recommended_broadcast_priority), int(row.recommended_security_level)
352             ,
353             int(row.recommended_transport_level), float(row.recommended_ticket_adjustment),
354         )
355         dynamic_rows.append(evaluated)
356         assert_close(row.updated_p_team_a_advance, env.p_team_a_advance, f"P3 advancement A mismatch {
row.match_id}", atol=5e-9)
357         assert_close(row.updated_p_team_b_advance, env.p_team_b_advance, f"P3 advancement B mismatch {
row.match_id}", atol=5e-9)
358         assert_close(row.updated_expected_attendance, evaluated["attendance"], f"P3 attendance mismatch
{row.match_id}", atol=0.011)
359         assert_close(row.updated_expected_tv_viewers, evaluated["broadcast_viewers"], f"P3 viewers
mismatch {row.match_id}", atol=0.011)
360         assert_close(row.updated_ticket_revenue_usd, evaluated["ticket_value"], f"P3 ticket value
mismatch {row.match_id}", atol=0.011)
361         assert_close(row.updated_broadcast_value_usd, evaluated["broadcast_value"], f"P3 broadcast value
mismatch {row.match_id}", atol=0.011)
362         assert_close(row.resource_cost_index, evaluated["resource_cost"], f"P3 resource cost mismatch {
row.match_id}")
363         assert_close(row.risk_exposure_index, evaluated["risk"], f"P3 risk mismatch {row.match_id}",
atol=5e-9)
364         dynamic, _ = p3_normalize(pd.DataFrame(dynamic_rows), bounds)
365         assert_true(p3_constraints_audit(dynamic, self.data, p2)["all_passed"], "P3 dynamic constraints
failed")
366
367         static_environment = static_p3_environment(self.data, p2)
368         static_actions, static_bounds = p3_normalize(build_p3_actions(self.data, static_environment))
369         static_selected, _ = solve_p3_actions(static_actions, self.data["dynamic_resource_limits"])
370         recorded_static = self.diagnostics["problem3_static_decisions"]
371         for row in static_selected.itertuples(index=False):
372             recorded = recorded_static[row.match_id]
373             assert_true(
374                 [int(row.broadcast), int(row.security), int(row.transport)]
375                 == [recorded["broadcast"], recorded["security"], recorded["transport"]],
376                 f"P3 static pre-tournament action mismatch {row.match_id}",
377             )
378             assert_close(row.delta, recorded["delta"], f"P3 static delta mismatch {row.match_id}", atol=1e
-10)
379             assert_true(recorded["selected_using"] == "pre-tournament_static_information_only", "P3 static
information marker invalid")
380             static_rows = []
381             for match_id, decision in recorded_static.items():
382                 env = environment_index.loc[match_id].copy(); env["match_id"] = match_id
383                 static_rows.append(evaluate_p3_action(
384                     self.data, env, decision["broadcast"], decision["security"], decision["transport"], decision
["delta"]
385                 ))
386             static_eval, _ = p3_normalize(pd.DataFrame(static_rows), bounds)
387             assert_true(p3_constraints_audit(static_eval, self.data, p2)["all_passed"], "P3 fixed static
constraints failed")
388             dynamic_index, static_index = dynamic.set_index("match_id"), static_eval.set_index("match_id")
389             for row in output.itertuples(index=False):

```

```

389         assert_close(row.dynamic_net_value, dynamic_index.loc[row.match_id, "net_value"], f"P3 dynamic
net mismatch {row.match_id}", atol=5e-10)
390         assert_close(row.static_net_value, static_index.loc[row.match_id, "net_value"], f"P3 static net
mismatch {row.match_id}", atol=5e-10)
391         static_net = float(row.static_net_value)
392         if abs(static_net) < 1e-12:
393             assert_true(pd.isna(row.improvement_rate), f"P3 zero static net must have blank improvement
{row.match_id}")
394         else:
395             expected = (float(row.dynamic_net_value) - static_net) / abs(static_net)
396             assert_close(row.improvement_rate, expected, f"P3 improvement mismatch {row.match_id}", atol
=5e-9)
397             assert_close(dynamic["net_value"].sum(), self.diagnostics["problem3_dynamic_total_net"], "P3 dynamic
total mismatch", atol=1e-9)
398             assert_close(static_eval["net_value"].sum(), self.diagnostics["problem3_static_total_net"], "P3
static total mismatch", atol=1e-9)
399             robustness = self.diagnostics["problem3_robustness"]
400             assert_true(
401                 len(robustness["seeds"]) == 5
402                 and len(set(robustness["seeds"])) == 5
403                 and len(robustness["runs"]) == 5,
404                 "P3 robustness must contain five unique seeds",
405             )
406             for run in robustness["runs"]:
407                 assert_close(
408                     sum(run["advancement_probs"].values()), 32,
409                     f"P3 robustness seed {run['seed']} advancement sum", atol=1e-9,
410                 )
411                 assert_true(
412                     set(run["decision_signature"]) == third_ids,
413                     f"P3 robustness seed {run['seed']} decision set mismatch",
414                 )
415             dynamic_totals = [run["dynamic_total_net"] for run in robustness["runs"]]
416             static_totals = [run["static_total_net"] for run in robustness["runs"]]
417             improvement_rates = [run["overall_improvement_rate"] for run in robustness["runs"]]
418             assert_true(
419                 np.allclose(robustness["dynamic_total_net_range"], [min(dynamic_totals), max(dynamic_totals)])
420                 and np.allclose(robustness["static_total_net_range"], [min(static_totals), max(static_totals)])
421                 and np.allclose(
422                     robustness["overall_improvement_rate_range"],
423                     [min(improvement_rates), max(improvement_rates)],
424                 ),
425                 "P3 robustness ranges do not match five recorded runs",
426             )
427             primary_signature = robustness["runs"][0]["decision_signature"]
428             expected_stability = [
429                 float(np.mean([
430                     run["decision_signature"][match_id] == primary_signature[match_id]
431                     for match_id in primary_signature
432                 ]))
433                 for run in robustness["runs"]
434             ]
435             assert_true(
436                 np.allclose(
437                     robustness["dynamic_decision_match_fraction_vs_primary"],
438                     expected_stability,
439                 ),
440                 "P3 robustness decision stability mismatch",
441             )
442             assert_true(static_bounds == self.diagnostics["problem3_static_pre_normalization_bounds"], "P3
static bounds mismatch")
443
444     def check_problem4(self) -> None:

```

```

445     material_paths = {
446         "schedule": MATERIAL_DIR / "worldcup_2022_openfootball.json",
447         "elo": MATERIAL_DIR / "worldcup_2022_pre_tournament_elo.csv",
448         "mappings": MATERIAL_DIR / "entity_mappings.json",
449     }
450     assert_true(all(path.is_file() for path in material_paths.values()), "P4 one or more material files
are missing")
451     raw_source = json.loads(material_paths["schedule"].read_text(encoding="utf-8"))
452     raw_matches = raw_source["matches"]
453     assert_true(len(raw_matches) == 64, "P4 raw JSON must contain 64 matches")
454     assert_true(sum("group" in match for match in raw_matches) == 48, "P4 raw JSON must contain 48 group
matches")
455     elo = pd.read_csv(material_paths["elo"])
456     assert_true(
457         len(elo) == 32 and elo["team_name"].nunique() == 32
458         and not elo["team_name"].duplicated().any() and not elo.isna().any().any(),
459         "P4 Elo table must contain 32 unique complete teams",
460     )
461     mappings = json.loads(material_paths["mappings"].read_text(encoding="utf-8"))
462     raw_teams = {match["team1"] for match in raw_matches if "group" in match} | {
463         match["team2"] for match in raw_matches if "group" in match
464     }
465     assert_true(raw_teams == set(mappings["team_aliases"]), "P4 team alias coverage is incomplete")
466     assert_true(mappings["team_aliases"].get("USA") == "United States", "P4 USA alias is incorrect")
467     raw_stadiums = {match["ground"].split(",")[0].strip() for match in raw_matches if "group" in match}
468     assert_true(raw_stadiums == set(mappings["stadium_aliases"]), "P4 stadium alias coverage is
incomplete")
469     mapped_stadiums = set(mappings["stadium_aliases"].values())
470     assert_true(len(mapped_stadiums) == 8 and mapped_stadiums == set(mappings["stadiums"]), "P4 must map
exactly eight stadiums")
471     required_stadium_fields = {
472         "name", "city", "capacity", "latitude", "longitude", "timezone",
473         "capacity_source_url", "geodata_source_url", "retrieval_date",
474     }
475     for stadium_name, metadata in mappings["stadiums"].items():
476         assert_true(required_stadium_fields.issubset(metadata), f"P4 stadium metadata incomplete: {
stadium_name}")
477         assert_true(metadata["name"] == stadium_name, f"P4 stadium canonical name mismatch: {
stadium_name}")
478         assert_true(
479             int(metadata["capacity"]) > 0
480             and np.isfinite(float(metadata["latitude"]))
481             and np.isfinite(float(metadata["longitude"])),
482             f"P4 stadium numeric metadata invalid: {stadium_name}",
483         )
484         assert_true(
485             str(metadata["capacity_source_url"]).startswith("https://")
486             and str(metadata["geodata_source_url"]).startswith("https://")
487             and bool(str(metadata["retrieval_date"])),
488             f"P4 stadium source metadata invalid: {stadium_name}",
489         )
490     actual = self.output["actual_schedule_2022.csv"]
491     comparison = self.output["result_4_schedule_comparison.csv"]
492     assert_true(len(actual) == 48 and actual["match_id"].is_unique, "P4 actual schedule must have 48
unique matches")
493     teams = sorted(set(actual["team_a"]) | set(actual["team_b"]))
494     assert_true(len(teams) == 32, "P4 actual schedule must have 32 teams")
495     for team in teams:
496         assert_true(((actual["team_a"].eq(team)) | (actual["team_b"].eq(team))).sum() == 3, f"P4 {team}
match count")
497     assert_true((actual.groupby("group_id").size() == 6).all(), "P4 each group must have six matches")
498     assert_true((actual.groupby("round_in_group").size() == 16).all(), "P4 each round must have 16
matches")

```

```

499     expected_url = mappings["sources"]["openfootball_file"]
500     assert_true(actual["source_url"].eq(expected_url).all(), "P4 source URL is not the pinned concrete
file")
501     assert_true(bool(re.search(r"/blob/[0-9a-f]{40}/2022/worldcup\.json$", expected_url)), "P4 source
URL is not commit-pinned")
502     regenerated = actual_schedule_2022()
503     assert_true(
504         actual.drop(columns=["retrieval_date"]).equals(regenerated.drop(columns=["retrieval_date"])),
505         "P4 actual CSV differs from ordinary project source file",
506     )
507     for row in actual.itertuples(index=False):
508         local = pd.Timestamp(f"{row.date} {row.local_kickoff_time}", tz="Asia/Qatar")
509         assert_true(row.utc_datetime == local.tz_convert("UTC").strftime("%Y-%m-%d %H:%M"), f"P4 UTC
conversion mismatch {row.match_id}")
510         third = actual[actual["round_in_group"].eq(3)]
511         assert_true((third.groupby("group_id")["utc_datetime"].nunique() == 1).all(), "P4 third-round group
matches are not simultaneous")
512         source_hash = hashlib.sha256((MATERIAL_DIR / "worldcup_2022_openfootball.json").read_bytes()).
hexdigest()
513         assert_true(source_hash == "f4f0499c30076bc8e3702f400765900797fb22212a32aaa8137e1dfc49449a68", "P4
raw source hash changed")
514
515     required = {
516         "schedule_span_days", "total_travel_km", "avg_travel_per_team_km", "avg_travel_per_match_km",
517         "cross_timezone_transitions", "average_rest_hours", "minimum_rest_hours", "rest_hours_std",
518         "matches_per_venue", "matches_per_active_venue_day", "venue_active_day_utilization",
519         "capacity_match_ratio", "prime_time_share", "prime_matches_per_team", "prime_count_range",
520         "expected_attendance_per_match", "proxy_ticket_value_per_match_usd",
521         "proxy_broadcast_value_per_match_usd", "proxy_uncertainty_mean",
522         "proxy_attractiveness_mean", "proxy_cost_per_match_musd", "proxy_travel_burden",
523         "proxy_fairness_penalty", "proxy_execution_risk", "proxy_Z2",
524         "proxy_Z2_second_nearest", "prime_time_share_second_nearest",
525     }
526     assert_true(required.issubset(set(comparison["indicator_name"])), "P4 comparison indicators
incomplete")
527     assert_true(len(comparison) == 30 and comparison["indicator_name"].is_unique, "P4 must contain 30
unique indicators")
528     assert_true({"structure", "proxy_nearest", "proxy_sensitivity"}.issubset(set(comparison["
indicator_category"]))), "P4 layers not separated")
529     for row in comparison.itertuples(index=False):
530         actual_value, optimized_value = float(row.actual_schedule_value), float(row.
optimized_schedule_value)
531         assert_close(row.absolute_difference, optimized_value - actual_value, f"P4 absolute difference {
row.indicator_name}")
532         if row.preferred_direction == "neutral" or abs(actual_value) < 1e-12:
533             assert_true(pd.isna(row.relative_improvement), f"P4 relative improvement should be blank {
row.indicator_name}")
534         else:
535             sign = 1 if row.preferred_direction == "larger_better" else -1
536             expected = sign * (optimized_value - actual_value) / abs(actual_value)
537             assert_close(row.relative_improvement, expected, f"P4 relative improvement {row.
indicator_name}")
538         assert_true(self.diagnostics["problem4_scale_basis"]["T_B_C"].startswith("per_match"), "P4 cross-
size normalization basis missing")
539
540     def run(self) -> list[str]:
541         self.check_environment()
542         self.read_outputs()
543         self.check_problem1()
544         self.check_problem2()
545         self.check_problem3()
546         self.check_problem4()
547         return [

```

```

548         "fixed Python/dependency environment checks passed",
549         "problem1 formula/reproducibility checks passed",
550         "problem2 identity/formula/hard-constraint/group-boundary checks passed",
551         "problem3 simulation/static-dynamic recomputation checks passed",
552         "problem4 three-material/source/structure/proxy checks passed",
553     ]
554
555
556 def main() -> None:
557     args = parse_args()
558     output_dir = Path(args.output_dir)
559     if not output_dir.is_absolute():
560         output_dir = ROOT / output_dir
561     acceptance = Acceptance(output_dir)
562     messages = acceptance.run()
563     print("CHECK PASSED")
564     for message in messages:
565         print("-", message)
566     print("Problem 2 group adjacent-round boundary gaps (hours):")
567     for key, gap in acceptance.group_round_summary["gaps"].items():
568         print(f"- {key}: {gap:.1f}")
569     print(
570         "Problem 2 group boundary summary: "
571         f"minimum={acceptance.group_round_summary['minimum']:.1f} hours, "
572         f"violations={acceptance.group_round_summary['violations']}, "
573         f"all_passed={acceptance.group_round_summary['all_passed']}"
574     )
575
576
577 if __name__ == "__main__":
578     main()

```

D.4 论文分析与作图程序

```

1  #!/usr/bin/env python3
2  """Generate reproducible paper-only analyses without changing formal outputs."""
3
4  from __future__ import annotations
5
6  import argparse
7  import hashlib
8  import json
9  import math
10 import re
11 import sys
12 from pathlib import Path
13
14 import matplotlib
15
16 matplotlib.use("Agg")
17 import matplotlib.pyplot as plt
18 from matplotlib import font_manager
19 import numpy as np
20 import pandas as pd
21 import seaborn as sns
22 from sklearn.metrics import mean_absolute_error, mean_squared_error
23
24
25 ROOT = Path(__file__).resolve().parents[1]
26 sys.path.insert(0, str(ROOT / "code"))
27 import pipeline_core as pc # noqa: E402
28
29

```

```

30 FIGURE_DIR: Path
31 TABLE_DIR: Path
32 COLORS = {
33     "blue": "#2563EB", "teal": "#0F766E", "orange": "#D97706",
34     "red": "#B91C1C", "gray": "#64748B", "green": "#15803D",
35     "purple": "#7E22CE", "light": "#E2E8F0",
36 }
37
38
39 def parse_args() -> argparse.Namespace:
40     parser = argparse.ArgumentParser(description=__doc__)
41     parser.add_argument("--output-dir", default="output", help="Formal output directory.")
42     parser.add_argument("--generated-dir", default="paper/generated", help="Paper-only generated directory."
43     )
44     parser.add_argument("--skip-p2-sensitivity", action="store_true")
45     return parser.parse_args()
46
47 def resolve(path: str) -> Path:
48     candidate = Path(path)
49     return candidate if candidate.is_absolute() else ROOT / candidate
50
51
52 def configure_plotting() -> None:
53     for path in (
54         "/mnt/c/Windows/Fonts/msyh.ttc", "/mnt/c/Windows/Fonts/msykbd.ttc",
55         "/mnt/c/Windows/Fonts/simhei.ttf",
56         "/mnt/c/Windows/Fonts/times.ttf",
57     ):
58         if Path(path).exists():
59             font_manager.fontManager.addfont(path)
60     sns.set_theme(style="whitegrid", context="paper")
61     plt.rcParams.update({
62         "font.family": ["Microsoft YaHei", "DejaVu Sans"],
63         "font.size": 9.0, "axes.titlesize": 10.5, "axes.labelsize": 9.5,
64         "axes.unicode_minus": False, "figure.dpi": 130, "savefig.dpi": 320,
65         "pdf.fonttype": 42, "ps.fonttype": 42, "svg.fonttype": "none",
66         "axes.spines.top": False, "axes.spines.right": False,
67     })
68
69
70 def save_figure(fig: plt.Figure, name: str) -> None:
71     fig.tight_layout()
72     for suffix in ("pdf", "svg", "png"):
73         fig.savefig(
74             FIGURE_DIR / f"{name}.{suffix}", bbox_inches="tight",
75             metadata={"Creator": "code/paper_analysis.py", "Title": name},
76         )
77     plt.close(fig)
78
79
80 def write_table(frame: pd.DataFrame, name: str, column_format: str | None = None) -> None:
81     frame.to_csv(TABLE_DIR / f"{name}.csv", index=False, encoding="utf-8-sig")
82     def escape(value: object) -> str:
83         if pd.isna(value):
84             return "--"
85         if isinstance(value, (float, np.floating)):
86             text = f"{float(value):.6f}"
87         else:
88             text = str(value)
89         replacements = {
90             "\\": r"\textbackslash{}", "&": r"&", "%": r"%", "$": r"\$",
91             "#": r"#", "_": r"_", "{": r"{", "(": r"(", "}: r"}",

```

```

92     }
93     return re.sub(r"[\&%$#_{}]", lambda match: replacements[match.group(0)], text)
94
95     alignment = column_format or "l" + "r" * (len(frame.columns) - 1)
96     lines = [rf"\begin{{{tabular}}}{{{{alignment}}}}", r"\toprule"]
97     lines.append(" & ".join(escape(column) for column in frame.columns) + r" \\")
98     lines.append(r"\midrule")
99     lines.extend(" & ".join(escape(value) for value in row) + r" \\\" for row in frame.itertuples(index=False
100 , name=None))
101 lines.extend([r"\bottomrule", r"\end{tabular}", ""])
102 latex = "\n".join(lines)
103 (TABLE_DIR / f"{name}.tex").write_text(latex, encoding="utf-8")
104
105 def clean_feature_name(name: str) -> str:
106     name = name.replace("numeric__", "").replace("categorical__", "")
107     replacements = {
108         "competition": "赛事=", "stage_group": "阶段=", "fan_base_index": "球迷基础",
109         "market_value_musd": "市值", "strength_score": "综合实力", "style_attack": "进攻风格",
110         "style_defense": "防守风格", "star_index": "明星指数", "rank": "排名",
111         "elo": "Elo", "avg_age": "平均年龄", "host_flag": "东道主标记",
112         "win_prob": "胜率", "prob_entropy": "概率熵", "p_draw": "平局概率",
113         "neutral": "中立场", "weekday_num": "星期", "month": "月份",
114         "_mean": "均值", "_min": "最小", "_max": "最大", "_absdiff": "绝对差",
115     }
116     for old, new in replacements.items():
117         name = name.replace(old, new)
118     return name
119
120
121 def analyze_problem1(data: dict[str, pd.DataFrame]) -> dict:
122     historical = data["historical_matches"].copy()
123     train = historical[historical["dataset_split"].eq("train") & historical["tv_viewers"].notna()].copy()
124     test = historical[historical["dataset_split"].eq("test")].copy()
125     features = pc.build_prediction_features(train, data["teams"])
126     target_log = np.logp(train["tv_viewers"].to_numpy(float) / 1_000_000)
127     order = np.argsort(pd.to_datetime(train["date"]).to_numpy())
128     folds = []
129     for fold_no, fraction in enumerate((0.60, 0.70, 0.80), start=1):
130         cut = int(len(order) * fraction)
131         end = min(len(order), cut + max(30, int(len(order) * 0.10)))
132         folds.append((fold_no, order[:cut], order[cut:end]))
133
134     metric_rows, elastic_rows = [], []
135     oof_rows = []
136     for model_name in ("mean", "elastic_net", "extra_trees"):
137         for fold_no, train_idx, valid_idx in folds:
138             model = pc.make_p1_model(model_name)
139             model.fit(features.iloc[train_idx], target_log[train_idx])
140             prediction = np.expm1(model.predict(features.iloc[valid_idx])).clip(min=0)
141             truth = np.expm1(target_log[valid_idx])
142             mse = mean_squared_error(truth, prediction)
143             row = {
144                 "model": model_name, "fold": fold_no,
145                 "train_n": len(train_idx), "valid_n": len(valid_idx),
146                 "train_start": pd.to_datetime(train.iloc[train_idx]["date"]).min().date().isoformat(),
147                 "train_end": pd.to_datetime(train.iloc[train_idx]["date"]).max().date().isoformat(),
148                 "valid_start": pd.to_datetime(train.iloc[valid_idx]["date"]).min().date().isoformat(),
149                 "valid_end": pd.to_datetime(train.iloc[valid_idx]["date"]).max().date().isoformat(),
150                 "mse": float(mse), "rmse": float(math.sqrt(mse)),
151                 "mae": float(mean_absolute_error(truth, prediction)),
152             }
153             metric_rows.append(row)

```

```

154         if model_name == "elastic_net":
155             elastic_rows.append(row)
156             oof_rows.extend({
157                 "fold": fold_no, "match_id": train.iloc[idx]["match_id"],
158                 "actual_million": float(actual), "predicted_million": float(pred),
159                 "residual_million": float(actual - pred),
160             } for idx, actual, pred in zip(valid_idx, truth, prediction))
161
162     final_model = pc.make_pl_model("elastic_net")
163     final_model.fit(features, target_log)
164     names = final_model.named_steps["preprocess"].get_feature_names_out()
165     coefficients = final_model.named_steps["regressor"].coef_
166     coefficient_table = pd.DataFrame({
167         "feature_raw": names, "feature": [clean_feature_name(name) for name in names],
168         "coefficient": coefficients, "absolute_coefficient": np.abs(coefficients),
169     }).sort_values("absolute_coefficient", ascending=False)
170
171     future_raw = data["groups_matches"].merge(
172         data["base_predictions"].drop(columns=["group_id", "round_in_group", "team_a", "team_b"], errors="
173         ignore"),
174         on="match_id", how="left",
175     )
176     future_features = pc.build_prediction_features(future_raw, data["teams"])
177     train_competitions = sorted(features["competition"].astype(str).unique())
178     future_competitions = sorted(future_features["competition"].astype(str).unique())
179     transform = final_model.named_steps["preprocess"]
180     transformed_train = transform.transform(features)
181     transformed_future = transform.transform(future_features)
182     if hasattr(transformed_train, "toarray"):
183         transformed_train = transformed_train.toarray()
184         transformed_future = transformed_future.toarray()
185     mean_shift = (np.asarray(transformed_future).mean(axis=0) - np.asarray(transformed_train).mean(axis=0))
186     * coefficients
187     shift_table = pd.DataFrame({
188         "feature": [clean_feature_name(name) for name in names],
189         "mean_log_prediction_shift": mean_shift,
190     }).assign(abs_shift=lambda x: x["mean_log_prediction_shift"].abs()).sort_values("abs_shift", ascending=
191     False)
192
193     metrics = pd.DataFrame(metric_rows)
194     elastic = pd.DataFrame(elastic_rows)
195     oof = pd.DataFrame(oof_rows)
196     write_table(elastic[[
197         "fold", "train_n", "valid_n", "train_start", "train_end", "valid_start", "valid_end", "mse", "rmse",
198         "mae",
199     ]], "q1_expanding_folds")
200     write_table(coefficient_table.head(15)[["feature", "coefficient", "absolute_coefficient"]], "
201     q1_top_coefficients")
202     write_table(shift_table.head(10)[["feature", "mean_log_prediction_shift"]], "q1_future_mean_shift")
203     oof_summary = {
204         "correlation": float(oof[["actual_million", "predicted_million"]].corr().iloc[0, 1]),
205         "mean_residual_million": float(oof["residual_million"].mean()),
206         "residual_std_million": float(oof["residual_million"].std(ddof=1)),
207         "lowest_actual_decile_mean_residual": float(oof.nsmallest(max(1, len(oof) // 10), "actual_million")["
208         residual_million"].mean()),
209         "highest_actual_decile_mean_residual": float(oof.nlargest(max(1, len(oof) // 10), "actual_million")["
210         residual_million"].mean()),
211     }
212
213     fig, axes = plt.subplots(2, 2, figsize=(10.8, 7.4))
214     ax = axes[0, 0]
215     width = 0.24
216     x = np.arange(3)

```

```

210 for offset, metric, color in [(-width, "rmse", COLORS["blue"]), (0, "mae", COLORS["teal"])]:
211     ax.bar(x + offset, elastic[metric], width=width, label=metric.upper(), color=color)
212 ax.set_xticks(x - width / 2, [f"第 {i} 折" for i in elastic["fold"]])
213 ax.set_ylabel("误差 (百万人)")
214 ax.set_title("(a) Elastic Net 扩窗验证误差")
215 ax.legend(frameon=False)
216
217 top = coefficient_table.head(15).sort_values("coefficient")
218 axes[0, 1].barh(top["feature"], top["coefficient"], color=np.where(top["coefficient"] >= 0, COLORS["blue"],
219     COLORS["orange"]))
219 axes[0, 1].axvline(0, color="black", linewidth=0.7)
220 axes[0, 1].set_xlabel("模型系数")
221 axes[0, 1].set_title("(b) 系数绝对值前 15 (数值特征已标准化)")
222
223 axes[1, 0].scatter(oof["actual_million"], oof["predicted_million"], s=18, alpha=0.58, color=COLORS["blue"],
224     edgecolors="none")
225 low = min(oof["actual_million"].min(), oof["predicted_million"].min())
226 high = max(oof["actual_million"].max(), oof["predicted_million"].max())
227 axes[1, 0].plot([low, high], [low, high], "--", color=COLORS["gray"], linewidth=1)
228 axes[1, 0].set_xlabel("真实观看人数 (百万人)", ylabel="预测观看人数 (百万人)")
229 axes[1, 0].set_title(f"(c) 扩窗验证预测 (n={len(oof)})")
230
231 axes[1, 1].scatter(oof["predicted_million"], oof["residual_million"], s=18, alpha=0.58, color=COLORS["teal"],
232     edgecolors="none")
233 axes[1, 1].axhline(0, color=COLORS["gray"], linestyle="--", linewidth=1)
234 axes[1, 1].set_xlabel("预测值 (百万人)", ylabel="残差: 真实值-预测值")
235 axes[1, 1].set_title("(d) 残差与预测值")
236 save_figure(fig, "fig_q1_validation_coefficients_residuals")
237
238 return {
239     "fold_metrics": metric_rows,
240     "elastic_fold_summary": {
241         key: {"mean": float(elastic[key].mean()), "std": float(elastic[key].std(ddof=1)),
242             "min": float(elastic[key].min()), "max": float(elastic[key].max())}
243         for key in ("mse", "rmse", "mae")
244     },
245     "top_coefficients": coefficient_table.head(15).to_dict(orient="records"),
246     "future_mean_shift_top": shift_table.head(10).to_dict(orient="records"),
247     "competition_categories": {
248         "train": train_competitions, "future72": future_competitions,
249         "future_unseen_in_train": sorted(set(future_competitions) - set(train_competitions)),
250     },
251     "competition_normalization": pc.competition_normalization_diagnostics(train, test, future_raw),
252     "model_seed": pc.P1_MODEL_SEED,
253     "oof_summary": oof_summary,
254     "oof_n": len(oof),
255 }
256
257 def draw_information_flow() -> None:
258     fig, ax = plt.subplots(figsize=(10.5, 2.8))
259     ax.set_axis_off()
260     boxes = [
261         (0.02, "问题一\n赛前收视预测", "历史赛前特征\nElastic Net"),
262         (0.27, "问题二\n赛程与场馆", "72 场收视预测\n分解 MILP"),
263         (0.52, "问题三\n动态资源", "固定第三轮赛程\n第二轮后快照"),
264         (0.77, "问题四\n实际赛程比较", "优化赛程\n2022 结构与代理"),
265     ]
266     for x, title, subtitle in boxes:
267         ax.add_patch(plt.Rectangle((x, 0.28), 0.20, 0.48, facecolor="#F8FAFC", edgecolor=COLORS["blue"],
268             linewidth=1.4))
269         ax.text(x + 0.10, 0.59, title, ha="center", va="center", fontsize=11, weight="bold")
270         ax.text(x + 0.10, 0.37, subtitle, ha="center", va="center", fontsize=8, color="#334155")

```

```

269 for x in (0.22, 0.47, 0.72):
270     ax.annotate("", xy=(x + 0.045, 0.52), xytext=(x, 0.52), arrowprops={"arrowstyle": "-|>", "lw": 1.4,
271 "color": COLORS["teal"]})
272 ax.text(0.5, 0.08, "信息只沿时间向后传递; 第三轮 24 场共用开赛前同一截面", ha="center", color="#334155")
273 save_figure(fig, "fig_information_flow")
274
275 def materialize_schedule(assignments: pd.DataFrame, ctx: pc.P2Context, data: dict[str, pd.DataFrame]) -> pd.
    DataFrame:
276     venues = data["venues"].set_index("venue_id")
277     slots = data["time_slots"].set_index("slot_id")
278     venue_metric = ctx.venue_candidates.set_index(["match_id", "venue_id"])
279     schedule = ctx.matches.merge(assignments, on="match_id", how="left")
280     rows = []
281     for row in schedule.itertuples(index=False):
282         venue, slot = venues.loc[row.venue_id], slots.loc[row.slot_id]
283         metric = venue_metric.loc[(row.match_id, row.venue_id)]
284         utc = pc.utc_timestamp(slot.reference_utc_time)
285         local = utc.tz_convert(pc.ZoneInfo(str(venue.timezone)))
286         rows.append({
287             **row._asdict(), "city": venue.city, "country": venue.country,
288             "reference_date": pd.to_datetime(slot.date).date().isoformat(),
289             "reference_kickoff_time": str(slot.reference_kickoff_time),
290             "utc_datetime": utc.strftime("%Y-%m-%d %H:%M"),
291             "local_datetime": local.strftime("%Y-%m-%d %H:%M"),
292             "expected_attendance": float(metric.expected_attendance),
293         })
294     return pd.DataFrame(rows)
295
296 def perturbed_weights(indicator: str, factor: float) -> dict[str, float]:
297     result = dict(pc.P2_WEIGHTS)
298     result[indicator] *= factor
299     scale = sum(abs(value) for value in result.values())
300     return {key: value / scale for key, value in result.items()}
301
302 def weighted_p2_metrics(schedule: pd.DataFrame, ctx: pc.P2Context, data: dict[str, pd.DataFrame], weights:
    dict[str, float]) -> dict:
303     base = pc.p2_metrics(schedule, ctx, data)
304     contributions = {key: weights[key] * base["normalized"][key] for key in weights}
305     return {**base, "weighted_contributions": contributions, "Z2": float(sum(contributions.values()))}
306
307 def p2_model_sizes(ctx: pc.P2Context, data: dict[str, pd.DataFrame], formal: pd.DataFrame) -> dict:
308     matches, slots = ctx.matches.reset_index(drop=True), data["time_slots"].reset_index(drop=True)
309     slot_constraints = len(matches) + 48 * 2 + 12 * 2 * 4 + 20 + 48 * 2 + 1
310     pairs = ctx.venue_candidates[["match_id", "venue_id"]].drop_duplicates().merge(formal[["match_id", "
311 slot_id"]], on="match_id")
312     venue_index = data["venues"].set_index("venue_id")
313     slot_index = data["time_slots"].set_index("slot_id")
314     local_groups, utc_groups = {}, {}
315     for k, pair in pairs.iterrows():
316         venue = venue_index.loc[pair.venue_id]
317         utc = pc.utc_timestamp(slot_index.loc[pair.slot_id, "reference_utc_time"])
318         local_day = utc.tz_convert(pc.ZoneInfo(str(venue.timezone))).date().isoformat()
319         local_groups.setdefault((pair.venue_id, local_day), []).append(k)
320         utc_groups.setdefault((pair.venue_id, utc.isoformat()), []).append(k)
321     venue_constraints = 72 + pairs["venue_id"].nunique() + len(local_groups) + sum(len(v) > 1 for v in
322 utc_groups.values()) + 48 * 2
323     return {
324         "slot": {"variables": len(matches) * len(slots) + 2, "constraints": slot_constraints},
325         "venue": {"variables": len(pairs) + 2, "constraints": int(venue_constraints)},
326

```

```

327     }
328
329
330 def analyze_problem2(data: dict[str, pd.DataFrame], formal_dir: Path, diagnostics: dict, skip_sensitivity:
    bool) -> tuple[dict, pc.P2Context]:
331     p1 = pd.read_csv(formal_dir / "result_1_match_prediction.csv")
332     formal = pd.read_csv(formal_dir / "result_2_group_schedule.csv")
333     ctx = pc.build_p2_context(data, p1)
334     sizes = p2_model_sizes(ctx, data, formal)
335     solver_rows = []
336     for key, label in (("slot_subproblem", "时段 MILP"), ("venue_subproblem_given_slots", "条件场馆 MILP")):
337         values = diagnostics["problem2_solver"][key]
338         size = sizes["slot" if key == "slot_subproblem" else "venue"]
339         solver_rows.append({
340             "subproblem": label, "variables": size["variables"], "constraints": size["constraints"],
341             "runtime_seconds": values["runtime_seconds"], "mip_gap": values["mip_gap"],
342             "status": values["interpretation"],
343         })
344     write_table(pd.DataFrame(solver_rows), "q2_solver_scale")
345
346     formal_index = formal.set_index("match_id")
347     sensitivity_rows = [{
348         "scenario": "baseline", "indicator": "baseline", "factor": 1.0,
349         "Z2_scenario": diagnostics["problem2_total_objective"],
350         "Z2_original_weights": diagnostics["problem2_total_objective"], "slot_change_rate": 0.0,
351         "venue_change_rate": 0.0, "assignment_change_rate": 0.0,
352         **{key: diagnostics["problem2_indicator_details"]["raw"][key] for key in ("T", "B", "C", "D", "F", "R")},
353         "all_constraints_passed": True,
354     }]
355     if not skip_sensitivity:
356         for indicator in ("T", "B", "C", "D", "F", "R"):
357             for factor in (0.8, 1.2):
358                 weights = perturbed_weights(indicator, factor)
359                 slots, slot_diag = pc.solve_p2_slots(ctx, data, weights)
360                 assignments, venue_diag = pc.solve_p2_venues(ctx, data, slots, weights)
361                 schedule = materialize_schedule(assignments, ctx, data)
362                 metrics = weighted_p2_metrics(schedule, ctx, data, weights)
363                 original_weight_metrics = weighted_p2_metrics(schedule, ctx, data, pc.P2_WEIGHTS)
364                 audit = pc.p2_hard_constraint_audit(schedule, ctx, data)
365                 candidate = schedule.set_index("match_id")
366                 slot_change = float((candidate["slot_id"] != formal_index["slot_id"]).mean())
367                 venue_change = float((candidate["venue_id"] != formal_index["venue_id"]).mean())
368                 both = float(((candidate["slot_id"] != formal_index["slot_id"]) | (candidate["venue_id"] !=
369                     formal_index["venue_id"])).mean())
370                 sensitivity_rows.append({
371                     "scenario": f"{indicator}_{factor:.1f}", "indicator": indicator, "factor": factor,
372                     "Z2_scenario": metrics["Z2"], "Z2_original_weights": original_weight_metrics["Z2"],
373                     "slot_change_rate": slot_change,
374                     "venue_change_rate": venue_change, "assignment_change_rate": both,
375                     **{key: metrics["raw"][key] for key in ("T", "B", "C", "D", "F", "R")},
376                     "all_constraints_passed": audit["all_passed"],
377                     "slot_runtime_seconds": slot_diag["runtime_seconds"], "venue_runtime_seconds":
378                     venue_diag["runtime_seconds"],
379                 })
380                 print(f"P2 sensitivity {indicator} x {factor:.1f}: Z2={metrics['Z2']:.6f}, changed={both:.3f}
381                 ), flush=True)
382     sensitivity = pd.DataFrame(sensitivity_rows)
383     write_table(sensitivity, "q2_weight_sensitivity")
384
385     # Date by UTC kickoff occupancy.
386     heat = formal.assign(
387         date=pd.to_datetime(formal["utc_datetime"], utc=True).dt.strftime("%m-%d"),

```

```

385     hour=pd.to_datetime(formal["utc_datetime"], utc=True).dt.strftime("%H:%M"),
386 ).pivot_table(index="date", columns="hour", values="match_id", aggfunc="count", fill_value=0)
387 heat = heat.reindex(sorted(heat.columns, key=lambda value: int(value[:2]) * 60 + int(value[3:])), axis
388 =1)
389 fig, ax = plt.subplots(figsize=(10.6, 6.0))
390 sns.heatmap(heat, cmap="YlGnBu", linewidths=0.35, annot=True, fmt=".0f", cbar_kws={"label": "比赛场数"},
391 ax=ax)
392 ax.set(xlabel="UTC 开球时刻", ylabel="参考日期", title="72 场比赛的日期—UTC 时段分布")
393 save_figure(fig, "fig_q2_schedule_heatmap")
394
395 venue_counts = formal.groupby("venue_id").size().sort_values()
396 rests = []
397 for team in sorted(set(formal["team_a"]) | set(formal["team_b"])):
398     subset = formal[(formal["team_a"].eq(team)) | (formal["team_b"].eq(team))].sort_values("
399 round_in_group")
400     rests.extend(pd.to_datetime(subset["utc_datetime"], utc=True).diff().dropna().dt.total_seconds().div
401 (3600).tolist())
402 contribution = pd.Series(diagnostics["problem2_indicator_details"]["weighted_contributions"])
403 gap = pd.Series(diagnostics["problem2_hard_constraint_audit"]["group_round_gaps_hours"]).sort_values()
404 fig, axes = plt.subplots(2, 2, figsize=(11.2, 7.4))
405 axes[0, 0].barh(venue_counts.index, venue_counts.values, color=COLORS["teal"])
406 axes[0, 0].set(xlabel="承办场次", title="(a) 16 个场馆的负荷")
407 axes[0, 1].hist(rests, bins=np.arange(60, max(rests) + 25, 24), color=COLORS["blue"], edgecolor="white")
408 axes[0, 1].axvline(60, color=COLORS["red"], linestyle="--", label="60 小时下限")
409 axes[0, 1].set(xlabel="同队相邻比赛间隔 (小时)", ylabel="频数", title=f"(b) 48 队的 96 个相邻比赛间隔 (最
410 小 {min(rests):.0f}) ")
411 axes[0, 1].legend(frameon=False)
412 axes[1, 0].bar(contribution.index, contribution.values, color=[COLORS["blue"] if value >= 0 else COLORS[
413 "orange"] for value in contribution])
414 axes[1, 0].axhline(0, color="black", linewidth=0.7)
415 axes[1, 0].set(ylabel="对 $Z_2$ 的加权贡献", title="(c) 八项指标的目标贡献")
416 axes[1, 1].barh(gap.index, gap.values, color=np.where(gap.values < 100, COLORS["teal"], COLORS["gray"]))
417 axes[1, 1].axvline(60, color=COLORS["red"], linestyle="--", linewidth=1)
418 axes[1, 1].set(xlabel="轮次整体边界间隔 (小时)", title="(d) 24 个同组相邻轮次边界")
419 save_figure(fig, "fig_q2_load_rest_contributions_gaps")
420
421 if len(sensitivity) > 1:
422     plot = sensitivity.iloc[1:].copy()
423     fig, axes = plt.subplots(1, 2, figsize=(11.4, 4.2))
424     x = np.arange(len(plot))
425     axes[0].bar(x, plot["Z2_original_weights"], color=COLORS["blue"])
426     axes[0].axhline(sensitivity.iloc[0]["Z2_original_weights"], color=COLORS["red"], linestyle="--",
427 label="正式方案")
428     axes[0].set_xticks(x, plot["scenario"], rotation=45, ha="right")
429     axes[0].set(ylabel="按正式权重回评的 $Z_2$", title="(a) 汇总指标稳定性")
430     axes[0].legend(frameon=False)
431     width = 0.25
432     axes[1].bar(x - width, 100 * plot["slot_change_rate"], width, label="时段", color=COLORS["blue"])
433     axes[1].bar(x, 100 * plot["venue_change_rate"], width, label="场馆", color=COLORS["teal"])
434     axes[1].bar(x + width, 100 * plot["assignment_change_rate"], width, label="完整指派", color=COLORS["
435 orange"])
436     axes[1].set_xticks(x, plot["scenario"], rotation=45, ha="right")
437     axes[1].set(ylabel="变化比例 (%)", title="(b) 具体排程替代性")
438     axes[1].legend(frameon=False, ncol=3, fontsize=8)
439     save_figure(fig, "fig_q2_weight_sensitivity")
440
441 return {
442     "model_sizes": sizes, "solver": solver_rows,
443     "venue_match_counts": venue_counts.to_dict(),
444     "team_rest_hours": pc.distribution(rests),
445     "sensitivity": sensitivity_rows,
446 }, ctx

```

```

440
441 def analyze_problem3(data: dict[str, pd.DataFrame], formal_dir: Path) -> dict:
442     p2 = pd.read_csv(formal_dir / "result_2_group_schedule.csv")
443     formal = pd.read_csv(formal_dir / "result_3_dynamic_strategy.csv")
444     standings = pc.standings_from_live(data)
445     static_environment = pc.static_p3_environment(data, p2)
446     static_actions, _ = pc.p3_normalize(pc.build_p3_actions(data, static_environment))
447     static_decisions, _ = pc.solve_p3_actions(static_actions, data["dynamic_resource_limits"])
448
449     runs = []
450     primary = None
451     for n_sim in (2000, 5000, 10000, 20000):
452         simulation_diag, simulation = pc.simulate_advancement(data, standings, n_sim, pc.P3_CONVERGENCE_SEED
453         )
454         environment = pc.dynamic_p3_environment(data, p2, simulation, standings)
455         actions, bounds = pc.p3_normalize(pc.build_p3_actions(data, environment))
456         dynamic, _ = pc.solve_p3_actions(actions, data["dynamic_resource_limits"])
457         static_eval = pc.evaluate_fixed_p3(data, environment, static_decisions, bounds)
458         signature = {row.match_id: (int(row.broadcast), int(row.security), int(row.transport), round(float(
459         row.delta, 8)) for row in dynamic.itertuples(index=False))}
460         runs.append({
461             "simulations": n_sim, "advancement_probs": simulation_diag["advancement_probs"],
462             "dynamic_total_net": float(dynamic["net_value"].sum()),
463             "static_total_net": float(static_eval["net_value"].sum()),
464             "improvement_rate": float((dynamic["net_value"].sum() - static_eval["net_value"].sum()) / abs(
465             static_eval["net_value"].sum())),
466             "signature": signature,
467         })
468         if n_sim == 20000:
469             primary = (simulation, environment, dynamic, static_eval)
470             print(f"P3 convergence n={n_sim}: dynamic={runs[-1]['dynamic_total_net']:.6f}", flush=True)
471             assert primary is not None
472             reference = runs[-1]
473             for run in runs:
474                 differences = [abs(run["advancement_probs"][team] - reference["advancement_probs"][team]) for team
475                 in reference["advancement_probs"]]
476                 run["advancement_max_abs_diff_vs_20000"] = float(max(differences))
477                 run["advancement_mean_abs_diff_vs_20000"] = float(np.mean(differences))
478                 run["decision_match_rate_vs_20000"] = float(np.mean([run["signature"][mid] == reference["signature"
479                 ][mid] for mid in reference["signature"]]))
480
481             simulation, environment, dynamic, static_eval = primary
482             env_index, dynamic_index, static_index = environment.set_index("match_id"), dynamic.set_index("match_id"
483             ), static_eval.set_index("match_id")
484             static_decision_index = static_decisions.set_index("match_id")
485             formal_index = formal.set_index("match_id")
486             case_ids = sorted(formal.loc[
487                 (formal["dynamic_net_value"] - formal["static_net_value"]).abs().gt(5e-10), "match_id"
488             ].astype(str).tolist())
489             case_rows = []
490             for match_id in case_ids:
491                 env, dyn, sta = env_index.loc[match_id], dynamic_index.loc[match_id], static_index.loc[match_id]
492                 pre = static_decision_index.loc[match_id]
493                 out = formal_index.loc[match_id]
494                 case_rows.append({
495                     "match_id": match_id, "match": f"{env.team_a}-{env.team_b}",
496                     "p_a": env.p_team_a_advance, "p_b": env.p_team_b_advance, "importance": env.Q_i,
497                     "stakeless": env.stakeless_risk, "collusion": env.collusion_risk,
498                     "attendance_dynamic": dyn.attendance, "viewers_dynamic": dyn.broadcast_viewers,
499                     "static_action": f"B{int(pre.broadcast)}/S{int(pre.security)}/T{int(pre.transport)}/{100*pre.
500                     delta:.3f}%",
501                     "dynamic_action": f"B{int(dyn.broadcast)}/S{int(dyn.security)}/T{int(dyn.transport)}/{100*dyn.
502                     delta:.3f}%",

```

```

495         "static_net": sta.net_value, "dynamic_net": dyn.net_value,
496         "improvement_rate": out.improvement_rate,
497     })
498     cases = pd.DataFrame(case_rows)
499     write_table(cases, "q3_changed_cases")
500     write_table(cases[["match_id", "p_a", "p_b", "importance", "stakeless", "collusion"]].rename(columns={
501         "match_id": "比赛", "p_a": "A队晋级", "p_b": "B队晋级", "importance": "重要性",
502         "stakeless": "无激励", "collusion": "默契",
503     }), "q3_changed_cases_risk")
504     write_table(cases[["match_id", "static_action", "dynamic_action", "static_net", "dynamic_net", "
505         improvement_rate"]].rename(columns={
506         "match_id": "比赛", "static_action": "静态动作", "dynamic_action": "动态动作",
507         "static_net": "静态净值", "dynamic_net": "动态净值", "improvement_rate": "改善率",
508     }), "q3_changed_cases_actions")
509
510     limits = data["dynamic_resource_limits"].set_index("reference_date")
511     daily_rows = []
512     for day, subset in dynamic.groupby("reference_date"):
513         limit = limits.loc[day]
514         daily_rows.append({
515             "date": day,
516             "high_broadcast_used": int(subset["broadcast"].ge(3).sum()), "high_broadcast_capacity": int(
517                 limit.high_broadcast_capacity),
518             "high_security_used": int(subset["security"].ge(3).sum()), "high_security_capacity": int(limit.
519                 high_security_capacity),
520             "enhanced_transport_used": int(subset["transport"].ge(3).sum()), "enhanced_transport_capacity":
521                 int(limit.enhanced_transport_capacity),
522             "budget_used": float(subset["resource_cost"].sum()), "budget_capacity": float(limit.
523                 daily_resource_budget_index),
524         })
525     daily = pd.DataFrame(daily_rows).sort_values("date")
526     for resource in ("high_broadcast", "high_security", "enhanced_transport"):
527         daily[resource + "_binding"] = daily[resource + "_used"].eq(daily[resource + "_capacity"])
528         daily["budget_ratio"] = daily["budget_used"] / daily["budget_capacity"]
529         daily["budget_near_binding"] = daily["budget_ratio"].ge(0.95)
530     write_table(daily, "q3_daily_resources")
531     daily_compact = pd.DataFrame({
532         "日期": pd.to_datetime(daily["date"]).dt.strftime("%m-%d"),
533         "高转播_用量/容量": daily["high_broadcast_used"].astype(str) + "/" + daily["high_broadcast_capacity"]
534         ].astype(str),
535         "高安保_用量/容量": daily["high_security_used"].astype(str) + "/" + daily["high_security_capacity"].
536         astype(str),
537         "强交通_用量/容量": daily["enhanced_transport_used"].astype(str) + "/" + daily["
538         enhanced_transport_capacity"].astype(str),
539         "预算使用率%": 100 * daily["budget_ratio"],
540     })
541     write_table(daily_compact, "q3_daily_resources_compact")
542     convergence = pd.DataFrame([{"key": value for key, value in run.items() if key not in ("advancement_probs"
543         , "signature")} for run in runs])
544     write_table(convergence, "q3_monte_carlo_convergence")
545     write_table(convergence[[
546         "simulations", "advancement_max_abs_diff_vs_20000", "decision_match_rate_vs_20000",
547         "dynamic_total_net", "static_total_net", "improvement_rate",
548     ]].rename(columns={
549         "simulations": "模拟次数", "advancement_max_abs_diff_vs_20000": "晋级概率最大差",
550         "decision_match_rate_vs_20000": "动作一致率", "dynamic_total_net": "动态总值",
551         "static_total_net": "静态总值", "improvement_rate": "总改善率",
552     }), "q3_monte_carlo_convergence_compact")
553
554     risk = environment[["match_id", "p_team_a_advance", "p_team_b_advance", "Q_i", "stakeless_risk", "
555         collusion_risk"]].copy()
556     risk = risk.sort_values("Q_i", ascending=False).reset_index(drop=True)
557     fig, axes = plt.subplots(2, 2, figsize=(11.2, 7.6), sharex=True)

```

```

548 x = np.arange(len(risk))
549 axes[0, 0].scatter(x - 0.12, risk["p_team_a_advance"], color=COLORS["blue"], s=22, label="球队 A")
550 axes[0, 0].scatter(x + 0.12, risk["p_team_b_advance"], color=COLORS["teal"], s=22, label="球队 B")
551 axes[0, 0].set_ylim(0, 1.02)
552 axes[0, 0].set_title("双方晋级概率")
553 axes[0, 0].set_ylabel("概率")
554 axes[0, 0].legend(frameon=False, ncol=2)
555 panels = [
556     (axes[0, 1], "Q_i", "晋级重要性", COLORS["purple"]),
557     (axes[1, 0], "stakeless_risk", "无激励风险", COLORS["orange"]),
558     (axes[1, 1], "collusion_risk", "默契风险", COLORS["red"]),
559 ]
560 for ax, column, label, color in panels:
561     ax.scatter(x, risk[column], color=color, s=25)
562     ax.vlines(x, 0, risk[column], color=color, alpha=0.35, linewidth=0.8)
563     ax.set_ylim(0, 1.02)
564     ax.set_title(label)
565     ax.set_ylabel("指数 / 概率")
566 for ax in axes[1]:
567     ax.set_xticks(x, risk["match_id"], rotation=70, fontsize=7)
568 fig.suptitle("按晋级重要性排序的 24 场第三轮比赛", y=1.01)
569 save_figure(fig, "fig_q3_advancement_and_risk")
570
571 fig, axes = plt.subplots(2, 2, figsize=(11.2, 7.1))
572 x = np.arange(len(daily)); labels = pd.to_datetime(daily["date"]).dt.strftime("%m-%d")
573 width = 0.24
574 for offset, resource, label, color in [
575     (-width, "high_broadcast", "高转播", COLORS["blue"]),
576     (0, "high_security", "高安保", COLORS["orange"]),
577     (width, "enhanced_transport", "强化交通", COLORS["teal"]),
578 ]:
579     axes[0, 0].bar(x + offset, daily[resource + "_used"], width=width, label=label, color=color)
580     axes[0, 0].scatter(x + offset, daily[resource + "_capacity"], marker="_", s=180, color="black",
581                       linewidth=1.2)
582 axes[0, 0].set_xticks(x, labels, rotation=45)
583 axes[0, 0].set_ylabel("使用量 (横线为容量)", title="(a) 每日三类高级资源")
584 axes[0, 0].legend(frameon=False, ncol=3, fontsize=8)
585 axes[0, 1].bar(x, 100 * daily["budget_ratio"], color=np.where(daily["budget_near_binding"], COLORS["red"],
586                                                                COLORS["gray"]))
587 axes[0, 1].axhline(100, color="black", linestyle="--")
588 axes[0, 1].set_xticks(x, labels, rotation=45)
589 axes[0, 1].set_ylabel("预算使用率 (%)", title="(b) 每日资源预算")
590 axes[1, 0].plot(convergence["simulations"], convergence["advancement_max_abs_diff_vs_20000"], "o-",
591                color=COLORS["blue"], label="晋级概率最大绝对差")
592 axes[1, 0].plot(convergence["simulations"], 1 - convergence["decision_match_rate_vs_20000"], "s-", color=
593                COLORS["orange"], label="动作不一致比例")
594 axes[1, 0].set_xscale("log")
595 axes[1, 0].set_ylabel("与 20000 次的差异", title="(c) Monte Carlo 收敛")
596 axes[1, 0].legend(frameon=False)
597 axes[1, 1].plot(convergence["simulations"], convergence["dynamic_total_net"], "o-", label="动态", color=
598                COLORS["blue"])
599 axes[1, 1].plot(convergence["simulations"], convergence["static_total_net"], "s-", label="静态固定动作",
600                color=COLORS["gray"])
601 axes[1, 1].set_xscale("log")
602 axes[1, 1].set_ylabel("总净值", title="(d) 总净值随模拟次数的变化")
603 axes[1, 1].legend(frameon=False)
604 save_figure(fig, "fig_q3_daily_resources_and_convergence")
605
606 return {
607     "convergence_seed": pc.P3_CONVERGENCE_SEED,
608     "convergence_sequence_relation": (
609         "same_seed_reinitialized_for_each_sample_size; not strict prefixes because "
610         "home/away Poisson arrays are generated in separate calls"
611     )
612 }

```

```

605     ),
606     "changed_case_ids": case_ids,
607     "convergence": [{key: value for key, value in run.items() if key not in ("advancement_probs", "signature")}] for run in runs],
608     "changed_cases": case_rows,
609     "daily_resources": daily_rows,
610     "binding_dates": {
611         resource: daily.loc[daily[resource + "_binding"], "date"].tolist()
612         for resource in ("high_broadcast", "high_security", "enhanced_transport")
613     },
614     "budget_near_binding_dates": daily.loc[daily["budget_near_binding"], "date"].tolist(),
615 }
616
617
618 def describe_mapping(frame: pd.DataFrame, label: str) -> list[dict]:
619     rows = []
620     for column, unit in [
621         ("match_mapping_distance", "标准化 Elo 距离"),
622         ("venue_capacity_difference", "人"), ("slot_hour_difference", "小时"),
623     ]:
624         values = frame[column]
625         rows.append({
626             "mapping": label, "metric": column, "unit": unit,
627             "mean": values.mean(), "median": values.median(), "p75": values.quantile(0.75),
628             "p90": values.quantile(0.90), "p95": values.quantile(0.95), "max": values.max(),
629         })
630     return rows
631
632
633 def analyze_problem4(data: dict[str, pd.DataFrame], formal_dir: Path, ctx: pc.P2Context, diagnostics: dict)
634     -> dict:
635     actual = pd.read_csv(formal_dir / "actual_schedule_2022.csv")
636     p1 = pd.read_csv(formal_dir / "result_1_match_prediction.csv")
637     p2 = pd.read_csv(formal_dir / "result_2_group_schedule.csv")
638     nearest = pc.map_actual_matches(actual, data, ctx, 0)
639     second = pc.map_actual_matches(actual, data, ctx, 1)
640     mapping_stats = pd.DataFrame(describe_mapping(nearest, "nearest") + describe_mapping(second, "second_nearest"))
641     write_table(mapping_stats, "q4_mapping_quality")
642     compare = nearest.merge(second, on="actual_match_id", suffixes=("_nearest", "_second"))
643     change_counts = {
644         "match": int((compare["mapped_match_id_nearest"] != compare["mapped_match_id_second"]).sum()),
645         "venue": int((compare["mapped_venue_id_nearest"] != compare["mapped_venue_id_second"]).sum()),
646         "slot": int((compare["mapped_slot_id_nearest"] != compare["mapped_slot_id_second"]).sum()),
647     }
648
649     actual_proxy = diagnostics["problem4_proxy"]["actual_nearest"]
650     second_proxy = diagnostics["problem4_proxy"]["actual_second_nearest"]
651     optimized_proxy = diagnostics["problem4_proxy"]["optimized"]
652     rng = np.random.default_rng(pc.P4_WEIGHT_SENSITIVITY_SEED)
653     weight_rows = []
654     for run in range(2000):
655         multipliers = rng.uniform(0.8, 1.2, len(pc.P2_WEIGHTS))
656         weights = {key: value * multipliers[i] for i, (key, value) in enumerate(pc.P2_WEIGHTS.items())}
657         scale = sum(abs(value) for value in weights.values())
658         weights = {key: value / scale for key, value in weights.items()}
659         actual_z = sum(weights[key] * actual_proxy["normalized"][key] for key in weights)
660         second_z = sum(weights[key] * second_proxy["normalized"][key] for key in weights)
661         optimized_z = sum(weights[key] * optimized_proxy["normalized"][key] for key in weights)
662         weight_rows.append({
663             "run": run, "actual_Z2": actual_z, "second_Z2": second_z, "optimized_Z2": optimized_z,
664             "gap_nearest": optimized_z - actual_z, "gap_second": optimized_z - second_z,
665             "reversal_nearest": optimized_z < actual_z, "reversal_second": optimized_z < second_z,

```

```

665     })
666     weight_sensitivity = pd.DataFrame(weight_rows)
667     weight_summary = pd.DataFrame([
668         "runs": len(weight_sensitivity),
669         "nearest_gap_min": weight_sensitivity["gap_nearest"].min(),
670         "nearest_gap_median": weight_sensitivity["gap_nearest"].median(),
671         "nearest_gap_max": weight_sensitivity["gap_nearest"].max(),
672         "nearest_reversal_count": int(weight_sensitivity["reversal_nearest"].sum()),
673         "second_gap_min": weight_sensitivity["gap_second"].min(),
674         "second_gap_median": weight_sensitivity["gap_second"].median(),
675         "second_gap_max": weight_sensitivity["gap_second"].max(),
676         "second_reversal_count": int(weight_sensitivity["reversal_second"].sum()),
677     ])
678     write_table(weight_summary, "q4_weight_sensitivity")
679
680     structure = diagnostics["problem4_structure"]
681     selected_structure = [
682         ("avg_travel_per_team_km", "队均旅行"), ("minimum_rest_hours", "最小休息"),
683         ("rest_hours_std", "休息标准差"), ("venue_active_day_utilization", "场馆活跃日利用率"),
684         ("prime_count_range", "黄金时段极差"),
685     ]
686     auxiliary = [
687         ("prime_time_share", "黄金时段占比"), ("capacity_match_ratio", "容量匹配率"),
688         ("expected_attendance_per_match", "题内预计场均观众"),
689     ]
690     proxy_keys = list(pc.P2_WEIGHTS)
691     fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.1))
692     labels = [label for _, label in selected_structure]
693     actual_structure_values = np.asarray([structure["actual"][key] for key, _ in selected_structure])
694     optimized_structure_values = np.asarray([structure["optimized"][key] for key, _ in selected_structure])
695     structure_max = np.maximum(actual_structure_values, optimized_structure_values)
696     y = np.arange(len(selected_structure))
697     axes[0].scatter(actual_structure_values / structure_max, y, label="2022 实际", color=COLORS["gray"])
698     axes[0].scatter(optimized_structure_values / structure_max, y, label="优化赛程", color=COLORS["teal"])
699     axes[0].set_yticks(y, labels)
700     axes[0].set(xlabel="组内相对值 (仅供作图)", title="(a) 直接结构事实")
701     axes[0].legend(frameon=False)
702     aux_labels = [label for _, label in auxiliary]
703     actual_aux = [structure["actual"][key] for key, _ in auxiliary]
704     optimized_aux = [structure["optimized"][key] for key, _ in auxiliary]
705     # Each auxiliary is scaled by its pair maximum only for display.
706     pair_max = np.maximum(actual_aux, optimized_aux)
707     y = np.arange(len(auxiliary))
708     axes[1].scatter(np.asarray(actual_aux) / pair_max, y, label="2022 映射", color=COLORS["gray"])
709     axes[1].scatter(np.asarray(optimized_aux) / pair_max, y, label="优化赛程", color=COLORS["blue"])
710     axes[1].set_yticks(y, aux_labels)
711     axes[1].set(xlabel="组内相对值 (仅供作图)", title="(b) 映射辅助指标")
712     axes[1].legend(frameon=False)
713     y = np.arange(len(proxy_keys))
714     axes[2].scatter([actual_proxy["normalized"][k] for k in proxy_keys], y, label="2022 最近邻", color=
715         COLORS["gray"])
715     axes[2].scatter([optimized_proxy["normalized"][k] for k in proxy_keys], y, label="优化赛程", color=
716         COLORS["blue"])
716     axes[2].set_yticks(y, proxy_keys)
717     axes[2].set(xlabel="题内统一归一化值", title="(c) 完整题内代理")
718     axes[2].legend(frameon=False)
719     save_figure(fig, "fig_q4_three_layer_comparison")
720
721     fig, axes = plt.subplots(2, 2, figsize=(11.0, 7.0))
722     axes[0, 0].hist(nearest["match_mapping_distance"], bins=12, color=COLORS["blue"], edgecolor="white")
723     axes[0, 0].set(xlabel="标准化 Elo 映射距离", ylabel="场次", title="(a) 比赛最近邻距离")
724     axes[0, 1].hist(nearest["venue_capacity_difference"], bins=12, color=COLORS["teal"], edgecolor="white")
725     axes[0, 1].set(xlabel="场馆容量绝对差 (人)", ylabel="场次", title="(b) 场馆容量映射")

```

```

726 axes[1, 0].hist(nearest["slot_hour_difference"], bins=np.arange(-0.25, nearest["slot_hour_difference"].
727 max() + 0.75, 0.5), color=COLORS["orange"], edgecolor="white")
728 axes[1, 0].set(xlabel="UTC 循环小时差", ylabel="场次", title="(c) 开球时刻映射")
729 actual_contrib = pd.Series(actual_proxy["weighted_contributions"])
730 optimized_contrib = pd.Series(optimized_proxy["weighted_contributions"])
731 x = np.arange(len(proxy_keys)); width = 0.36
732 axes[1, 1].bar(x - width / 2, actual_contrib[proxy_keys], width, label="2022 最近邻", color=COLORS["gray"])
733 axes[1, 1].bar(x + width / 2, optimized_contrib[proxy_keys], width, label="优化赛程", color=COLORS["blue"])
734 axes[1, 1].axhline(0, color="black", linewidth=0.7)
735 axes[1, 1].set_xticks(x, proxy_keys)
736 axes[1, 1].set(ylabel="加权贡献", title="(d) 代理综合值的贡献来源")
737 axes[1, 1].legend(frameon=False)
738 save_figure(fig, "fig_q4_mapping_quality_and_contributions")
739
740 fig, ax = plt.subplots(figsize=(8.4, 3.5))
741 ax.hist(weight_sensitivity["gap_nearest"], bins=30, color=COLORS["purple"], alpha=0.72, edgecolor="white",
742 label="最近邻")
743 ax.hist(weight_sensitivity["gap_second"], bins=30, color=COLORS["teal"], alpha=0.58, edgecolor="white",
744 label="第二近邻")
745 ax.axvline(0, color=COLORS["red"], linestyle="--", label="方向反转界线")
746 ax.set(xlabel="$Z_2^{opt}-Z_2^{2022}$", ylabel="权重扰动样本数", title="2000 组±20% 权重扰动下的代理综合值差")
747 ax.legend(frameon=False)
748 save_figure(fig, "fig_q4_weight_sensitivity")
749
750 return {
751     "weight_sensitivity_seed": pc.P4_WEIGHT_SENSITIVITY_SEED,
752     "weight_sensitivity_scope": "fixed schedules, mappings and indicators; re-evaluation only",
753     "mapping_quality": mapping_stats.to_dict(orient="records"),
754     "nearest_second_change_counts": change_counts,
755     "weight_sensitivity": weight_summary.iloc[0].to_dict(),
756 }
757
758 def sha256(path: Path) -> str:
759     digest = hashlib.sha256()
760     with path.open("rb") as handle:
761         for chunk in iter(lambda: handle.read(1024 * 1024), b""):
762             digest.update(chunk)
763     return digest.hexdigest()
764
765 def main() -> None:
766     global FIGURE_DIR, TABLE_DIR
767     args = parse_args()
768     formal_dir, generated_dir = resolve(args.output_dir), resolve(args.generated_dir)
769     FIGURE_DIR, TABLE_DIR = generated_dir / "figures", generated_dir / "tables"
770     FIGURE_DIR.mkdir(parents=True, exist_ok=True)
771     TABLE_DIR.mkdir(parents=True, exist_ok=True)
772     configure_plotting()
773     data = pc.read_data()
774     diagnostics = json.loads((formal_dir / "diagnostics.json").read_text(encoding="utf-8"))
775
776     draw_information_flow()
777     results = {
778         "random_seeds": pc.RANDOM_SEEDS,
779         "problem1": analyze_problem1(data),
780     }
781     results["problem2"], context = analyze_problem2(data, formal_dir, diagnostics, args.skip_p2_sensitivity)
782     results["problem3"] = analyze_problem3(data, formal_dir)
783     results["problem4"] = analyze_problem4(data, formal_dir, context, diagnostics)

```

```

783 generated_files = sorted(path for path in generated_dir.rglob("*") if path.is_file())
784 results["traceability"] = {
785     "formal_output_directory": str(formal_dir),
786     "formal_diagnostics_sha256": sha256(formal_dir / "diagnostics.json"),
787     "formal_csv_sha256": {path.name: sha256(path) for path in sorted(formal_dir.glob("*.csv"))},
788     "script": str(Path(__file__).relative_to(ROOT)),
789     "generated_files": [str(path.relative_to(ROOT)) for path in generated_files],
790 }
791 (generated_dir / "paper_analysis_results.json").write_text(
792     json.dumps(
793         results, ensure_ascii=False, indent=2,
794         default=lambda value: value.item() if isinstance(value, np.generic) else str(value),
795     ),
796     encoding="utf-8",
797 )
798 print(f"Paper analyses written to {generated_dir}")
799
800
801
802 if __name__ == "__main__":
803     main()

```